

Projektarbeit

lego::lab Karlsruhe: Schnittstellen zu externen Displays

im Studiengang
Informatik Master

vorgelegt von

Florian Kaiser
Matrikelnummer: 79728
kaf11018@h-ka.de

an der Hochschule Karlsruhe
am 03.10.2026

Prüfer: Prof. Dr. Uwe Haneke, Matthias Mruzek-Vering

ERKLÄRUNG

Hiermit erkläre ich, dass ich die vorliegende Arbeit eigenständig und ohne unerlaubte fremde Hilfe angefertigt habe. Textpassagen, die wörtlich oder dem Sinn nach auf Publikationen oder Vorträgen anderer Autoren beruhen, sind als solche kenntlich gemacht. Der Einsatz von KI-Werkzeugen erfolgte ausschließlich im Rahmen der nachstehenden Erklärung zur Nutzung von KI-Werkzeugen. Die Arbeit wurde bisher keiner anderen Prüfungsbehörde vorgelegt und auch noch nicht veröffentlicht.

Karlsruhe, 03.10.2026

Florian Kaiser

ERKLÄRUNG ZUR NUTZUNG VON KI-WERKZEUGEN

Bei der Erstellung dieser Arbeit wurden KI-Werkzeuge unterstützend eingesetzt, namentlich Claude (Anthropic), und zwar für Textentwürfe und sprachliche Überarbeitung des Fließtextes, für Teile des Programmcodes sowie für die Auswertungsskripte der Messungen. Nicht eingesetzt wurden sie für den Entwurf der Architektur, die Durchführung der Messungen und die Interpretation der Ergebnisse. Alle inhaltlichen und konzeptionellen Entscheidungen, die Durchführung und Auswertung, die Überprüfung sämtlicher Ergebnisse sowie die endgültige Fassung des Textes wurden von dem Autor selbst erstellt und geprüft.

Bei der Nutzung wurden die allgemein anerkannten Standards guter wissenschaftlicher Praxis sowie die einschlägigen Vorgaben des Fachbereichs zum Umgang mit KI-generierten Inhalten beachtet. Jede übernommene Aussage wurde eigenständig auf Plausibilität und Richtigkeit geprüft; wo eine Aussage nicht durch eine Quelle oder eine eigene Messung belegt ist, ist das im Text vermerkt. Es wurden keine personenbezogenen, urheberrechtlich geschützten oder vertraulichen Daten an die KI-Systeme übermittelt. Die eigenständige Leistung des Autors ist durchgängig gewahrt.

ZUSAMMENFASSUNG

Diese Arbeit untersucht, wie sich externe Peripherie an eine eingebettete Plattform anbinden lässt, deren Verbindung dafür nur eingeschränkt geeignet ist. Die betrachtete Verbindung ist anfragegetrieben, überträgt je Paket lediglich 16 Byte und steht beschränkten Laufzeitumgebungen zur Verfügung. Entworfen und umgesetzt wird eine Middleware, die entfernte Objekte über eine solche Verbindung nutzbar macht und die Eigenheiten der Übertragung vor der Anwendung verbirgt. Der Entwurf gliedert sich in fünf Schichten von der Busanbindung bis zur Anwendung und bündelt im Entwurf alles, was vom Übertragungsweg abhängt, in der untersten Schicht. Als Anwendungsfall dient die Anbindung eines Touchscreen-Displays an einen LEGO Education SPIKE Prime Hub über einen ESP32. Die Auswertung zeigt, dass die Dauer eines Aufrufs über alle gemessenen Konfigurationen hinweg allein von der Anzahl der Übertragungen abhängt, da ein Round Trip dort unabhängig von Format und Nachrichtenlänge konstant 114 ms kostet. Eine eigene Messreihe weist nach, dass davon 100 ms auf eine Wartezeit zwischen Schreiben und Lesen entfallen, die während der Entwicklung auf diesen Wert gesetzt wurde. Ein schlanker Header und ein binäres Serialisierungsformat halbieren die Anzahl der nötigen Übertragungen gegenüber der ersten Fassung. Drei der sechs Kommunikationsanforderungen werden vollständig erfüllt und drei teilweise. Die geforderte Antwortzeit von 100 ms ist nur über Reaktionen erreichbar, die die Anzeigeeinheit selbst ausführt, und der Speicherbedarf ist nur auf der Steuereinheit gemessen.

Inhaltsverzeichnis

Abbildungs- und Tabellenverzeichnis	VII
Abkürzungsverzeichnis	VIII
1 Einleitung	1
1.1 Problemstellung	1
1.2 Zielsetzung der Arbeit	1
1.3 Aufbau der Arbeit	2
2 Grundlagen und verwandte Arbeiten	3
2.1 Entfernter Prozeduraufruf und Middleware	3
2.2 Serialisierung von Nachrichten	3
2.3 Eingebettete Systeme und ihre Kommunikation	4
2.4 Einordnung und Auswahlkriterien	5
2.5 Etablierte Rahmenwerke für entfernte Aufrufe	5
2.6 Serialisierung für ressourcenarme Geräte	6
2.7 Protokolle für eingebettete Systeme und Sensornetze	6
2.8 Der betrachtete Bus und PUPRemote als Transportbasis	7
2.9 Abgrenzung	8
3 Systemanalyse und Anforderungen	10
3.1 Zielsystem und Hardwareumgebung	10
3.1.1 Der LEGO Education SPIKE Prime Hub als Ausgangsplattform	10
3.1.2 Notwendigkeit einer externen Erweiterung	10
3.2 Anforderungen an die Display-Integration	11
3.2.1 Anforderungen an ein externes Display	11
3.2.2 Anforderungen an einen externen Mikrocontroller	11
3.2.3 Resultierende Systemstruktur	11
3.3 Kommunikationsanforderungen	12
3.4 Einschränkungen der Plattform	12
3.4.1 Eingeschränkte Kommunikationsrichtung	13
3.4.2 Begrenzte Paketgröße	13
3.4.3 Beschränkte Laufzeitumgebung	13
3.4.4 Wechselwirkung der Einschränkungen	14
3.5 Verworfenen Alternativen	15
3.6 Verallgemeinerung des Problemraums	15
4 Architektur des Systems	17
4.1 Zielsetzung der Architektur	17
4.2 Schichtenmodell	17
4.2.1 Busanbindung	18
4.2.2 Transportschicht	18

4.2.3	Kommunikationsschicht	18
4.2.4	Objektmodell	19
4.2.5	Anwendung und Anzeige	19
4.3	Rollen im Entwurf	19
4.3.1	Interceptor	19
4.3.2	Dispatcher	20
4.3.3	Stellvertreter und Adapter	20
4.4	Kommunikationsmodell	20
4.5	Datenfluss im System	20
5	Entwurf von Transport- und Kommunikationsschicht	22
5.1	Motivation und Randbedingungen	22
5.2	Transportschicht	22
5.2.1	Interceptor-Abstraktion	23
5.2.2	Frame-Struktur	24
5.2.3	Nutzlastlänge und Formatunabhängigkeit	25
5.2.4	Kosten der Rahmung	25
5.2.5	Fragmentierung und Rekonstruktion	26
5.2.6	Serialisierung	27
5.2.7	Zuverlässigkeitsmechanismen	28
5.3	Kommunikationsschicht	29
5.3.1	Aufbau einer Aufrufnachricht	29
5.3.2	Zuordnung von Anfrage und Antwort	30
5.3.3	Verteilung eingehender Aufrufe	30
5.3.4	Anfragegetriebener Rückkanal	30
6	Referenzimplementierung	32
6.1	Laufzeitumgebung	32
6.2	Hardwareaufbau	33
6.2.1	Aufbau und Komponenten	33
6.2.2	Signalzuordnung und Aufbau der Adapterplatine	34
6.3	Busanbindung	34
6.4	Transportschicht	35
6.4.1	Frame	35
6.4.2	Message	37
6.4.3	Serialisierung	38
6.5	Kommunikationsschicht	39
6.5.1	Aufrufnachricht	39
6.5.2	Verteilung eingehender Aufrufe	41
6.5.3	Rückkanal	41
6.6	Objektmodell	42
6.6.1	Referenzen	43

6.6.2 Lebenszyklus	44
6.6.3 Veraltete Referenzen	44
6.6.4 Erweiterung um einen Objekttyp	45
6.7 Ansteuerung der Anzeige	45
6.8 Ablauf eines vollständigen Aufrufs	46
6.8.1 Zustände eines Aufrufs	47
6.9 Nebenläufigkeit und Fehlerbehandlung	48
6.9.1 Nebenläufigkeit	49
6.9.2 Fehlerbehandlung	49
7 Evaluation	52
7.1 Funktionaler Nachweis	52
7.2 Nachweis durch die Testumgebung	53
7.3 Übertragungsaufwand	54
7.4 Reaktionszeit einer Berührung	56
7.5 Ressourcenverbrauch	58
7.6 Bewertung des Hardwareaufbaus	58
7.7 Diskussion der Ergebnisse	59
8 Fazit und Ausblick	61
8.1 Limitierungen	61
8.1.1 Kommunikationsmodell	61
8.1.2 Latenz und Durchsatz	61
8.1.3 Abstraktionsgrenzen	62
8.2 Übertragbarkeit auf andere Verbindungsklassen	62
8.3 Mögliche Erweiterungen	63
8.3.1 Ergebnis mit der Bestätigung	63
8.3.2 Kürzere Wartezeit der Busanbindung	64
8.3.3 Ereignisgetriebener Rückkanal	65
8.3.4 Trennung der Busanbindung	66
8.3.5 Alternative Serialisierung	66
8.3.6 Integrierter Hardwareaufbau	66
8.3.7 Weitere Peripherieklassen	67
8.4 Zusammenfassung	67
Bibliografie	68
Anhang	71
Quelltext und Testumgebung	71

Abbildungs- und Tabellenverzeichnis

Abbildungen

Abbildung 1	Schichtenmodell der Architektur. Die gepunktete Linie verbindet die Gegenstücke beider Seiten.	17
Abbildung 2	Ablauf eines Aufrufs an der Interceptor-Schnittstelle	23
Abbildung 3	Klassendiagramm des Codecs	38
Abbildung 4	Klassen des Objektmodells auf beiden Seiten	43
Abbildung 5	Zustände eines Aufrufs auf dem Client	47
Abbildung 6	Zustände eines Aufrufs auf dem Server, je Nachrichtenkennung	48
Abbildung 7	Round Trips je Aufruf über der Textlänge, Median aus je 20 Aufrufen	54
Abbildung 8	Vorderseite (rechts) und Rückseite (links) der Adapterplatine als 3D-Rendering	58
Abbildung 9	Schaltplan der Adapterplatine	73

Tabellen

Tabelle I	Vergleich der betrachteten Ansätze entlang der Auswahlkriterien	8
Tabelle II	Gegenüberstellung der Kommunikationsanforderungen und der Einschränkungen der Plattform	14
Tabelle III	Zuordnung der Module zu den Schichten aus Kapitel 4	33
Tabelle IV	Aufbau des Frame-Headers	35
Tabelle V	Werte für die Art des Frames und ihre Bedeutung	36
Tabelle VI	Aufbau der Nutzlast je Ausprägung	39
Tabelle VII	Grenzen des Rückkanals	42
Tabelle VIII	Aktionen, die an ein Ereignis gebunden werden können	46
Tabelle IX	Fehlerfälle und ihre Behandlung	50
Tabelle X	Erfüllung der Kommunikationsanforderungen	53
Tabelle XI	Spanne der vier Konfigurationen über alle acht Textlängen, je 160 Aufrufe ...	55
Tabelle XII	Zeit von der Berührung bis zur sichtbaren Reaktion	56
Tabelle XIII	Erkannte Berührungen je Variante, gezählt auf beiden Geräten	57
Tabelle XIV	Aufrufe mit abgeholtem und mit mitgeliefertem Ergebnis, Mediane aus je 20 Aufrufen	64
Tabelle XV	Wirkung der Wartezeit, je Einstellung 160 Aufrufe über acht Textlängen, Dauer je Round Trip als Mittelwert	64
Tabelle XVI	Aufbau des Quelltextes	71

Code-Listings

Listing 1	Registrierung des einzigen Kommandos	34
Listing 2	Funktion zur Erzeugung eines Frames aus Bytes	36
Listing 3	Funktion zur Umwandlung eines Frames in Bytes	36
Listing 4	Funktion zur Erzeugung von Frames aus einer Nachricht	37
Listing 5	Berechnung der Methodenkennung aus dem Methodennamen	40
Listing 6	Funktion zur Registrierung einer Methode	40
Listing 7	Beantwortung der beiden Anfragen auf dem Server	41
Listing 8	Auflösung einer Referenz in der Registratur	43
Listing 9	Beispielprogramm auf dem Hub	72

Abkürzungsverzeichnis

ARQ – Automatic Repeat reQuest
BLE – Bluetooth Low Energy
CBOR – Concise Binary Object Representation
CoAP – Constrained Application Protocol
CORBA – Common Object Request Broker Architecture
gRPC – gRPC Remote Procedure Call
HTTP – Hypertext Transfer Protocol
I2C – Inter-Integrated Circuit
JSON – JavaScript Object Notation
LED – Light Emitting Diode
LPF2 – LEGO Power Functions 2.0
LVGL – Light and Versatile Graphics Library
MQTT – Message Queuing Telemetry Transport
MTU – Maximum Transmission Unit
ONC RPC – Open Network Computing Remote Procedure Call
OSI – Open Systems Interconnection
PUP – LEGO Powered Up
REST – Representational State Transfer
RFC – Request for Comments
RMI – Remote Method Invocation
RPC – Remote Procedure Call
RTU – Remote Terminal Unit
SPI – Serial Peripheral Interface
SRAM – Static Random Access Memory
TCP – Transmission Control Protocol
TFT – Thin-Film-Transistor
UART – Universal Asynchronous Receiver Transmitter
UDP – User Datagram Protocol
XML – Extensible Markup Language

1 Einleitung

Die Vermittlung von Informatik- und Technikenntnissen an Kinder und Jugendliche gewinnt zunehmend an Bedeutung. Plattformen wie die Education-Systeme von LEGO bieten hierfür einen niedrigschwelligen Einstieg in Programmierung und Robotik. Durch vereinfachte Hardware und abstrahierte Softwareumgebungen sind erste praktische Erfahrungen auch ohne umfangreiche Vorkenntnisse möglich.

Mit zunehmender Nutzung entsteht häufig der Wunsch, solche Systeme über ihren ursprünglichen Funktionsumfang hinaus zu erweitern. Insbesondere zusätzliche Hardwarekomponenten eröffnen neue Möglichkeiten, etwa bei der Visualisierung und der Interaktion. Displays mit Touch-Funktionalität sind in diesem Zusammenhang eine naheliegende Erweiterung, da sie eine direkte und intuitive Bedienung ermöglichen.

Die Erweiterung bestehender Embedded-Systeme um externe Komponenten ist allerdings nicht trivial. Neben der reinen Hardwareanbindung spielen vor allem die Kommunikation und die Integration in die bestehende Software eine wichtige Rolle. Die beteiligten Komponenten müssen koordiniert zusammenarbeiten, wobei die vorhandenen Schnittstellen und Laufzeitumgebungen oft Einschränkungen mit sich bringen.

Vor diesem Hintergrund beschäftigt sich die vorliegende Arbeit mit der Frage, wie sich externe Peripherie an ein solches System anbinden lässt, wenn die verfügbare Verbindung dafür nur eingeschränkt geeignet ist. Im Mittelpunkt steht dabei weniger die Ansteuerung eines einzelnen Geräts als eine Softwareschicht, die zwischen Anwendung und Verbindung vermittelt und die Eigenheiten der Übertragung vor der Anwendung verbirgt. Als Anwendungsfall dient die Anbindung eines **Thin-Film-Transistor (TFT)**-Touchscreen-Displays, an dem der Entwurf entwickelt, umgesetzt und bewertet wird.

1.1 Problemstellung

Die Anbindung zusätzlicher Peripherie an eine solche Plattform wird durch drei technische Einschränkungen erschwert, die gemeinsam auftreten und sich gegenseitig verstärken.

Erstens ist die Verbindung zwischen den Systemkomponenten anfragegetrieben. Sie ist zwar in beide Richtungen nutzbar, die übergeordnete Seite erfährt von Daten der untergeordneten aber erst, wenn sie selbst nachfragt. Zweitens ist die je Paket übertragbare Datenmenge sehr klein und fest vorgegeben, sodass Nachrichten üblicher Größe nicht in ein einzelnes Paket passen. Drittens stehen stark beschränkte Laufzeitumgebungen zur Verfügung, die zumindest auf einer Seite weder einen vollständigen Netzwerkstack noch die üblichen Mittel nebenläufiger Programmierung bereitstellen.

Diese Rahmenbedingungen stehen im Gegensatz zu den Anforderungen interaktiver Benutzeroberflächen, die eine flexible Übertragung strukturierter Daten und eine zeitnahe Rückmeldung von Benutzereingaben voraussetzen. Gerade die Verarbeitung von Eingaben und die Übertragung zusammengesetzter Zustände sind unter diesen Bedingungen eine Herausforderung.

Daraus ergibt sich die zentrale Problemstellung dieser Arbeit: Wie lässt sich eine Middleware entwerfen, die entfernte Objekte über eine anfragegetriebene Verbindung mit sehr kleiner Paketgröße auf beschränkten Laufzeitumgebungen transparent nutzbar macht?

Die Frage ist dabei nicht an eine bestimmte Plattform gebunden, denn die Kombination aus anfragegetriebener Übertragung, kleiner Paketgröße und beschränkter Laufzeitumgebung tritt in verschiedenen Ausprägungen auf, wie Abschnitt 3.6 zeigt.

1.2 Zielsetzung der Arbeit

Ziel dieser Arbeit ist der Entwurf einer Middleware, die entfernte Objekte über eine Verbindung der beschriebenen Art transparent nutzbar macht.

Der Entwurf soll die drei genannten Einschränkungen auf wenige Schichten eingrenzen, sodass sie oberhalb davon nicht mehr sichtbar sind. Dazu gehören:

- die Zerlegung von Nachrichten in Pakete und deren Rahmung,
- ein anfragegetriebener Rückkanal für Ergebnisse und Ereignisse
- sowie eine Abstraktion, die entfernte Objekte wie lokale erscheinen lässt.

Ein weiterer Schwerpunkt liegt darauf, den Entwurf vom konkreten Übertragungsweg zu lösen, damit er auf andere Verbindungen derselben Klasse übertragbar bleibt.

Die Bewertung stützt sich auf drei Kriterien. Das erste ist der funktionale Nachweis, dass sich eine interaktive Oberfläche über die entwickelte Abstraktion umsetzen lässt. Das zweite ist der Übertragungsaufwand je Aufruf, gemessen in benötigten Paketen und in der daraus folgenden Verzögerung. Das dritte ist der Ressourcenverbrauch auf beiden Einheiten. Die Laufzeitumgebung ist auf der Steuereinheit am stärksten eingeschränkt, der Arbeitsspeicher dagegen auf der Anzeigeeinheit am knappsten.

1.3 Aufbau der Arbeit

Die vorliegende Arbeit ist wie folgt aufgebaut.

Kapitel 2 führt die Grundlagen zum entfernten Prozeduraufruf, zur Serialisierung und zur Kommunikation eingebetteter Systeme ein. Anschließend ordnet es die Arbeit in bestehende Ansätze ein und benennt die Lücke, die sie schließt.

Kapitel 3 analysiert das Zielsystem und leitet die Anforderungen an die Display-Integration und an die Kommunikation ab. Den Anforderungen werden die Einschränkungen der Plattform gegenübergestellt, bevor das Kapitel mit einer Verallgemeinerung des Problemraums schließt.

Kapitel 4 beschreibt die Architektur mit ihrem Schichtenmodell, den wiederkehrenden Rollen im Entwurf und dem Kommunikationsmodell. Kapitel 5 vertieft den Entwurf der Transport- und der Kommunikationsschicht, wobei beide Kapitel auf der konzeptionellen Ebene bleiben.

Kapitel 6 stellt die Referenzimplementierung vor und bildet den Entwurf auf die konkrete Hardware, die Laufzeitumgebungen und den Bus ab. Kapitel 7 bewertet die Lösung hinsichtlich Funktionalität, Übertragungsaufwand und Ressourcenverbrauch. Kapitel 8 diskutiert die Grenzen des Ansatzes und seine Übertragbarkeit auf andere Verbindungen und fasst die Ergebnisse zusammen. Der Anhang enthält das Beispielprogramm der Steuereinheit und den Schaltplan der Adapterplatine.

2 Grundlagen und verwandte Arbeiten

Die in dieser Arbeit entwickelte Middleware baut auf bekannten Ansätzen für entfernte Aufrufe und für die Kommunikation in verteilten und eingebetteten Systemen auf. Die ersten drei Abschnitte führen die Begriffe ein, die in den Entwurfskapiteln benötigt werden. Anschließend wird die eigene Lösung in dieses Umfeld eingeordnet. Da das Feld sehr breit ist, werden ausgewählte Vertreter betrachtet und anhand einheitlicher Kriterien verglichen. Ziel ist es, die Lücke zu benennen, die die vorliegende Arbeit schließt.

2.1 Entfernter Prozeduraufruf und Middleware

Bei einem entfernten Prozeduraufruf, englisch [Remote Procedure Call \(RPC\)](#), ruft ein Programm eine Prozedur auf, die auf einem anderen Rechner ausgeführt wird. Im Quelltext sieht dieser Aufruf wie ein gewöhnlicher lokaler Aufruf aus. Die Idee wurde von Birrell und Nelson beschrieben und bildet seitdem die Grundlage der meisten Middleware-Systeme [1]. Der Aufruf wird auf der aufrufenden Seite von einem Stellvertreter entgegengenommen, der dieselbe Signatur besitzt wie die eigentliche Prozedur, sie aber nicht ausführt. Er verpackt den Namen der Prozedur und die Argumente zu einer Nachricht und übergibt sie dem Transport. Auf der ausführenden Seite nimmt ein Gegenstück die Nachricht entgegen, liest die Argumente aus, ruft die eigentliche Prozedur auf und schickt deren Rückgabewert zurück. Dieses Gegenstück wird klassisch als Skelett bezeichnet. Das Umwandeln von Argumenten und Rückgabewerten in eine übertragbare Form heißt Marshalling.

Das Ziel dieses Aufbaus ist die Aufrufstransparenz. Der Aufrufer soll nicht wissen müssen, ob eine Prozedur lokal oder entfernt ausgeführt wird. Vollständig erreichen lässt sich das allerdings nicht. Ein entfernter Aufruf dauert um Größenordnungen länger als ein lokaler, er kann teilweise fehlschlagen, während der Aufrufer weiterläuft, und Zeiger in den lokalen Speicher haben auf der Gegenseite keine Bedeutung. Waldo et al. zeigen, dass sich diese Unterschiede nicht dauerhaft verbergen lassen und in der Schnittstelle sichtbar bleiben sollten [2]. Transparenz wird deshalb meist als Aussage über die Aufrufsyntax verstanden und weniger als Zusicherung gleicher Laufzeiteigenschaften.

Die Schicht, die Stellvertreter, Marshalling und Transport bereitstellt, wird als Middleware bezeichnet. Sie liegt zwischen der Anwendung und der Verbindung, bietet der Anwendung eine einheitliche Schnittstelle an und verbirgt die Eigenheiten der Übertragung [3]. Ein häufig verwendeter Aufbau für solche Systeme ist das Broker-Muster. Es beschreibt den Stellvertreter auf der Client-Seite, den vermittelnden Broker und das Gegenstück auf der Server-Seite als eigenständige Rollen [4].

Der ursprüngliche Ansatz kennt nur einen festen Satz von Prozeduren. Objektorientierte Middleware erweitert ihn auf entfernte Objekte, die zur Laufzeit entstehen und über eine Referenz angesprochen werden. Der Stellvertreter entspricht dann dem Proxy-Muster [5]. Er implementiert dieselbe Schnittstelle wie das entfernte Objekt, führt dessen Referenz mit und gibt jeden Aufruf zusammen mit dieser Referenz weiter. Bekannte Vertreter dieses Ansatzes sind [Common Object Request Broker Architecture \(CORBA\)](#) [6] und Java [Remote Method Invocation \(RMI\)](#) [7]. Die Aufteilung in einen Stellvertreter auf der aufrufenden und ein Gegenstück auf der ausführenden Seite findet sich in Kapitel 4 wieder, wo das Gegenstück als Adapter bezeichnet wird.

2.2 Serialisierung von Nachrichten

Eine Nachricht liegt im Programm als strukturierter Wert vor, etwa als Objekt, als Liste oder als Abbildung. Übertragen wird dagegen eine Folge von Bytes. Die interne Darstellung eines Werts im Arbeitsspeicher lässt sich dafür nicht direkt verwenden, da sie Verweise enthält und von Wortbreite, Byte-Reihenfolge und Laufzeitumgebung abhängt. Serialisierung bezeichnet die Umwandlung eines strukturierten Werts in eine Bytefolge, Deserialisierung den umgekehrten Weg.

Die gebräuchlichen Formate unterscheiden sich vor allem in zwei Punkten. Der erste ist die Kodierung. Textbasierte Formate wie [JavaScript Object Notation \(JSON\)](#) oder [Extensible Markup Language \(XML\)](#) sind ohne Werkzeuge lesbar und erleichtern damit die Fehlersuche, benötigen aber mehr Platz, weil Schlüssel und Strukturzeichen als Klartext mit übertragen werden. Binäre Formate stellen dieselben Werte deutlich kürzer dar, ihre Ausgabe ist ohne Kenntnis des Formats jedoch nicht mehr zu deuten. Der zweite Punkt ist die Bindung an ein Schema. Schemafreie Formate führen die Struktur in der Nachricht selbst mit, sodass Sender und Empfänger nichts vorab vereinbaren müssen. Schemagebundene Formate setzen eine geteilte Beschreibung der Datenstruktur voraus und sparen dafür die Feldnamen in der Nachricht ein. Welche Vertreter für ein Gerät mit sehr kleinen Paketen in Frage kommen, behandelt Abschnitt 2.6.

Für diese Arbeit ist eine weitere Eigenschaft binärer Formate wichtig, denn ihre Ausgabe kann jedes beliebige Byte enthalten, auch das Nullbyte. Ein Empfänger kann die Länge der Nutzlast deshalb nicht aus dem Inhalt ableiten, etwa indem er Nullbytes am Ende eines Pakets als Füllung abschneidet, sodass die Rahmung die Länge ausdrücklich mitführen muss. Bei einem textbasierten Format fällt dieser Unterschied nicht auf, da dessen Ausgabe kein Nullbyte enthält, und er zeigt sich erst beim Wechsel auf eine binäre Kodierung. Abschnitt 5.2.3 greift das auf.

2.3 Eingebettete Systeme und ihre Kommunikation

Eingebettete Systeme verfügen über erheblich weniger Speicher und Rechenleistung als ein Arbeitsplatzrechner. [Request for Comments \(RFC\) 7228](#) fasst solche Geräte unter dem Begriff *constrained nodes* zusammen und ordnet sie nach verfügbarem Arbeits- und Programmspeicher in Klassen ein [8]. Die betrachteten Größenordnungen liegen bei einigen zehn bis einigen hundert Kilobyte Arbeitsspeicher. In diesem Rahmen werden Entwurfsentscheidungen wichtig, die auf einem gewöhnlichen Rechner kaum eine Rolle spielen.

Auf solchen Geräten kommen angepasste Laufzeitumgebungen zum Einsatz, beispielsweise MicroPython als eine für Mikrocontroller ausgelegte Implementierung von Python [9]. Der Sprachkern bleibt dabei weitgehend erhalten, während die Standardbibliothek stark verkleinert ist und ihr Umfang vom jeweiligen Port abhängt. Auf stark beschränkter Hardware fehlen zudem häufig ein vollständiger Netzwerkstack und die üblichen Mittel nebenläufiger Programmierung. Das ist kein Sonderfall einer einzelnen Firmware, sondern eine verbreitete Entwurfsentscheidung. Betriebssysteme für solche Geräte verzichten bewusst auf Threads je Aufgabe und arbeiten ereignisgetrieben, weil ein eigener Stack je Thread auf einem Gerät mit wenigen Kilobyte Arbeitsspeicher zu teuer ist [10], [11]. Ohne Netzwerkstack stehen auch die darauf aufbauenden Abstraktionen von Sockets bis zu fertigen Protokollbibliotheken nicht zur Verfügung, und ohne Nebenläufigkeit gibt es keinen Hintergrundprozess, der Daten entgegennimmt, während die Anwendung weiterläuft. Hinzu kommt der kleine Heap, bei dem jede Zuweisung den Druck auf die Speicherbereinigung erhöht, weshalb Puffer nach Möglichkeit vorab angelegt und wiederverwendet werden.

Die Verbindung zwischen einem Mikrocontroller und seiner Peripherie erfolgt über Bussysteme, die für kurze Strecken und wenige Leitungen ausgelegt sind. Viele von ihnen arbeiten nach dem Master-Slave-Prinzip. Eine Seite, der Master, stößt jede Übertragung an, die andere Seite antwortet ausschließlich. [Inter-Integrated Circuit \(I2C\)](#) [12] und [Serial Peripheral Interface \(SPI\)](#) [13] sind verbreitete Vertreter, auf Feldebene folgt Modbus im [Remote Terminal Unit \(RTU\)](#)-Modus demselben Muster [14]. Ein Slave, bei dem Daten anfallen, kann diese nicht von sich aus schicken. Er hält sie bereit, bis der Master sie abfragt. Dieses wiederholte Abfragen wird als Polling bezeichnet. Wie schnell ein Ereignis auf der Slave-Seite beim Master ankommt, hängt damit vom Abfragetakt des Masters ab und nicht vom Zeitpunkt, an dem es entstanden ist.

Die je Übertragung nutzbare Datenmenge ist auf solchen Bussen klein und häufig fest vorgegeben. Die größte in einem Paket übertragbare Nutzlast wird als [Maximum Transmission Unit \(MTU\)](#) bezeichnet. Ist eine Nachricht länger, muss sie in mehrere Pakete zerlegt werden. Damit der Empfänger sie wieder zusammensetzen kann, trägt jedes Paket zusätzliche Steuer-

informationen, die üblicherweise als Header vorangestellt werden. Diese Steuerinformationen gehen von der ohnehin knappen Nutzlast ab, sodass ihre Größe bei kleinen Paketen unmittelbar auf die Anzahl der nötigen Übertragungen durchschlägt. Wie sich diese Eigenschaften am betrachteten Zielsystem konkret darstellen, zeigt Abschnitt 3.4.

2.4 Einordnung und Auswahlkriterien

Für den Vergleich werden Kriterien herangezogen, die sich direkt aus den Rahmenbedingungen des Zielsystems ergeben. Das Zielsystem nutzt einen Master-Slave-Bus mit sehr kleinen, festen Paketen und läuft in stark eingeschränkten Laufzeitumgebungen.

Folgende Achsen werden betrachtet:

- **Abstraktionsgrad.** Bietet der Ansatz nur den Transport von Nachrichten, einen entfernten Prozeduraufruf oder darüber hinaus die Abstraktion entfernter Objekte?
- **Serialisierung.** Werden Daten textbasiert oder binär kodiert, und ist dafür ein vorab geteiltes Schema nötig?
- **Transportbindung.** Ist der Ansatz an einen konkreten Übertragungsweg gebunden oder von diesem unabhängig?
- **Fragmentierung.** Bringt der Ansatz Mechanismen für die Rahmung und das Zerlegen großer Nachrichten in kleine Pakete mit?
- **Eignung für einen Master-Slave-Bus.** Setzt der Ansatz voraus, dass die untergeordnete Seite eigenständig zustellen kann, oder kommt er mit einem rein anfragegetriebenen Kanal aus?
- **Ressourcenbedarf.** Lässt sich der Ansatz innerhalb der Grenzen von MicroPython und Pybricks betreiben?

Die betrachteten Ansätze liegen dabei nicht alle auf derselben Ebene, denn einige sind vollständige Rahmenwerke, andere beschränken sich auf die Beschreibung eines Nachrichtenformats, und wieder andere behandeln lediglich die Kodierung von Daten oder die Anbindung an einen bestimmten Bus. Ein Vergleich, der diese Unterschiede außer Acht lässt, stellt somit Ansätze nebeneinander, die sich gegenseitig gar nicht ersetzen können. Im abschließenden Vergleich wird die Ebene deshalb in einer eigenen Spalte ausgewiesen.

Diese Kriterien strukturieren die folgende Betrachtung und münden in den abschließenden Vergleich in Tabelle I.

2.5 Etablierte Rahmenwerke für entfernte Aufrufe

Entfernte Prozeduraufrufe wurden früh standardisiert. Der [Open Network Computing Remote Procedure Call \(ONC RPC\)](#) [15] legt ein Protokoll und mit der External Data Representation ein eigenes Serialisierungsformat fest [16]. Der Ansatz ist auf leistungsfähigere Netzwerke ausgelegt und nicht für Kanäle mit Paketgrößen im Bereich weniger Byte gedacht.

JSON-RPC verfolgt eine deutlich schlankere Idee [17], da die Spezifikation ausschließlich das Format der Anfrage- und Antwortnachrichten auf Basis von [JSON](#) beschreibt und den Transport bewusst der Anwendung überlässt. Diese Trennung ist dem hier verfolgten Ansatz ähnlich, allerdings setzt JSON-RPC voraus, dass eine darunterliegende Schicht vollständige Nachrichten zustellen kann. Es definiert weder eine Rahmung noch eine Fragmentierung für sehr kleine Pakete und sieht keinen Mechanismus für einen anfragegetriebenen Rückkanal auf einem Master-Slave-Bus vor.

Am anderen Ende des Spektrums steht [gRPC Remote Procedure Call \(gRPC\)](#) [18]. Das Rahmenwerk verbindet Protocol Buffers als schemagebundene Serialisierung mit [Hypertext Transfer Protocol \(HTTP\)/2](#) als Transport und unterstützt unter anderem Datenströme in beide Richtungen. Diese Leistungsfähigkeit erfordert jedoch einen vollständigen [HTTP/2](#)-Stack und setzt eine bidirektionale Verbindung voraus. Für den betrachteten LEGO-Bus und die eingesetzten Laufzeitumgebungen ist [gRPC](#) zu schwergewichtig und zu stark an seinen Transport gebunden.

Näher an der hier betrachteten Geräteklasse liegen zwei jüngere Ansätze. eRPC [19] überträgt entfernte Aufrufe zwischen Mikrocontrollern über [Universal Asynchronous Receiver Transmitter \(UART\)](#), [SPI](#) oder [I2C](#) und bringt dafür eine eigene Rahmung mit. Der Aufruf wird allerdings aus einer Schnittstellenbeschreibung erzeugt, das heißt die aufrufbaren Funktionen stehen zur Übersetzungszeit fest, und ein Objektmodell mit zur Laufzeit erzeugten Referenzen ist nicht vorgesehen. DDS-XRCE [20] richtet sich an sehr kleine Geräte, arbeitet über einen vermittelnden Agenten und fragmentiert dabei auch über serielle Verbindungen. Eingesetzt wird es etwa von micro-ROS, das ROS 2 auf Mikrocontroller überträgt und dort einen in C geschriebenen Client über einen Agenten auf einem leistungsfähigeren Rechner anbindet [21]. Der Client stößt jede Übertragung an, was dem hier betrachteten Rückkanal entspricht. Beiden gemeinsam ist, dass die Schnittstelle vorab beschrieben und daraus Code erzeugt wird. Was aufgerufen werden kann, steht damit vor dem Start fest.

Zusammengefasst setzen die etablierten Rahmenwerke entweder einen leistungsfähigen Transport voraus oder standardisieren nur die Nachrichtenebene. Rahmung, Fragmentierung und der Rückkanal bleiben dann offen, und genau diese darunterliegende Schicht behandelt die vorliegende Arbeit.

2.6 Serialisierung für ressourcenarme Geräte

Da die Serialisierung die Nachrichtengröße und damit die Anzahl der nötigen Pakete bestimmt, ist sie für ein System mit kleinen Paketen besonders relevant. [JSON](#) ist schemafrei und gut lesbar, erzeugt aber durch Klartextschlüssel und Strukturzeichen vergleichsweise große Ausgaben [22]. Binäre und schemafreie Formate wie [MessagePack](#) [23] und die [Concise Binary Object Representation \(CBOR\)](#) [24], [25] erreichen bei gleichem Datenmodell deutlich kompaktere Darstellungen. Schemagebundene Formate wie [Protocol Buffers](#) gehen noch einen Schritt weiter und setzen eine geteilte Beschreibung der Datenstruktur voraus [26]. Für sehr kleine Geräte existiert mit [Nanopb](#) [27] eine ressourcenschonende Implementierung dieses Ansatzes. Welches dieser Formate die Referenzimplementierung verwendet, warum sie zunächst ein anderes benutzte und was der Wechsel gekostet hat, beschreibt Abschnitt 6.4.3.

2.7 Protokolle für eingebettete Systeme und Sensornetze

Neben den allgemeinen Rahmenwerken existieren Protokolle, die näher an der betrachteten Domäne liegen. [Firmata](#) [28] ermöglicht die Steuerung eines Mikrocontrollers von einem übergeordneten Rechner aus und überträgt dazu einen festgelegten Satz von Befehlen über eine serielle Verbindung. Die Aufteilung in einen steuernden Rechner und ein reagierendes Gerät ähnelt der hier vorliegenden Situation. [Firmata](#) beschränkt sich allerdings auf einen vordefinierten Befehlsumfang und bietet weder einen allgemeinen entfernten Aufruf noch eine Abstraktion entfernter Objekte.

[Message Queuing Telemetry Transport \(MQTT\)](#) [29] ist auf ressourcenarme Geräte und kleine Pakete ausgelegt und kommt ohne eine durchgehende [Transmission Control Protocol \(TCP\)](#)-Verbindung aus. Es folgt jedoch einem Modell aus Veröffentlichen und Abonnieren, benötigt ein Gateway sowie einen Vermittler und setzt voraus, dass das Gerät eigenständig veröffentlichen kann. Auf einem Bus, auf dem die übergeordnete Seite von Daten erst beim Nachfragen erfährt, ist dieses Modell nicht unmittelbar anwendbar.

Näher am hier betrachteten Problem liegen Protokolle, die eine Nachricht über mehrere kleine Pakete hinweg übertragen und dabei ohne eine darunterliegende Transportschicht auskommen. Das Transportprotokoll [ISO 15765-2](#), im Fahrzeugbereich als [ISO-TP](#) bekannt, überträgt Nachrichten von bis zu 4095 Byte über CAN-Rahmen mit acht Byte Nutzlast [30]. Es nummeriert die Folgerahmen fortlaufend und lässt den Empfänger im Flusskontrollrahmen angeben, wie viele Rahmen er am Stück annimmt. Die Aufgabenstellung entspricht damit weitgehend der Rahmung und Fragmentierung dieser Arbeit, allerdings bleibt [ISO-TP](#) auf den Transport beschränkt und kennt weder Aufrufe noch entfernte Objekte.

Ein ähnliches Vorgehen findet sich in CANopen, wo das Service Data Object Datensätze segmentiert überträgt, die größer sind als ein einzelner CAN-Rahmen [31]. Dort wird zudem jeder Segmentabschnitt einzeln bestätigt, was dem Stop-and-Wait dieser Arbeit entspricht. Die adressierten Daten sind jedoch Einträge eines vorab festgelegten Objektverzeichnisses und keine zur Laufzeit erzeugten Objekte mit eigenen Methoden.

Im Umfeld ressourcenarmer Geräte löst [Constrained Application Protocol \(CoAP\)](#) dieselbe Aufgabe auf Anwendungsebene [32]. Der Block-wise Transfer aus RFC 7959 zerlegt eine Repräsentation in nummerierte Blöcke, deren Größe zwischen 16 und 1024 Byte ausgehandelt wird, und überträgt jeden Block mit einer eigenen Anfrage des Clients [33]. Damit ist der Ansatz dem hier entwickelten Rückkanal sehr ähnlich, denn auch dort fordert der Client jeden Block einzeln an. Die kleinste vorgesehene Blockgröße liegt allerdings bereits über der Nutzlast, die der betrachtete Bus je Paket zulässt, und [CoAP](#) setzt einen Datagrammdienst wie [User Datagram Protocol \(UDP\)](#) voraus.

Am nächsten an der Aufgabe dieser Arbeit liegen serielle Displays mit eigener Oberflächenlogik, etwa die Module von Nextion [34]. Seiten und Bedienelemente werden dort vorab in einem Editor entworfen und auf dem Display abgelegt. Der Host spricht sie anschließend über Textbefehle mit ihrem Namen an, wobei jeder Befehl mit drei Bytes `0xFF` abschließt. Auf Ereignisse reagiert das Display mit eigenem Code, der auch Bedingungen und Schleifen enthalten darf, sich aber nicht über die serielle Verbindung übertragen lässt. Das entspricht dem Objektmodell zusammen mit den Bindings dieser Arbeit, allerdings mit einer festen, herstellereigenen Befehlssprache und einer Oberfläche, die nicht zur Laufzeit entsteht, sondern vorab feststeht.

Diese Protokolle liegen nah an der betrachteten Domäne, lösen aber jeweils einen Teil der Aufgabe. Sie zielen auf Gerätesteuerung mit festem Befehlssatz, auf vorab entworfene Oberflächen, auf nachrichtenbasiertes Veröffentlichen und Abonnieren oder auf die segmentierte Übertragung von Daten. Einen allgemeinen, objektorientierten entfernten Aufruf über einen anfragegetriebenen Kanal mit Paketen von wenigen Byte bietet keines von ihnen.

2.8 Der betrachtete Bus und PUPRemote als Transportbasis

Der Hub und die angeschlossenen Geräte verkehren über [LEGO Power Functions 2.0 \(LPF2\)](#), ein Protokoll, das auf einer [UART](#)-Verbindung über die Sensorports aufsetzt [35]. Der Hub ist dabei der Master. Ein Gerät meldet sich beim Anstecken mit den Modi an, die es unterstützt. Jeder Modus beschreibt einen Kanal mit festem Namen, fester Länge und festem Datentyp. Der Hub wählt einen dieser Modi aus und liest dessen Wert oder schreibt einen Wert hinein.

PUPRemote [36] nimmt in diesem Umfeld eine Sonderstellung ein, da es in dieser Arbeit als Baustein der eigenen Lösung eingesetzt wird. Die Bibliothek erschließt den [LEGO Powered Up \(PUP\)](#)- und [LPF2](#)-Bus und stellt darauf benannte Aufrufe bereit. Jeder Aufruf belegt einen eigenen Modus des Busses und ist über feste Formatangaben für beide Richtungen beschrieben. Der Hub schreibt die Argumente in den zugehörigen Modus und liest anschließend den Rückgabewert aus demselben Modus zurück. Mit PUPRemote werden auf beiden Seiten Befehle registriert, die anschließend vom Master aufgerufen werden können. Die Länge der Befehlsnamen, der Argumente und der Rückgabewerte ist dabei durch die festen Formatangaben begrenzt. PUPRemote besitzt damit durchaus eine eigene Aufrufsemantik und ist nicht auf reinen Datentransport beschränkt.

Diese Semantik trägt jedoch nur so weit, wie sich ein Aufruf auf einen vorab registrierten Modus und ein einzelnes Paket fester Größe abbilden lässt. Die Anzahl der Modi ist durch den Bus begrenzt, die Formate stehen zur Übersetzungszeit fest, und für Nachrichten, die größer sind als ein Paket, ist kein Mechanismus vorgesehen. Eine Middleware, die beliebige Methoden auf einer zur Laufzeit wachsenden Menge entfernter Objekte anbieten soll, kann daher nicht einen Modus je Aufruf vergeben.

Die vorliegende Arbeit nutzt PUPRemote deshalb nur als Busanbindung. Registriert wird ein einziges Kommando, dessen Nutzlast aus Sicht von PUPRemote ein Block fester Länge

ohne Struktur ist. Die Zerlegung von Nachrichten in Pakete, deren Rahmung, die Zuordnung von Antworten zu Anfragen, der anfragegetriebene Rückkanal sowie das Objektmodell aus Stellvertretern und serverseitigen Adaptern liegen vollständig darüber. PUPRemote übernimmt damit die Rolle der Busanbindung und kann durch andere Bibliotheken ersetzt werden, die denselben Austausch von Blöcken leisten.

2.9 Abgrenzung

Tabelle I fasst die betrachteten Ansätze entlang der eingangs definierten Kriterien zusammen.

Tabelle I: Vergleich der betrachteten Ansätze entlang der Auswahlkriterien

Ansatz	Ebene	Abstraktionsgrad	Serialisierung	Transportbindung	Fragmentierung	Eignung kleiner Master-Slave-Bus	MicroPython und Pybricks
gRPC	Middleware	RPC und Datenströme	Protocol Buffers, schemagebunden	an HTTP/2 gebunden	über HTTP/2	ungeeignet, setzt Push voraus	nein, kein HTTP/2
JSON-RPC	Nachrichtenformat	nur Nachrichtenformat	JSON, textbasiert	unabhängig, offen	nicht definiert	offen, Rückkanal fehlt	ja, <code>ujson</code> vorhanden
MessagePack und CBOR	Serialisierung	nur Serialisierung	binär, schemafrei	entfällt	entfällt	entfällt	ja, mit eigener Umsetzung
Firmata	Geräteprotokoll	Gerätesteuerung mit festen Befehlen	binär, fest	seriell, fest	nein	passend, aber fester Befehlssatz	nein, Geräteseite in C
ISO-TCP	Transportprotokoll	nur Transport	entfällt	an CAN gebunden	ja, bis 4095 Byte	passend, aber ohne Aufrufe	entfällt, kein CAN
CANopen SDO	Geräteprotokoll	Zugriff auf ein Objektverzeichnis	feste Einträge	an CAN gebunden	ja, segmentiert	passend, aber festes Verzeichnis	entfällt, kein CAN
CoAP Block-wise	Anwendungsprotokoll	Representational State Transfer (REST)-artiger Zugriff	binär, schemafrei	setzt UDP voraus	ja, ab 16 Byte	Blockgröße zu groß	nein, kein UDP
MQTT	Nachrichtenprotokoll	Veröffentlichen und Abonnieren	binär, kompakt	unabhängig, braucht Gateway	teilweise	setzt eigenständiges Sendenvoraus	nein, kein Netzwerkstack
eRPC	Middleware	entfernter Aufruf, ein Schnittstelle zur Übersetzungszeit	eigenes Format, schemagebunden	austauschbar, seriell möglich	ja	passend, aber ohne Objektmodell	nein, C-Werkzeugkette
DDS-XRCE	Middleware	Datenaustausch über einen Agenten	binär, schemagebunden	austauschbar, seriell möglich	ja	passend, aber Agent nötig	nein, C-Bibliothek
PUPRemote	Transportanbindung, hier als Baustein verwendet	entfernter Aufruf, ein Modus je Aufruf	feste Formatangaben je Modus	an PUP und LPF2 gebunden	nein, ein Paket	passend, aber nur ein Paket	ja
Diese Arbeit	Middleware	entfernter Aufruf und entfernte Objekte	MessagePack, gekapselt und austauschbar	gekapselt, PUPRemote als erste Bindung	ja, eingebaut	passend, anfragegetriebener Rückkanal	ja

Aus dem Vergleich wird die Lücke deutlich, die diese Arbeit schließt. Sie ist als Ergebnis der hier getroffenen Auswahl zu lesen und nicht als Aussage über den gesamten Stand der Technik, denn dafür wäre eine systematische Recherche nötig, die den Rahmen einer Projektarbeit überschreitet. Die leistungsfähigen Rahmenwerke setzen einen Transport voraus, den das Zielsystem nicht bietet. Die schlanken Ansätze standardisieren nur Teilaspekte und lassen Rahmung, Fragmentierung und den Rückkanal offen. Die domänennahen Protokolle lösen eine andere Aufgabe oder setzen voraus, dass die untergeordnete Seite eigenständig zustellen kann. Am nächsten kommen eRPC und DDS-XRCE, die beide für Mikrocontroller und serielle Verbindungen gedacht sind.

Sie verlangen jedoch eine zur Übersetzungszeit festgelegte Schnittstelle und eine Werkzeugkette, die auf den hier eingesetzten Laufzeitumgebungen nicht zur Verfügung steht, und sie bieten kein Objektmodell, dessen Referenzen zur Laufzeit entstehen. PUPRemote liegt schließlich eine Ebene tiefer. Es erschließt den Bus und wird in dieser Arbeit als Busanbindung genutzt, ersetzt aber nicht die gesuchte Middleware.

Die vorliegende Arbeit verbindet diese Aspekte zu einer zusammenhängenden Lösung. Sie stellt einen transportunabhängig entworfenen entfernten Aufruf samt Objektmodell bereit, bringt Rahmung und Fragmentierung für sehr kleine Pakete mit und setzt einen anfragegetriebenen Rückkanal für einen Master-Slave-Bus um. All das läuft innerhalb der Grenzen von MicroPython und Pybricks. Der Beitrag liegt damit weniger in einem einzelnen neuen Baustein als in der Kombination bekannter Konzepte für einen Anwendungsfall, den keiner der betrachteten Ansätze abdeckt.

3 Systemanalyse und Anforderungen

Dieses Kapitel analysiert das Zielsystem und leitet daraus die Anforderungen ab. Zunächst wird die vorhandene Plattform betrachtet und begründet, warum eine externe Erweiterung nötig ist, bevor die Anforderungen an Display, Mikrocontroller und Kommunikation formuliert und den Einschränkungen der Plattform gegenübergestellt werden. Das Kapitel schließt mit einer Verallgemeinerung des Problemraums.

3.1 Zielsystem und Hardwareumgebung

Ausgangspunkt der vorliegenden Arbeit ist eine bestehende Hardwareplattform aus dem LEGO-Education-Umfeld, die die technischen Rahmenbedingungen festlegt, in denen die in Kapitel 1 motivierte Display-Integration umgesetzt werden muss. Im Folgenden wird zunächst die zentrale Hardwarekomponente vorgestellt und anschließend im Hinblick auf die Anbindung eines Touchscreen-Displays analysiert, woraus sich die Notwendigkeit einer externen Erweiterung ergibt.

3.1.1 Der LEGO Education SPIKE Prime Hub als Ausgangsplattform

Den Kern des Zielsystems bildet der LEGO Education SPIKE Prime Hub [37]. Hierbei handelt es sich um einen programmierbaren Steuerbaustein, der primär für den Einsatz im Bildungsbereich konzipiert ist und über sechs Ein- und Ausgabeports zum Anschluss von LEGO-kompatiblen Sensoren und Aktoren verfügt. Für die Interaktion mit dem Nutzer stellt der Hub eine Matrix aus 5×5 [Light Emitting Diodes \(LEDs\)](#), vier Tasten und einen einfachen Lautsprecher bereit.

Intern basiert der Hub auf einem Mikrocontroller vom Typ STM32F413VG mit einem ARM-Cortex-M4-Kern, einer Taktfrequenz von bis zu 100 MHz, 1 MB Flash-Speicher sowie 320 kB [Static Random Access Memory \(SRAM\)](#) [38], [39]. Die Programmierung kann grafisch über die offizielle LEGO-Software oder textbasiert mittels MicroPython erfolgen. Welche Firmware konkret eingesetzt wird und wie sich diese Wahl auf die spätere Lösung auswirkt, wird in Abschnitt 3.4 vertieft.

Für die vorliegende Arbeit sind drei Eigenschaften des Hubs von besonderer Bedeutung:

- Der Hub stellt ausschließlich passive, an die Sensor- und Motorports gebundene Erweiterungsmöglichkeiten bereit. Eine direkte Anbindung beliebiger Peripheriegeräte über offene Schnittstellen wie [SPI](#) [13] oder [I2C](#) [12] ist nicht vorgesehen.
- Die verfügbaren Anzeige- und Eingabemöglichkeiten beschränken sich auf die genannte [LED-Matrix](#) sowie die Hub-Tasten.
- Rechenleistung und Arbeitsspeicher sind für einen Mikrocontroller zwar typisch, für grafische Benutzeroberflächen jedoch stark begrenzt.

Diese Eigenschaften bilden den Rahmen, in dem die Display-Integration entworfen werden muss.

3.1.2 Notwendigkeit einer externen Erweiterung

Aus den in Abschnitt 1.1 motivierten Anforderungen an eine grafische Benutzeroberfläche mit Touch-Funktionalität ergibt sich ein direkter Konflikt mit der beschriebenen Plattform. Die im Hub integrierten Ausgabe- und Eingabemöglichkeiten beschränken sich auf eine niedrig auflösende [LED-Matrix](#) und einfache Tasten und sind damit für die Darstellung komplexer Inhalte sowie für eine differenzierte Benutzerinteraktion ungeeignet. Eine Erweiterung des Hubs selbst ist ebenfalls nicht vorgesehen, da er als geschlossene Komponente konzipiert ist und keine Schnittstellen zum direkten Anschluss grafischer Peripherie bereitstellt.

Die geforderte Display-Funktionalität lässt sich daher nur außerhalb des Hubs umsetzen. Die Ansteuerung des Displays muss an eine externe Hardwarekomponente abgegeben werden, die an einen der vorhandenen Sensorports des Hubs angeschlossen wird. Hieraus ergeben sich zwei eigenständige Teilprobleme, die in den folgenden Abschnitten weiter ausgearbeitet werden:

- die Wahl einer geeigneten externen Hardware zur Ansteuerung eines Displays sowie

- eine geeignete Kommunikation zwischen dem Hub und dieser externen Komponente.

3.2 Anforderungen an die Display-Integration

Die Anforderungen werden in drei Gruppen gegliedert. Die Anforderungen D1 bis D3 betreffen das Display selbst, S1 bis S3 den Mikrocontroller, der es ansteuert, und K1 bis K6 im folgenden Abschnitt die Kommunikation zwischen beiden Einheiten.

3.2.1 Anforderungen an ein externes Display

Bevor konkrete Hardwarekomponenten betrachtet werden, lassen sich aus der Zielsetzung der Arbeit allgemeine Anforderungen an ein externes Display ableiten:

- **D1 Grafische Darstellung.** Das Display soll die Darstellung einfacher grafischer Elemente wie Schaltflächen, Beschriftungen und Anzeigen ermöglichen.
- **D2 Berührungseingabe.** Es soll zugleich Benutzereingaben über einen Touchscreen entgegennehmen.
- **D3 Embedded-typische Auslegung.** Auflösung, Größe und Energieaufnahme orientieren sich an typischen Embedded-Anwendungen. Damit kommen kleine [TFT](#)-Displays mit einer Auflösung von etwa 240×320 Pixeln in Frage.

Solche Displays sind in unterschiedlichen Ausführungen verfügbar, wobei im Embedded-Bereich Module mit ILI9341-Display-Controller [40] weit verbreitet sind. Sie unterstützen eine Auflösung von 240×320 Pixeln bei einer Farbtiefe von 18 Bit und werden über [SPI](#) angesteuert. Für die Erkennung von Berührungen kommen typischerweise kapazitive Touchcontroller wie der FT6x06 [41] zum Einsatz, die über [I2C](#) angebunden werden und neben Einzelberührungen auch Mehrfingergesten erfassen können. Solche Module sind kostengünstig verfügbar und werden in der Maker- und Embedded-Community häufig in Verbindung mit Mikrocontrollern der ESP32-Familie eingesetzt.

3.2.2 Anforderungen an einen externen Mikrocontroller

Da der SPIKE Prime Hub selbst kein [TFT](#)-Display über [SPI](#) anbinden kann, muss zwischen Hub und Display ein zusätzlicher Mikrocontroller vermitteln. Er übernimmt die Ansteuerung des Displays und die Verarbeitung der Touch-Eingaben und kommuniziert gleichzeitig mit dem Hub. Aus dieser Doppelrolle ergeben sich folgende Anforderungen an die Hardware:

- **S1 Displayschnittstellen.** Sie muss über geeignete Schnittstellen zur Ansteuerung typischer [TFT](#)-Displays verfügen, insbesondere [SPI](#) für die Bildschirmdaten und [I2C](#) für den Touch-Controller.
- **S2 Rechenleistung und Speicher.** Sie muss ausreichend Rechenleistung und Speicher bieten, um eine grafische Bibliothek wie etwa [Light and Versatile Graphics Library \(LVGL\)](#) [42] ausführen zu können.
- **S3 Busanbindung.** Sie muss über eine Schnittstelle verfügen, die kompatibel zu den Sensorports des SPIKE Prime Hubs ist, oder eine entsprechende Anbindung über [LPF2](#) unterstützen.

Diese Anforderungen erfüllen Erweiterungsboards aus dem Bildungs- und Maker-Bereich, die speziell für die Anbindung an LEGO-Hubs entworfen wurden. Ein Beispiel sind die ESP32-basierten Boards der LMS-ESP32-Reihe [43]. Sie geben sich gegenüber dem Hub als LEGO-Sensor aus und können dadurch über das [LPF2](#)-Protokoll kommunizieren. Welche Hardware in dieser Arbeit konkret eingesetzt wird und wie die Software dafür eingerichtet ist, beschreibt Kapitel 6.

3.2.3 Resultierende Systemstruktur

Aus diesen Überlegungen ergibt sich eine Systemstruktur aus drei Teilen. Der SPIKE Prime Hub führt die Anwendung aus, ein externer Mikrocontroller steuert das Display an, und ein [TFT](#)-Touchscreen-Display dient zur Anzeige und Eingabe, wobei Hub und Mikrocontroller über einen

Sensorport des Hubs und das LPF2-Protokoll verbunden sind. Im Folgenden wird der Hub als Steuereinheit bezeichnet, während Mikrocontroller und Display zusammen die Anzeigeeinheit bilden. Wie die Kommunikation zwischen beiden aussehen muss und welchen Einschränkungen sie unterliegt, betrachten die folgenden Abschnitte.

3.3 Kommunikationsanforderungen

Der vorige Abschnitt legt fest, aus welchen Komponenten das System besteht. Offen ist noch, was die Verbindung zwischen Steuer- und Anzeigeeinheit leisten muss. Die folgenden Anforderungen leiten sich aus dem Anwendungsfall einer interaktiven Oberfläche ab und beziehen sich nicht auf einen bestimmten Übertragungsweg. Sie sind mit K1 bis K6 nummeriert, damit Kapitel 7 später auf sie verweisen kann.

- **K1 Strukturierte Nachrichten unbestimmter Länge.** Ein Bedienelement lässt sich nicht durch einen einzelnen Wert beschreiben. Zum Erzeugen einer Schaltfläche gehören Position, Abmessungen und Beschriftung. Ein Beschriftungstext hat keine feste Länge, und eine Auswahlliste enthält unterschiedlich viele Einträge. Die Verbindung muss deshalb zusammengesetzte Werte übertragen können, deren Länge erst zur Laufzeit feststeht. Feste Formatangaben je Aufruf, wie sie die in Kapitel 2 betrachtete Transportbibliothek verwendet, reichen dafür nicht aus.
- **K2 Aufrufe mit zuordenbarem Ergebnis.** Viele Aufrufe liefern einen Rückgabewert. Das Erzeugen eines Objekts liefert eine Referenz, über die es später angesprochen wird. Das Auslesen eines Zustands liefert den entsprechenden Wert. Zu jeder Anfrage muss also eine Antwort zurückkommen. Werden mehrere Anfragen nacheinander abgesetzt, muss erkennbar bleiben, welche Antwort zu welcher Anfrage gehört.
- **K3 Rückfluss von Ereignissen.** Berührungen entstehen auf der Anzeigeeinheit, ausgewertet werden sie aber von der Anwendung auf der Steuereinheit. Informationen müssen daher auch dann von der Anzeige- zur Steuereinheit gelangen, wenn sie keine Antwort auf einen Aufruf sind. Ereignisse sollen dabei nicht verloren gehen. Lässt sich ein Verlust nicht vermeiden, muss er zumindest erkennbar sein.
- **K4 Antwortzeit.** Berührung und Reaktion auf dem Bildschirm sollen für den Benutzer zusammengehören. Als Richtwert dient die bei direkter Manipulation übliche Grenze von etwa 100 Millisekunden [44], [45]. Gemeint ist dabei der gesamte Vorgang aus Erkennung, Übertragung, Verarbeitung und Rückmeldung, nicht die Übertragung eines einzelnen Pakets.
- **K5 Begrenzter Speicherbedarf.** Auf beiden Seiten der Verbindung arbeitet ein Mikrocontroller. Puffer brauchen deshalb eine feste Obergrenze, die schon beim Entwurf bekannt ist. Verfahren, die im ungünstigen Fall beliebig viel Speicher belegen, etwa das Sammeln aller bisher aufgetretenen Ereignisse, kommen nicht in Frage.
- **K6 Unabhängigkeit vom Übertragungsweg.** Der Übertragungsweg ist durch die Plattform vorgegeben und gehört nicht zum eigentlichen Problem. Seine Eigenschaften sollen sich deshalb nur in einem klar abgegrenzten Teil des Entwurfs auswirken. Prüfen lässt sich das daran, wie viel geändert werden müsste, um denselben Dienst über einen anderen Weg anzubieten.

K1 bis K3 betreffen den Inhalt der Übertragung, K4 und K5 die Bedingungen, unter denen sie stattfindet, und K6 den Aufbau des Entwurfs. Der folgende Abschnitt stellt diesen Anforderungen die Eigenschaften der Plattform gegenüber.

3.4 Einschränkungen der Plattform

Steuer- und Anzeigeeinheit sind über einen Sensorport des Hubs und das LPF2-Protokoll gekoppelt, woraus sich drei Einschränkungen ergeben, die den Anforderungen aus dem vorigen Abschnitt entgegenstehen. Sie werden im Folgenden einzeln beschrieben und den betroffenen Anforderungen zugeordnet, wobei es sich um Eigenschaften der Kopplung selbst handelt und

nicht um Schwächen einer bestimmten Umsetzung. Wie der Entwurf darauf reagiert, zeigen Kapitel 4 und Kapitel 5, die konkrete Umsetzung beschreibt Kapitel 6.

3.4.1 Eingeschränkte Kommunikationsrichtung

Im LPF2-Protokoll ist der Hub der Master [35] und wählt den aktiven Modus des angeschlossenen Geräts aus. Anders als ein Slave bei I2C meldet sich das Gerät als Sensor an und sendet die Werte dieses Modus, auch ohne Anfrage. Das Programm auf dem Hub erfährt davon jedoch nicht. Die Firmware hält je Modus nur den zuletzt empfangenen Wert, und das Programm sieht ihn erst, wenn es liest.

Die in Kapitel 2 beschriebene Bibliothek PUPRemote bildet das in zwei Zugriffsmustern ab. Bei einem Kanal legt die Anzeigeeinheit einen Wert ab, den die Steuereinheit liest. Bei einem Kommando schreibt die Steuereinheit Daten in einen Modus und liest im selben Vorgang die Antwort zurück. Nur das Kommando trägt Daten in beide Richtungen, und nur damit lassen sich Aufrufe übertragen. Ein zusätzlicher Kanal für Rückmeldungen läge in einem zweiten Modus, und der Hub müsste bei jedem Wechsel zwischen Aufruf und Abfrage den Modus umschalten. Die Middleware verwendet deshalb allein das Kommando, sodass jeder Austausch aus Anfrage und Antwort besteht.

Für die Anforderungen bedeutet das, dass sich weder Ergebnisse (K2) noch Ereignisse (K3) zustellen lassen, sondern beides nur bereitgehalten und abgefragt werden kann. Entsteht ein Ergebnis erst nach der letzten Anfrage, ist eine weitere Anfrage nötig, und eine Berührung zwischen zwei Anfragen muss bis zur nächsten Anfrage zwischengespeichert werden. Wann eine Rückmeldung eintrifft, bestimmt damit die Seite, die sie abholt. Die zeitliche Auflösung von Ereignissen ist dadurch durch den Abfragetakt der Steuereinheit begrenzt, was sich direkt auf K4 auswirkt.

3.4.2 Begrenzte Paketgröße

Ein Modus des Busses überträgt eine feste, vorab vereinbarte Anzahl von Bytes. Zusammen mit der auf dem Hub eingesetzten Firmware [46] liegt die nutzbare Obergrenze bei 16 Byte je Übertragung, oberhalb dieses Werts treten Prüfsummenfehler auf. Die Umsetzung von LPF2 in PUPRemote nennt denselben Wert und gibt bei einem Prüfsummenfehler den Hinweis aus, die Paketgröße unter Pybricks auf 16 Byte zu senken [36]. Die eigenen Versuche bestätigen das. Da jedes Paket zusätzlich Steuerinformationen für das Zusammensetzen tragen muss, bleibt für die Nachricht selbst noch weniger übrig. In der Referenzimplementierung sind es 13 nutzbare Byte je Paket, wie Kapitel 6 zeigt.

Schon eine einfache Aufrufnachricht mit Objektreferenz, Methodenname und wenigen Argumenten überschreitet diesen Rahmen deutlich. K1 lässt sich mit einem einzelnen Paket also nicht erfüllen, und Nachrichten müssen in mehrere Pakete zerlegt werden. Hinzu kommt, dass die Steuerinformationen bei einer Nutzlast von wenigen Byte einen großen Anteil der Übertragung ausmachen.

Eine weitere Beschränkung ergibt sich aus der Stromversorgung, da ein Display mehr Strom benötigt, als die Logikversorgung des Ports liefert, weshalb die Anzeigeeinheit die höhere Versorgungsspannung über den Port anfordert. In diesem Fall verkürzt sich die zulässige Länge der Modusnamen von elf auf fünf Zeichen, wodurch auch die Namen der Kanäle knapp werden. Abschnitt 6.2 geht auf die Versorgung näher ein.

3.4.3 Beschränkte Laufzeitumgebung

Der Hub wird in einer für Mikrocontroller ausgelegten Python-Variante programmiert, wie sie Abschnitt 2.3 beschreibt und deren Funktionsumfang gegenüber einer vollständigen Laufzeitumgebung deutlich eingeschränkt ist. Konkret handelt es sich um MicroPython [9] in der Ausprägung von Pybricks [46], einer Firmware für LEGO-Hubs. Abschnitt 6.1 geht auf die Unterschiede zwischen beiden Seiten ein.

Ein Netzwerkstack und die darauf aufbauenden Abstraktionen stehen nicht zur Verfügung. Die üblichen Bausteine verteilter Kommunikation lassen sich deshalb nicht verwenden und müssen durch eigene ersetzt werden.

Schwerer wiegt, dass auf der Steuereinheit keine echte Nebenläufigkeit zur Verfügung steht, da es keinen Hintergrundprozess gibt, der Pakete entgegennimmt oder Ergebnisse einsammelt, während die Anwendung weiterläuft. Das Warten auf ein Ergebnis findet deshalb im Kontrollfluss der Anwendung selbst statt, sodass die Anwendung stillsteht, solange das Ergebnis eines Aufrufs nicht vorliegt. Mehrere Aufrufe können sich somit nicht überlappen, und ihre Übertragungszeiten addieren sich.

Dazu kommt der begrenzte Arbeitsspeicher auf beiden Seiten, wodurch K5 auf dieser Plattform zu einer harten Grenze wird. Puffer für unvollständig empfangene Nachrichten, für bereitgehaltene Ergebnisse und für zwischengespeicherte Ereignisse benötigen jeweils eine feste Obergrenze, zudem muss festgelegt sein, was beim Erreichen dieser Grenze geschieht.

3.4.4 Wechselwirkung der Einschränkungen

Einzeln betrachtet ließe sich jede der drei Einschränkungen bewältigen. Kleine Pakete lassen sich durch Zerlegung überbrücken, eine Seite, die von Daten der anderen erst beim Nachfragen erfährt, durch regelmäßiges Abfragen, und ein Ablauf lässt sich auch ohne Nebenläufigkeit sequenziell formulieren. Zusammen verstärken sie sich jedoch gegenseitig.

Durch die kleine Paketgröße zerfällt eine Nachricht in viele Pakete. Wegen der eingeschränkten Kommunikationsrichtung erfordert jedes dieser Pakete eine eigene Anfrage der Steuereinheit samt Antwort, und für das Abholen des Ergebnisses sind weitere Anfragen nötig. Ohne Nebenläufigkeit werden all diese Anfragen nacheinander abgearbeitet.

Für den weiteren Entwurf folgt daraus, dass die Dauer eines Aufrufs vor allem von der Anzahl der nötigen Übertragungen abhängt und weniger von der Geschwindigkeit des Busses. Ein zusätzliches Byte Steuerinformation je Paket kann bei einer Nutzlast von wenigen Byte die Anzahl der Pakete und damit die Dauer eines Aufrufs erhöhen. Ziel des Entwurfs ist deshalb vor allem, Übertragungen einzusparen. Kapitel 7 greift das auf und beziffert es.

Tabelle II fasst die Gegenüberstellung von Anforderungen und Einschränkungen zusammen.

Tabelle II: Gegenüberstellung der Kommunikationsanforderungen und der Einschränkungen der Plattform

Nr.	Anforderung	Entgegenstehende Einschränkung	Folge für den Entwurf
K1	Strukturierte Nachrichten unbestimmter Länge	Feste Paketgröße von 16 Byte	Nachrichten müssen zerlegt und wieder zusammengesetzt werden
K2	Aufrufe mit zuordenbarem Ergebnis	Steuereinheit sieht Daten der Anzeigeeinheit erst beim Lesen	Ergebnisse werden bereitgehalten und abgefragt, Zuordnung über eine mitgeführte Kennung
K3	Rückfluss von Ereignissen	Steuereinheit sieht Daten der Anzeigeeinheit erst beim Lesen	Ereignisse werden zwischengespeichert und von der Steuereinheit abgeholt
K4	Antwortzeit	Alle drei Einschränkungen gemeinsam	Anzahl der Übertragungen je Aufruf wird zur bestimmenden Größe
K5	Begrenzter Speicherbedarf	Beschränkter Arbeitsspeicher beider Seiten	Feste Obergrenzen für alle Puffer, festgelegtes Verhalten beim Überlauf
K6	Unabhängigkeit vom Übertragungsweg	Eigenheiten von Bus und PUPRemote	Alle wegabhängigen Anteile in einer austauschbaren Schicht bündeln

3.5 Verworfen Alternativen

Die bisher beschriebene Aufteilung, ein eigener Mikrocontroller am Sensorport und ein Aufruf je Änderung, ist nicht die einzige denkbare. Zwei Alternativen liegen nahe genug, um ihre Verwerfung zu begründen.

Eine Funkverbindung statt des Sensorports. Der Hub besitzt ein Funkmodul, und eine Verbindung über [Bluetooth Low Energy \(BLE\)](#) würde die Paketgrenze des Busses umgehen. Drei Gründe sprechen dagegen. Erstens verschiebt sie die Paketgrenze nur: die voreingestellte Paketgröße des Attributprotokolls beträgt 23 Byte, von denen 20 für einen Wert bleiben [47]. Zerlegung und Rahmung wären damit weiterhin nötig. Den Rückkanal könnte [BLE](#) dagegen abkürzen, da der Server den Client über Benachrichtigungen ohne Anfrage informieren kann. Zweitens muss die Anzeigeeinheit ohnehin versorgt werden, und die dafür nötige Leitung liegt am Sensorport bereits an. Eine Funkverbindung führt also nicht zu weniger Kabel, sondern zu einem zweiten Versorgungsweg. Drittens stellt die eingesetzte Firmware für den Funkkanal keine Schnittstelle bereit, mit der sich eine Verbindung zu einem beliebigen Gerät aufbauen ließe. Dieser dritte Grund schließt die Alternative bereits für sich genommen aus.

Eine Beschreibung der Oberfläche statt einzelner Aufrufe. Anstatt jedes Element über einen eigenen Aufruf zu erzeugen, könnte die Steuereinheit einmalig eine Beschreibung der gesamten Oberfläche übertragen, die die Anzeigeeinheit auswertet. Der Aufbau kostet dann eine Übertragung statt vieler. Der Aufbau ist jedoch nicht das Problem, denn er findet einmal statt. Wiederkehrend sind das Ändern von Zuständen und das Abfragen von Ereignissen, und daran ändert eine Beschreibung nichts. Hinzu kommt, dass eine Beschreibung, die mehr kann als Elemente zu platzieren, eine Sprache wird, die auf einem Gerät ausgewertet wird, auf dem sie sich kaum nachvollziehen lässt. Dieser Weg ist für Sensornetze mit Maté bereits gegangen worden, einer kleinen virtuellen Maschine, an die statt einzelner Aufrufe ein Programm übertragen wird [48]. Der Preis ist dort derselbe, nämlich eine zweite Sprache und eine zweite Laufzeitumgebung, die mitgepflegt werden müssen. Die in Abschnitt 4.2.4 eingeführten Bindings sind die bewusst begrenzte Fassung dieses Gedankens: ein geschlossener Satz von Aktionen ohne Bedingungen und ohne Zustand.

3.6 Verallgemeinerung des Problemraums

Die im vorigen Abschnitt beschriebenen Einschränkungen wurden am konkreten Zielsystem hergeleitet, sind jedoch nicht an dieses System gebunden. Das ist für die weitere Arbeit wichtig, denn davon hängt ab, ob der folgende Entwurf eine Einzellösung ist oder für eine ganze Klasse von Verbindungen gilt.

Die drei Eigenschaften lassen sich von der Plattform ablösen und als allgemeine Merkmale einer Verbindung formulieren. Erstens ist die Übertragung anfragegetrieben, das heißt, eine der beiden Seiten erfährt von Daten der anderen erst, wenn sie selbst nachfragt. Zweitens ist die Nutzlast je Übertragung klein und fest vorgegeben. Drittens steht auf mindestens einer Seite eine Laufzeitumgebung zur Verfügung, die weder einen vollständigen Netzwerkstack noch die üblichen Mittel nebenläufiger Programmierung bereitstellt.

Einzelne dieser Merkmale finden sich in vielen Verbindungen, alle drei zusammen dagegen seltener. Bei der Kommunikation über [I2C](#) treibt der Master den Bus, während ein Slave eine Übertragung nicht selbstständig beginnen kann, und Modbus [RTU](#) folgt demselben Muster aus Anfrage und Antwort. Beide teilen damit das erste Merkmal, begrenzen die Nutzlast aber nicht in gleicher Weise, denn bei [I2C](#) legt erst die Umsetzung des Slaves eine Grenze fest, und Modbus [RTU](#) erlaubt bis zu 252 Byte Daten je Anfrage [14]. Bei [BLE](#) ist es umgekehrt. Die voreingestellte Nutzlast liegt in derselben Größenordnung wie beim hier betrachteten Bus, über Benachrichtigungen kann der Server den Client aber ohne Anfrage informieren [47]. Die Kombination aller drei Merkmale tritt am ehesten bei Sensorschnittstellen wie der hier betrachteten auf, bei denen ein Gerät Werte fester Länge in vorab vereinbarten Kanälen bereitstellt. Jedes Merkmal wirft dabei für sich eine der folgenden Fragen auf, gleich in welcher Verbindung es auftritt. Wie werden

Nachrichten zerlegt und wieder zusammengesetzt? Wie gelangen Ergebnisse und Ereignisse zur anfragenden Seite zurück? Und wie lässt sich all das vor der Anwendung verbergen?

Die vorliegende Arbeit behandelt den betrachteten Bus daher als einen Vertreter dieser Klasse. Der Entwurf in den folgenden Kapiteln bezieht sich deshalb auf die genannten Merkmale und nicht auf Eigenschaften einer bestimmten Plattform. Inwieweit das gelungen ist, wird am Ende der Arbeit diskutiert.

4 Architektur des Systems

Dieses Kapitel leitet die Architektur der Lösung aus den Anforderungen des vorigen Kapitels ab. Die Darstellung bleibt auf der konzeptionellen Ebene und nennt weder konkrete Bibliotheken noch Datenformate oder Klassen. Beschrieben werden die Ziele der Architektur, der geschichtete Aufbau mit den Aufgaben der einzelnen Schichten, die wiederkehrenden Rollen innerhalb dieser Schichten sowie das Kommunikationsmodell. Wie sich der Entwurf auf eine konkrete Hardware, Laufzeitumgebung und einen konkreten Bus abbilden lässt, zeigt Kapitel 6.

4.1 Zielsetzung der Architektur

Die Architektur verfolgt mehrere Ziele, die sich direkt aus den zuvor formulierten Anforderungen ergeben. Im Mittelpunkt steht die Aufrufstransparenz auf der Anwendungsseite, denn die Anwendung soll mit gewöhnlichen Methodenaufrufen arbeiten und nichts über Pakete, Bestätigungen oder das Abholen von Ergebnissen wissen müssen.

Ein zweites Ziel ist die Unabhängigkeit vom konkreten Übertragungsweg. Die höheren Schichten sollen so entworfen sein, dass der Transport hinter einer einheitlichen Schnittstelle liegt und ausgetauscht werden kann. Hinzu kommen die Bewältigung der kleinen Paketgröße durch Rahmung und Fragmentierung, ein anfragegetriebener Rückkanal für Ergebnisse und Ereignisse sowie ein sparsamer Umgang mit Speicher und Rechenzeit. Schließlich soll die Architektur eine klare und symmetrische Struktur besitzen, in der beide Seiten dieselben Schnittstellen teilen.

Aus diesen Zielen ergibt sich ein geschichteter Aufbau nach dem Vorbild einer Middleware mit entferntem Aufruf. Jede Schicht kapselt eine klar umrissene Aufgabe und stellt der darüberliegenden Schicht eine höhere Abstraktion bereit. Der Vorteil dieses Aufbaus ist, dass sich die schwierigen Eigenschaften des Zielsystems, also die geringe Paketgröße und die eingeschränkte Kommunikationsrichtung, auf wenige Schichten eingrenzen lassen und oberhalb dieser Schichten nicht mehr sichtbar sind.

4.2 Schichtenmodell

Das System ist in fünf Schichten gegliedert, die auf beiden Seiten der Verbindung vorhanden sind, wie Abbildung 1 zeigt.

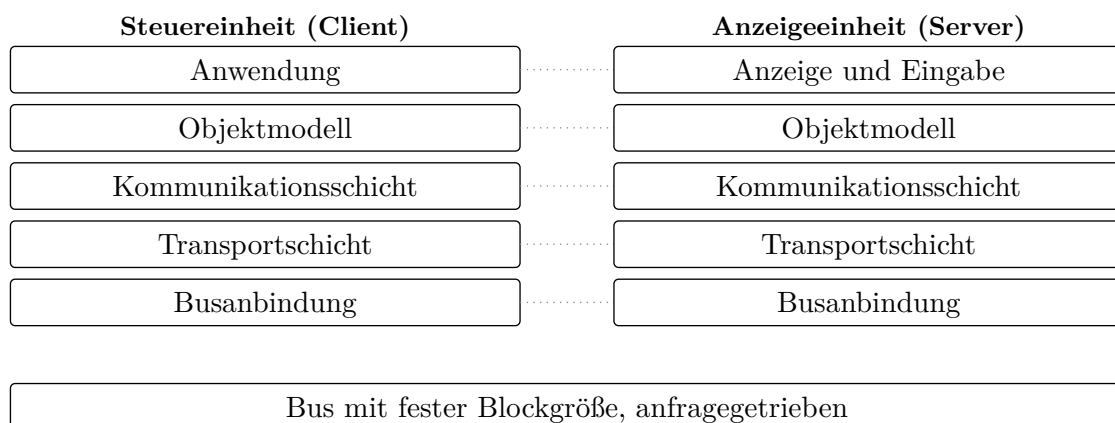


Abbildung 1: Schichtenmodell der Architektur. Die gepunktete Linie verbindet die Gegenstücke beider Seiten.

Zwei Eigenschaften dieses Aufbaus sind hervorzuheben. Zum einen ist er symmetrisch, da beide Seiten dieselben Schichten mit denselben Aufgaben besitzen und sich lediglich in der Richtung unterscheiden, aus der ein Aufruf eintrifft. Zum anderen ist er streng geschichtet, denn jede Schicht spricht ausschließlich mit der unmittelbar darunterliegenden, sodass eine Änderung innerhalb einer Schicht die übrigen nicht berührt.

Die beiden Seiten nehmen dabei feste Rollen ein. Die Steuereinheit ruft Methoden auf und beginnt jeden Austausch, weshalb sie im Folgenden als Client bezeichnet wird, während die Anzeigeeinheit die Aufrufe ausführt, ausschließlich antwortet und deshalb als Server bezeichnet wird. Die Rollen decken sich mit der Verteilung auf dem Bus, auf dem der Hub als Master arbeitet, und wechseln im Betrieb nicht, auch nicht bei Ereignissen, da auch diese vom Client abgefragt werden.

Die folgenden Abschnitte beschreiben die Schichten von unten nach oben. Für jede Schicht werden ihre Aufgabe, die nach oben angebotene Abstraktion und die Aufgaben benannt, die sie bewusst nicht übernimmt.

4.2.1 Busanbindung

Die Busanbindung ist die einzige Schicht mit Kenntnis des tatsächlichen Übertragungswegs. Ihre Aufgabe besteht darin, einen Block fester Länge zur Gegenseite zu befördern und einen ebensolchen Block als Antwort entgegenzunehmen, ohne den Inhalt dieses Blocks oder dessen Bedeutung zu kennen.

Nach oben bietet sie damit eine sehr schmale Abstraktion an, nämlich einen paketweisen, vom Client angestoßenen Austausch fester Größe. Das ist beabsichtigt, denn nur so bleibt die Schicht austauschbar, und jeder Übertragungsweg, der einen anfragegetriebenen Austausch von Blöcken fester Länge leisten kann, ist als Unterbau geeignet.

Nicht zu ihren Aufgaben gehören die Zerlegung größerer Nachrichten, die Zuordnung von Antworten zu Anfragen und jede Form der Auslegung des Inhalts. Die Paketgröße ist für diese Schicht eine gegebene Eigenschaft des Übertragungswegs und keine Größe, die sie beeinflusst.

4.2.2 Transportschicht

Die Transportschicht vermittelt zwischen Nachrichten beliebiger Länge und den kleinen Paketen des Übertragungswegs. In ihr wirkt sich damit die zentrale Einschränkung des Zielsystems aus.

Ihre erste Aufgabe ist die Serialisierung, bei der eine strukturierte Nachricht in eine Folge von Bytes überführt und auf der Gegenseite wieder ausgelesen wird. Ihre zweite Aufgabe ist die Rahmung, bei der jedes Paket Steuerinformationen erhält, mit denen die Gegenseite die ursprüngliche Nachricht wiederherstellen kann. Dazu gehören eine Kennung der Nachricht, die Position des Pakets innerhalb der Nachricht, die Angabe, ob es das letzte Paket ist, und die Anzahl der tatsächlich genutzten Bytes. Ihre dritte Aufgabe ist die Fragmentierung, also das Zerlegen einer Nachricht in eine Folge solcher Pakete und deren Zusammensetzen auf der Gegenseite.

Nach oben verbirgt diese Schicht die Paketgröße vollständig, sodass höhere Schichten mit Nachrichten arbeiten, deren Länge sie nicht zu beachten brauchen.

Zwei Entwurfsentscheidungen dieser Schicht wirken über sie hinaus. Die Rahmung ist vom Serialisierungsformat getrennt, und die Rahmung führt die Länge der Nutzlast ausdrücklich mit. Beide zusammen entscheiden darüber, welche Serialisierungsformate die Schicht überhaupt tragen kann. Abschnitt 5.2.3 begründet das im Einzelnen, Abschnitt 5.2.4 beziffert, was die Steuerinformationen bei wenigen Byte Nutzlast je Paket kosten.

4.2.3 Kommunikationsschicht

Die Kommunikationsschicht stellt den entfernten Aufruf her. Auf dem Client bildet sie aus dem Namen einer Methode, deren Argumenten und der Angabe des Scopes eine Nachricht, übergibt diese nach unten und nimmt anschließend das Ergebnis entgegen. Auf dem Server nimmt sie eine Nachricht entgegen, wählt den zuständigen Empfänger, ruft die benannte Methode auf und stellt deren Ergebnis zur Abholung bereit.

In dieser Schicht liegt außerdem die Behandlung des Rückkanals. Da der Client vom Server nur erfährt, was er selbst abfragt, wird das Ergebnis nicht zugestellt, sondern bereitgehalten und auf Anfrage herausgegeben. Die Kommunikationsschicht verbirgt diesen Umstand, sodass

ein Aufruf nach oben als gewöhnlicher, synchroner Methodenaufruf erscheint, der zurückkehrt, sobald das Ergebnis vorliegt.

Diese Schicht kennt die aufgerufenen Objekte nicht und befördert einen Aufruf zu einem benannten Empfänger, ohne zu wissen, was dieser darstellt.

4.2.4 Objektmodell

Das Objektmodell hebt den entfernten Aufruf auf die Ebene entfernter Objekte. Grafische Elemente wie Bildschirme, Beschriftungen und Schaltflächen werden als Objekte abgebildet, die auf dem Server bestehen und auf dem Client durch Stellvertreter vertreten werden.

Grundlage ist eine gemeinsame abstrakte Schnittstelle je Objekttyp, die auf beiden Seiten vorliegt. Damit ein Stellvertreter ein bestimmtes entferntes Objekt anspricht, vergibt der Server beim Erzeugen eines Objekts eine Referenz und verwaltet die Objekte in einer Registratur. Der Stellvertreter führt diese Referenz mit und übergibt sie bei jedem Aufruf. Auf diese Weise lassen sich viele Objekte erzeugen und gezielt ansprechen.

Hinzu kommen Reaktionen, die der Server selbst ausführt. Jede Reaktion über den Client durchläuft den anfragegetriebenen Rückkanal und ist damit an dessen Abfragetakt gebunden. Die Anwendung kann eine Reaktion deshalb auch vorab beim Server hinterlegen. Ein solches Binding verknüpft ein Ereignis auf einem Objekt mit einer Aktion auf einem anderen, und der Server führt die Aktion beim Eintreten des Ereignisses aus, ohne dass eine Übertragung nötig ist. Die Aktionen bilden einen kleinen, festen Satz, der nur die Darstellung betrifft, sodass die Logik der Anwendung auf dem Client bleibt. Die Umsetzung beschreibt Abschnitt 6.7.

Nach oben bietet diese Schicht eine Sicht an, die sich in der Schreibweise eines Aufrufs von einer rein lokalen Programmierung nicht unterscheidet. Das Ziel der Aufruftransparenz ist damit syntaktisch erreicht, im Zeitverhalten dagegen nicht, wie Abschnitt 2.1 bereits andeutet und Abschnitt 8.1 an der Umsetzung zeigt.

4.2.5 Anwendung und Anzeige

Die oberste Schicht unterscheidet sich auf beiden Seiten, da hier die eigentliche Aufgabe liegt. Auf der Steuereinheit ist es die Anwendung, die das Objektmodell nutzt und nichts von der Kommunikation weiß. Auf der Anzeigeeinheit ist es die Darstellung und die Entgegennahme von Eingaben, also die Ansteuerung der Anzeigehardware und die Erkennung von Berührungen.

Beide Seiten sind über die darunterliegenden Schichten vollständig entkoppelt, weshalb ein Wechsel der Anzeigehardware ausschließlich diese oberste Schicht auf der Anzeigeeinheit berührt.

4.3 Rollen im Entwurf

Innerhalb der Schichten treten drei wiederkehrende Rollen auf. Sie sind als Entwurfsmuster zu verstehen und legen fest, welche Verantwortung an welcher Stelle liegt, während die Umsetzung offenbleibt.

4.3.1 Interceptor

Der Interceptor bildet die Schnittstelle zwischen Kommunikationsschicht und Busanbindung. Der Name ist an das gleichnamige Muster aus [49] angelehnt, das dort einen festgelegten Punkt beschreibt, an dem sich ein Rahmenwerk um eigene Dienste erweitern lässt. Gemeinsam ist beiden der eine festgelegte Punkt, an dem eingegriffen wird. Hier wird jedoch nicht ein Dienst ergänzt, sondern der Übertragungsweg ausgetauscht. Näher liegt in dieser Hinsicht das Muster Forwarder-Receiver aus [4], das die Teilnehmer von den zugrunde liegenden Mechanismen der Kommunikation entkoppelt, indem es Senden und Empfangen in eigene Komponenten auslagert. Eine für beide Seiten gemeinsame Schnittstelle legt fest, wie ein Block zur Gegenseite gelangt und wie die Antwort zurückkommt. Auf dem Client verpackt der Interceptor einen Methodenaufruf, sendet dessen Pakete und holt anschließend das Ergebnis ab. Auf dem Server nimmt

er die Pakete entgegen, setzt die Nachricht zusammen und stellt das Ergebnis zur Abholung bereit.

Da diese Schnittstelle ohne Bezug auf einen konkreten Übertragungsweg formuliert ist, ist der Interceptor der Ansatzpunkt für die Unabhängigkeit vom Übertragungsweg. Ein anderer Übertragungsweg wird angebunden, indem eine weitere Umsetzung dieser Rolle bereitgestellt wird, ohne dass die höheren Schichten davon berührt werden.

4.3.2 Dispatcher

Der Dispatcher ordnet eine eingehende Nachricht der auszuführenden Methode zu. Jede Nachricht führt dazu die Angabe ihres Scopes, eine Methodenkennung und die Argumente mit sich. Anhand des Scopes wählt der Dispatcher den zuständigen Empfänger und ruft dort die benannte Methode auf.

Für jeden Objekttyp kann ein eigener Empfänger registriert werden, was das Routing der Nachrichten von der Umsetzung der Methoden entkoppelt. Das Objektmodell lässt sich dadurch schrittweise erweitern, ohne die darunterliegenden Schichten zu verändern.

4.3.3 Stellvertreter und Adapter

Stellvertreter und Adapter sind die beiden Seiten desselben Objekttyps. Der Stellvertreter implementiert die gemeinsame Schnittstelle auf dem Client und leitet jeden Methodenaufruf weiter. Der Adapter implementiert dieselbe Schnittstelle auf dem Server und enthält die eigentliche Logik, die der Dispatcher aufruft.

Dass beide dieselbe Schnittstelle erfüllen, ist die Grundlage der Aufrufstransparenz, denn für die Anwendung ist weder erkennbar noch relevant, ob sie mit einem Stellvertreter oder mit einer lokalen Umsetzung arbeitet.

4.4 Kommunikationsmodell

Aus Sicht der Anwendung ist die Kommunikation synchron, da ein Aufruf erst zurückkehrt, wenn das Ergebnis vorliegt. Intern folgt die Übertragung dagegen dem anfragegetriebenen Muster des Übertragungswegs, bei dem jeder Austausch vom Client ausgeht und der Server ausschließlich antwortet.

Logisch werden zwei Kanäle unterschieden. Über den ersten Kanal sendet der Client die Pakete eines Aufrufs, und der Server bestätigt jedes Paket. Über den zweiten Kanal fragt der Client den Zustand und das Ergebnis ab, indem er zunächst prüft, ob ein Ergebnis bereitsteht, und es anschließend Paket für Paket abholt.

Ereignisse wie Berührungen werden nach demselben Prinzip behandelt, indem der Server sie puffert und der Client sie bei Bedarf abfragt. Damit ist auch die Ereignisbehandlung an den Takt des Clients gebunden, was die zeitliche Auflösung begrenzt. Dieser anfragegetriebene Rückkanal ist die Antwort der Architektur auf die eingeschränkte Kommunikationsrichtung.

4.5 Datenfluss im System

Der Weg eines Aufrufs durch die Schichten lässt sich in vier Phasen gliedern. Zunächst nimmt der Stellvertreter den Methodenaufruf entgegen und übergibt ihn der Kommunikationsschicht. Diese bildet aus Methodennamen, Argumenten und Scope eine Nachricht, welche die Transportschicht serialisiert, in Pakete zerlegt und mit Steuerinformationen versieht.

In der zweiten Phase überträgt der Client die Pakete. Der Server bestätigt jedes empfangene Paket und setzt die Nachricht zusammen, sobald das letzte Paket eingetroffen ist. In der dritten Phase übergibt die Kommunikationsschicht die zusammengesetzte Nachricht dem Dispatcher, der die zugehörige Methode auf dem Adapter aufruft. Das Ergebnis wird wiederum zu einer Nachricht verpackt und zur Abholung bereitgestellt.

In der letzten Phase fragt der Client ab, ob das Ergebnis bereitsteht, und holt es anschließend Paket für Paket ab. Die Kommunikationsschicht setzt die Antwort zusammen, liest die Nutzlast aus und gibt das Ergebnis an den Stellvertreter zurück, der es an die Anwendung weiterreicht.

An diesem Ablauf zeigt sich, wofür die Übertragungen aufgewendet werden. Ein einzelner Aufruf zerfällt in eine Folge von Paketen, die jeweils einzeln bestätigt werden, gefolgt von wiederholten Abfragen nach dem Ergebnis. Nutzlast trägt davon nur der erste Teil. Was das für die Anzahl der Übertragungen bedeutet, beziffert Abschnitt 5.2.4.

5 Entwurf von Transport- und Kommunikationsschicht

Dieses Kapitel beschreibt den Entwurf der Transport- und der Kommunikationsschicht. Die beiden Schichten bilden die Grundlage für eine zuverlässige und effiziente Kommunikation zwischen der Anwendung und dem darunterliegenden Übertragungsweg.

Die Transportschicht, im [Open Systems Interconnection \(OSI\)](#)-Modell als Transport Layer bezeichnet, ist für die Zerlegung von Nachrichten beliebiger Länge in kleinere Pakete und deren korrekte Rekonstruktion auf der Empfängerseite verantwortlich. Dort stellt sie außerdem Funktionen wie Stauvermeidung (engl. congestion avoidance) und die Sicherstellung einer fehlerfreien Übertragung bereit [50].

Die Kommunikationsschicht setzt auf der Transportschicht auf und bildet die Schnittstelle zwischen der Anwendung und dem Transport. Sie kapselt die Details der Verbindung und stellt der Anwendung ein einfaches Kommunikationsmodell bereit. Im [OSI](#)-Modell entspricht sie am ehesten dem Session Layer, der für die Verwaltung von Kommunikationssitzungen und die Zuordnung von Anfragen und Antworten zuständig ist.

5.1 Motivation und Randbedingungen

Um die Entscheidungen in beiden Schichten nachvollziehen zu können, werden zunächst Motivation und Randbedingungen zusammengefasst. Aus den vorherigen Kapiteln geht hervor, dass die Transportschicht Nachrichten zuverlässig und effizient übertragen muss, unabhängig vom zugrunde liegenden Übertragungsweg. Dazu gehört auch die Serialisierung der Nutzlast, die in der Referenzimplementierung als Codec umgesetzt ist. Die Kommunikationsschicht muss dafür sorgen, dass Anfragen und Antworten zuverlässig zugeordnet werden, und sie stellt die Aufruftransparenz her.

Die Zuordnung zum [OSI](#)-Modell ist dabei bewusst nicht streng. Rahmung und Übertragungswiederholung gehören dort zur Sicherungsschicht, und der betrachtete Bus deckt diese Schicht auch ab, allerdings nur für einzelne Blöcke und ohne jeden Bezug zu einer Nachricht. Da die Blockgröße nicht veränderbar ist und der Bus keine Kennung je Nachricht kennt, müssen Rahmung und Wiederholung oberhalb davon erneut stattfinden, diesmal bezogen auf eine vollständige Nachricht. Ebenso wird die Serialisierung hier der Transportschicht zugeordnet und nicht der darüberliegenden Aufrufschicht, wie es bei klassischen [RPC](#)-Rahmenwerken üblich ist. Der Grund liegt in Abschnitt 5.2.3: Erst das Längenfeld der Rahmung erlaubt ein binäres Format, und damit hängt die Wahl des Formats unmittelbar an der Rahmung. Beide Zuordnungen folgen also der Aufgabe und nicht dem Modell.

5.2 Transportschicht

Wie bereits beschrieben, zerlegt die Transportschicht Nachrichten beliebiger Länge in Pakete fester Größe und setzt sie auf der Empfängerseite wieder zusammen. Ein solches Paket wird in diesem Kapitel als Block bezeichnet, sobald es um die Schnittstelle zum Übertragungsweg geht, und als Frame, sobald es die Steuerinformationen der Transportschicht trägt. Dazu muss jedes Paket Steuerinformationen tragen, wie es auch bei bekannten Protokollen wie [TCP](#) oder [UDP](#) der Fall ist.

Im Folgenden wird beschrieben, welchen Übertragungsweg die Schicht nach unten voraussetzt, wobei lediglich gefordert ist, dass er Blöcke begrenzter Größe befördert. Welche [OSI](#)-Schichten dieser Weg abdeckt, ist für den Entwurf ohne Belang und je nach System verschieden, beim hier betrachteten Bus sind es Bitübertragung und Sicherung. Anschließend werden die benötigten Steuerinformationen beschrieben, ohne feste Größen und Formate vorzugeben, damit der Entwurf auf andere Systeme übertragbar bleibt, während Abschnitt 6.4 die konkrete Umsetzung zeigt.

In verteilten Systemen durchläuft eine Nachricht den Schichtenstapel zweimal, auf der Senderseite abwärts und auf der Empfängerseite aufwärts. Jede Schicht fügt auf dem Weg nach

unten eigene Steuerinformationen hinzu, die auf dem Weg nach oben wieder ausgewertet und entfernt werden, was im Allgemeinen als Kapselung und Entkapselung bezeichnet wird [50]. Die folgenden Abschnitte betrachten deshalb stets beide Richtungen.

5.2.1 Interceptor-Abstraktion

Die Transportschicht setzt nach unten einen Übertragungsweg voraus, über den sie möglichst wenig wissen darf, damit die Anforderungen aus Abschnitt 3.3 erfüllt werden können. Jede Eigenschaft, die sie voraussetzt, schließt Übertragungswege aus, die diese Eigenschaft nicht bieten. Die Schnittstelle richtet sich deshalb danach, was die betrachtete Verbindungsklasse mindestens leisten muss, und nicht nach den Fähigkeiten eines bestimmten Busses.

Da der Client von Daten des Servers erst erfährt, wenn er selbst nachfragt, lässt sich jede Interaktion auf denselben Ablauf zurückführen:

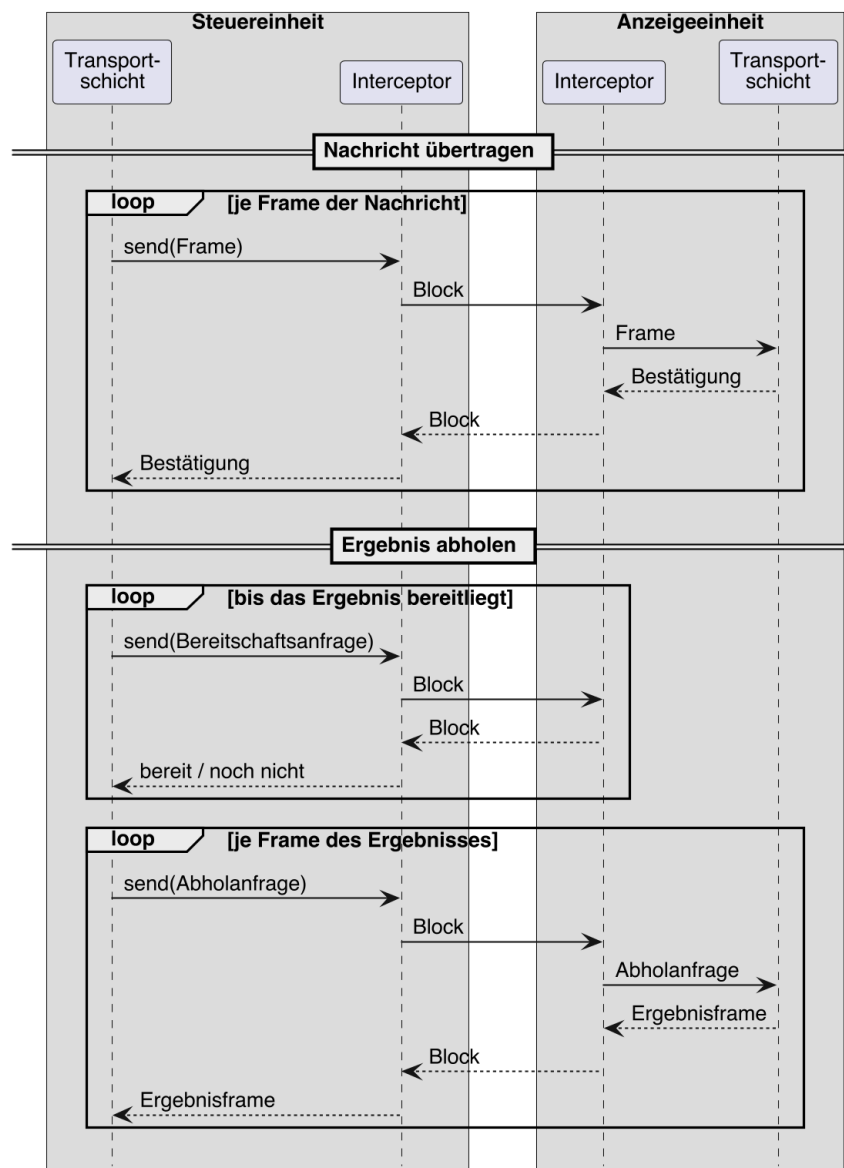


Abbildung 2: Ablauf eines Aufrufs an der Interceptor-Schnittstelle

Die Schnittstelle bietet daher eine einzige Operation, bei der ein Block begrenzter Größe übergeben wird und im selben Vorgang ein Block zurückkommt. Ob dieser Block Nutzlast, eine Bestätigung oder eine Anfrage nach dem Ergebnis enthält, spielt für die Schnittstelle keine

Rolle, da sie ihn nur befördert und nicht liest. Die drei Phasen in Abbildung 2 bestehen deshalb aus Wiederholungen derselben Operation, unterscheiden sich nur im Inhalt der Blöcke und werden alle vom Client angestoßen.

Eine umfangreichere Schnittstelle wäre denkbar, würde aber Übertragungswege ausschließen. Eine Zustellung ohne Anfrage setzt voraus, dass der Client eine Sendung des Servers bemerkt, ohne nachzufragen. Ein Warten auf eine Meldung setzt zusätzlich voraus, dass sie den Client weckt. Beides leistet der hier betrachtete Bus nicht, da der Client erst beim nächsten Lesen etwas bemerkt, und beides ist auch für die übrigen Vertreter der in Abschnitt 3.6 beschriebenen Klasse nicht vorauszusetzen. Die Schnittstelle bildet daher die Schnittmenge dessen, was alle Übertragungswege dieser Klasse leisten können.

Beide Seiten erfüllen dieselbe Schnittstelle und unterscheiden sich allein in der Richtung, aus der ein Block eintrifft. Der Client übergibt Blöcke und erhält Antworten, der Server nimmt Blöcke entgegen und beantwortet sie. Die Schnittstelle ist also symmetrisch, und eine Änderung des Übertragungswegs betrifft im Entwurf genau eine Stelle pro Seite.

Den Inhalt der Blöcke wertet die Schnittstelle nicht aus, denn sie kennt weder die Zugehörigkeit eines Blocks zu einer Nachricht noch seine Position darin, setzt nichts zusammen, erkennt keine Wiederholungen und ordnet keine Antworten zu. Alles, was eine Struktur voraussetzt, liegt darüber und wird in den folgenden Abschnitten beschrieben. Die Interceptor-Schnittstelle ist damit im Entwurf die Voraussetzung für die Anforderung K6 aus Abschnitt 3.3. Wie weit die Umsetzung sie erfüllt, bewertet Abschnitt 7.2.

5.2.2 Frame-Struktur

Für die Schnittstelle aus dem vorherigen Abschnitt genügt ein Block ohne weitere Struktur. Die Transportschicht gibt den Blöcken eine Struktur, um aus einer Folge unabhängiger Blöcke eine Nachricht beliebiger Länge zu machen. Dazu benötigt jeder Block neben der Nutzlast Steuerinformationen, die unter anderem die Zugehörigkeit zu einer Nachricht und die Position innerhalb der Nachricht beschreiben. Diese Steuerinformationen werden in einem Header zusammengefasst, der jedem Block vorangestellt wird. Ein Block mit dieser Struktur wird im Folgenden als Frame bezeichnet.

Die folgenden Felder sind für die Rahmung eines Frames notwendig. Größe und Format der Felder werden dabei bewusst nicht vorgegeben, damit der Entwurf auf andere Systeme übertragbar bleibt. Eine konkrete Ausprägung zeigt Abschnitt 6.4, und Kapitel 7 stellt ihr eine Fassung gegenüber, die dieselbe Aufgabe mit deutlich mehr Steuerinformationen löst.

Zugehörigkeit: Ohne Kennung der Nachricht lässt sich nicht entscheiden, ob ein eintreffender Block zu einer laufenden Übertragung gehört. Das ist auch bei einem rein synchronen Austausch nötig, bei dem immer nur eine Nachricht gleichzeitig unterwegs ist. Da direkt nach dem Absetzen einer Anfrage gelesen wird, hat die Gegenseite diese unter Umständen noch nicht verarbeitet. Gelesen wird dann die Antwort auf die vorangegangene Anfrage, und erst die Kennung macht solche veralteten Antworten erkennbar.

Position: Die Blöcke müssen in der richtigen Reihenfolge zusammengesetzt werden. Dazu muss die Position jedes Blocks innerhalb der Nachricht bekannt sein. Über die Position lassen sich außerdem Wiederholungen erkennen und verwerfen, die durch Übertragungsfehler oder Paketverlust entstehen.

Art: Über denselben Kanal werden sowohl Nutzlast als auch Steuerinformationen wie Bestätigungen, Anfragen nach dem Ergebnis und Fehlermeldungen übertragen. Da die Schnittstelle selbst nicht zwischen diesen Arten unterscheidet, muss der Header die Art des Blocks angeben.

Länge der Nutzlast: Der Übertragungsweg liefert Blöcke fester Größe, unabhängig davon, wie viel davon tatsächlich genutzt wird. Ohne ausdrückliche Angabe der Länge muss sie aus dem Inhalt abgeleitet werden. Warum das die Schicht an bestimmte Formate bindet, behandelt der nächste Abschnitt.

Ende der Nachricht: Der Empfänger kennt die Gesamtlänge der Nachricht nicht vorab. Daher muss das Ende der Nachricht explizit markiert werden, um zu erkennen, wann die Übertragung abgeschlossen ist. Man könnte das Ende daraus ableiten, dass der letzte Block nicht vollständig gefüllt ist. Ist die Nachrichtenlänge aber ein exaktes Vielfaches der Nutzlastgröße, ist auch der letzte Block voll und wird nicht als Ende erkannt. Der Empfänger würde dann auf einen Block warten, der nie kommt.

5.2.3 Nutzlastlänge und Formatunabhängigkeit

Neben den Steuerinformationen trägt ein Block die Nutzlast. Der Übertragungsweg liefert Blöcke fester Größe, die Länge der Nutzlast in einem Block ist aber nicht immer gleich. Eine fehlende Längenangabe ist unproblematisch, solange sich das Ende der Nutzlast aus ihrem Format ergibt. Bei textbasierten Formaten wie **JSON** oder **XML** ist das der Fall, denn sie enthalten keine Nullbytes, sodass Füllbytes am Ende eines Blocks eindeutig erkannt und entfernt werden können. Binäre Formate wie **CBOR** oder das in der Referenzimplementierung verwendete **MessagePack** besitzen diese Eigenschaft nicht.

Ohne Längenfeld ist die Transportschicht also an textbasierte Formate gebunden. Ein Wechsel des Formats oder die freie Wahl des Formats durch die Entwickler ist dann nicht möglich. Formatunabhängigkeit ist dabei keine ungewöhnliche Anforderung, denn Protokolle wie **HTTP** erlauben es beispielsweise, den Inhaltstyp der Nutzlast anzugeben, um verschiedene Formate zu unterstützen [51].

Um Formatunabhängigkeit zu erreichen, muss die Länge der Nutzlast ausdrücklich angegeben werden. Dazu nimmt der Header ein Längenfeld auf, das angibt, wie viele Bytes des Blocks tatsächlich zur Nutzlast gehören.

Die Breite dieses Feldes folgt aus der Blockgröße. Bezeichnet MTU die Größe eines Blocks und H die Größe des Headers, jeweils in Byte, so verbleibt als Nutzlast

$$P = MTU - H \quad (1)$$

Die MTU ist dabei die maximale Größe eines Blocks, die der Übertragungsweg befördern kann. Das Längenfeld muss jeden Wert von 0 bis P ausdrücken können. Seine Breite b in Bit beträgt damit

$$b = \lceil \log_2(P + 1) \rceil \quad (2)$$

Da das Längenfeld selbst Teil des Headers ist, hängen Formel 1 und Formel 2 voneinander ab: H bestimmt P , P bestimmt b , und b geht wieder in H ein. Aufgelöst wird das, indem der kleinste Header gewählt wird, für den beide Gleichungen zugleich gelten. Abschnitt 6.4 führt diese Rechnung für die konkrete Blockgröße aus.

Das Längenfeld ist damit die Voraussetzung für Formatunabhängigkeit.

5.2.4 Kosten der Rahmung

Jedes Byte des Headers ist Overhead und steht nicht für die Nutzlast zur Verfügung. Bei den hier betrachteten Blockgrößen bestimmt dieser Overhead den Aufwand der Übertragung maßgeblich.

Deutlich wird das, wenn Blöcke statt Bytes gezählt werden. Eine Nachricht der Länge L zerfällt in

$$N = \left\lceil \frac{L}{P} \right\rceil \quad (3)$$

Blöcke. Übertragen werden damit $N \cdot MTU$ Bytes, unabhängig davon, wie viel davon genutzt wird. Der letzte Block ist in der Regel nur teilweise gefüllt und wird trotzdem mit der vollen Blockgröße übertragen. Als Maß dient die Nutzlasteffizienz, also das Verhältnis von tatsächlich genutzten zu übertragenen Bytes:

$$\eta = \frac{L}{N \cdot \text{MTU}} \quad (4)$$

Daran lässt sich ablesen, wie stark Headergröße und Blockgröße die Effizienz beeinflussen. Wird der Header bei gleicher Blockgröße um ein Byte verkleinert, wächst P um eins, was nach Formel 3 bereits N um einen ganzen Block verringern kann. Einspart wird dann eine vollständige Übertragung und somit ein Round Trip.

Neben Blockgröße und Headerbreite beeinflusst auch die Länge der Nutzlast L die Effizienz, die selbst bei gleichem Inhalt vom gewählten Serialisierungsformat abhängt, wobei Kapitel 7 vergleicht, wie sich zwei verschiedene Formate auswirken. Die Nutzlasteffizienz hängt also von drei Faktoren ab, nämlich von Blockgröße, Headerbreite und Länge der Nutzlast.

Headerbreite und Nutzlastlänge wirken über dieselbe Aufrundung, jedoch auf unterschiedliche Weise, denn eine kürzere Nutzlast senkt N nur dann, wenn sie eine Blockgrenze unterschreitet, während ein kleinerer Header alle Blockgrenzen zugleich verschiebt. Bei kleiner MTU ist deshalb zu erwarten, dass die Rahmung stärker auf die Anzahl der Übertragungen wirkt als die Wahl des Serialisierungsformats, obwohl das Format die Nutzlast direkter verkleinert. Kapitel 7 prüft diese Erwartung.

5.2.5 Fragmentierung und Rekonstruktion

Die Zerlegung einer Nachricht folgt unmittelbar aus den bisherigen Festlegungen. Die serialisierte Nutzlast wird in Abschnitte von höchstens P Byte geteilt, jeder Abschnitt erhält die Kennung der Nachricht und seine Position, und der letzte wird als solcher markiert. Der letzte Abschnitt ist dabei in der Regel kürzer als P , die tatsächliche Länge trägt das Längenfeld.

Wichtig ist, dass diese Zuordnung eindeutig ist, denn zu einer Nachricht und einer Position gehört immer derselbe Block. Die Zerlegung ist damit wiederholbar und nicht an einen Fortschritt gebunden, also idempotent, sodass die Wiederholung einer Übertragung das Ergebnis nicht verändert.

Wann ist eine Nachricht vollständig? Die Gesamtlänge einer Nachricht ist dem Empfänger nicht bekannt, er kann die Vollständigkeit also nicht durch einen Vergleich mit ihr feststellen, sondern nur an den eingetroffenen Blöcken erkennen. Trägt der Block an Position n die Endmarkierung, besteht die Nachricht aus $n + 1$ Blöcken. Vollständig ist sie, sobald jede Position von 0 bis n mindestens einmal eingetroffen ist.

Die Steuerinformationen haben dabei verschiedene Aufgaben, denn Endmarkierung und Position bestimmen, wie viele Blöcke zur Nachricht gehören, während das Längenfeld angibt, wie viele Bytes eines Blocks dazugehören. Die Prüfung auf Vollständigkeit kommt somit ohne Kenntnis des Inhalts aus.

Was passiert bei Übertragungswiederholungen? Ein Block wird erneut übertragen, wenn die zugehörige Bestätigung ausbleibt. Das kann daran liegen, dass er den Empfänger nicht erreicht hat, dass ihn eine Prüfsumme des Übertragungswegs als beschädigt verworfen hat oder dass die Bestätigung selbst verloren ging. Für den Sender sind diese Fälle nicht unterscheidbar, weshalb er in allen gleich reagiert und den Block erneut sendet. Die Mechanismen dafür werden in Abschnitt 5.2.7 beschrieben.

Da die Zerlegung eindeutig ist, hat der wiederholte Block denselben Inhalt wie der ursprüngliche, was dem Empfänger allein jedoch nicht hilft, denn hängt er den Block ein zweites Mal an, entsteht eine falsche Nachricht. Die Rekonstruktion muss Wiederholungen deshalb selbst abfangen und Blöcke verwerfen, deren Position bereits vorliegt, wodurch auch sie idempotent wird.

Was passiert bei einer abgebrochenen Übertragung? Bricht eine Übertragung ab, bevor die Nachricht vollständig ist, bleiben die bereits eingetroffenen Blöcke liegen. Der Empfänger kann nicht entscheiden, ob die Übertragung noch fortgesetzt wird, da er weder die Gesamtzahl der Blöcke noch einen Zeitpunkt kennt, zu dem er aufgeben dürfte. Erkennbar wird der Abbruch erst daran, dass eine neue Übertragung beginnt, also ein Block an Position 0

eintrifft. Die Position allein genügt dafür allerdings nur, solange eine Nachricht mehr als einen Block umfasst. Besteht sie aus einem einzigen Block, sieht dessen Wiederholung genauso aus wie der Beginn einer neuen Übertragung. Unterscheiden lassen sich beide Fälle erst, wenn der Empfänger zusätzlich die Kennung mit der zuletzt empfangenen vergleicht. Da der Client nur eine Nachricht zugleich sendet, kann nur diese wiederholt werden, und eine gleiche Kennung an Position 0 ist damit stets eine Wiederholung.

Für den Umgang mit den liegengebliebenen Blöcken bestehen zwei Möglichkeiten:

- Die unvollständige Nachricht wird verworfen, sobald eine neue beginnt. Das ist der einfachere Weg, setzt aber voraus, dass nie mehr als eine Nachricht gleichzeitig übertragen wird.
- Die unvollständige Nachricht bleibt erhalten und kann später fortgesetzt werden. Das erlaubt mehrere gleichzeitige Übertragungen, verlangt aber einen Puffer je Kennung, eine Regel, wann eine Nachricht endgültig aufgegeben wird, und eine Laufzeitumgebung, die mehrere Übertragungen nebenläufig verwalten kann.

Der Entwurf wählt die erste Möglichkeit, da die betrachtete Verbindungsklasse anfragegetrieben ist und der Client ohnehin nur eine Nachricht zugleich senden kann. Hinzu kommt, dass die in Abschnitt 3.4 beschriebene Laufzeitumgebung die für die zweite Möglichkeit nötige Nebenläufigkeit nicht bereitstellt.

Wie viel Speicher benötigt die Rekonstruktion? Der Empfänger muss alle Blöcke einer Nachricht halten, bis sie vollständig ist. Anforderung K5 verlangt dafür eine zur Entwurfszeit bekannte Obergrenze, die sich aus den bisherigen Festlegungen ergibt. Ist b_{pos} die Breite des Positionsfelds in Bit und r die Anzahl der Werte, die für Frames ohne Position reserviert sind, so bleiben $2^{b_{\text{pos}}} - r$ nutzbare Positionen, und die längste übertragbare Nachricht umfasst höchstens

$$L_{\text{max}} = (2^{b_{\text{pos}}} - r) \cdot P \quad (5)$$

Bytes. Dieser Wert ist zugleich die Obergrenze des Puffers, den die Gegenseite bereithalten muss.

5.2.6 Serialisierung

Die in Abschnitt 2.2 eingeführte Serialisierung überführt einen strukturierten Wert in eine Bytefolge und liest ihn auf der Gegenseite wieder aus. Sie liegt damit zwischen der Kommunikationsschicht, die mit Werten arbeitet, und der Fragmentierung, die mit Bytes arbeitet. Aus Abschnitt 5.2.3 folgt dabei eine Reihenfolge, denn das Format kann erst dann frei gewählt werden, wenn die Rahmung die Länge der Nutzlast ausdrücklich mitführt.

Für die Transportschicht ist die Nutzlast ohne Struktur. Sie nimmt eine Bytefolge entgegen, zerlegt sie und setzt sie auf der Gegenseite wieder zusammen. Welche Werte darin vorkommen und wie sie angeordnet sind, wertet sie nicht aus, sodass das Format austauschbar ist, ohne dass die Rahmung geändert werden muss.

Die Wahl des Formats ist an drei Bedingungen gebunden, die sich aus den bisherigen Abschnitten ergeben.

Erstens muss das Format kompakt sein. Nach Formel 3 bestimmt die Länge der Nutzlast unmittelbar die Anzahl der Blöcke, in die eine Nachricht zerfällt, und damit die Anzahl der Übertragungen.

Zweitens muss es ohne Schema auskommen. Welche Objekte und Methoden es gibt, steht nicht vorab fest und ändert sich während der Entwicklung. Ein schemagebundenes Format würde einen Übersetzungsschritt einführen und die Kommunikationsschicht an diesen Schritt binden.

Drittens muss der Decoder klein sein. Er läuft auf beiden Seiten der Verbindung, und beide sind Mikrocontroller. Anforderung K5 gilt hier also auch für den Umfang der eingesetzten Bibliothek und nicht nur für die Puffer.

Welches Format diese Bedingungen erfüllt, zeigt Kapitel 6. Wie stark sich die Wahl auf die Anzahl der Übertragungen auswirkt, misst Kapitel 7.

5.2.7 Zuverlässigkeitsmechanismen

Die bisherigen Abschnitte setzen voraus, dass ein Block die Gegenseite erreicht, was jedoch nicht garantiert ist. Ein Block kann auf dem Weg verloren gehen oder vom Übertragungsweg als beschädigt verworfen werden, zudem kann die Bestätigung ausbleiben, obwohl der Block angekommen ist.

Für diese Fälle wird Stop-and-Wait [Automatic Repeat reQuest \(ARQ\)](#) [50] verwendet, bei dem ein Block gesendet und auf dessen Bestätigung gewartet wird, die bei Ausbleiben ein erneutes Senden des Blocks auslöst. Verfahren mit Sendefenster erlauben dagegen mehrere unbestätigte Blöcke gleichzeitig und erreichen dadurch einen höheren Durchsatz, weshalb sie weit verbreitet sind und beispielsweise in Mobilfunkstandards und in [TCP](#) eingesetzt werden. Sie setzen jedoch voraus, dass mehrere Blöcke gleichzeitig unterwegs sein können und die Gegenseite Bestätigungen ohne eigene Anfrage zurücksenden kann. In der betrachteten Verbindungsklasse besteht jede Übertragung aus Anfrage und Antwort, sodass ohnehin nie mehr als ein Block unterwegs ist. Stop-and-Wait bildet damit genau das ab, was die Verbindung zulässt.

Die drei genannten Fehlerfälle sind für den Sender nicht unterscheidbar, da er in allen Fällen dasselbe sieht, nämlich eine ausbleibende Bestätigung. Eine Unterscheidung würde zudem nichts bringen, weil die Reaktion immer dieselbe ist. Der Empfänger erkennt die Wiederholung an der Position und verwirft sie, wie in Abschnitt 5.2.5 beschrieben.

Wiederholungen brauchen eine Obergrenze, denn ohne sie wartet der Client endlos, sobald der Server dauerhaft nicht antwortet. Aus einem Fehler auf der einen Seite wird dann ein Stillstand der Anwendung auf der anderen. Mit einer Obergrenze scheitert der Aufruf nach einer bekannten Zeit und die Anwendung behält die Kontrolle. Das ist zugleich die Voraussetzung dafür, dass K4 geprüft werden kann, denn eine Antwortzeit lässt sich nur angeben, wenn sie nach oben begrenzt ist. Zu beachten ist dabei, dass eine Obergrenze je Phase noch keine Obergrenze für den Aufruf ergibt. Wie beide zusammenhängen, führt Abschnitt 8.1 für die Referenzimplementierung aus.

Dieselbe Überlegung gilt für das Abholen des Ergebnisses, denn der Client fragt so lange nach, bis ein Ergebnis vorliegt, und benötigt auch dafür eine Obergrenze. Für den Server folgt daraus die Zusicherung, dass jeder Aufruf ein Ergebnis oder eine Fehlermeldung hinterlassen muss, auch wenn die Ausführung selbst fehlschlägt. Bleibt beides aus, wartet der Client auf etwas, das nie eintritt. Die Fehlerbehandlung ist damit Teil des Protokolls und wird nicht der Umsetzung überlassen.

Zusammen mit der idempotenten Rekonstruktion aus Abschnitt 5.2.5 legt das die Aufrufsemantik fest. Ein wiederholter Block wird verworfen, ein wiederholter letzter Block löst die Ausführung kein zweites Mal aus, und ein Ergebnis wird beliebig oft unverändert herausgegeben. Ein Aufruf wird deshalb höchstens einmal ausgeführt, in der Einteilung von [1] also mit At-most-once-Semantik. Für eine Nachricht aus einem einzigen Block gilt das nur mit dem Vergleich der Kennung aus Abschnitt 5.2.5. Dass er mindestens einmal ausgeführt wird, sichert das Protokoll dagegen nicht zu: erreicht der letzte Block den Server nie, scheitert der Aufruf nach der Obergrenze, ohne dass auf dem Server etwas geschehen ist. Für die betrachteten Aufrufe ist das die richtige Wahl, da eine doppelt ausgeführte Änderung an der Anzeige nicht rückgängig zu machen wäre, ein gescheiterter Aufruf dagegen von der Anwendung wiederholt werden kann.

Drei Aufgaben übernimmt die Schicht ausdrücklich nicht. Sie korrigiert keine Fehler, sondern verlässt sich auf die Prüfsumme des Übertragungswegs und wiederholt den betroffenen Block. Sie sichert die Reihenfolge nicht gesondert ab, da der Kanal sequenziell arbeitet, wobei das Positionsfeld die Rekonstruktion gleichwohl von der Reihenfolge unabhängig macht. Außerdem betreibt sie keine Stauvermeidung, da auf einer Punkt-zu-Punkt-Verbindung mit einem Master und einer einzigen Nachricht gleichzeitig kein Stau entstehen kann.

5.3 Kommunikationsschicht

Mit der Transportschicht lassen sich Nachrichten beliebiger Länge übertragen, was für eine Anwendung jedoch noch keine geeignete Schnittstelle darstellt, da sie mit Methodenaufrufen arbeitet. Diese Lücke schließt die Kommunikationsschicht, indem sie einen Aufruf auf eine Nachricht abbildet, ihn über die Transportschicht befördert und das Ergebnis an die aufrufende Stelle zurückgibt.

Von der Transportschicht kennt sie nur zwei Vorgänge, nämlich das Senden einer Nachricht und das Eintreffen einer Antwort. In wie viele Blöcke die Nachricht dabei zerfällt, wie oft ein Block wiederholt wird und in welchem Format die Nutzlast vorliegt, bleibt verborgen.

Die Schicht stellt einen **RPC** her, wie er in Kapitel 2 eingeführt wurde. Die Anwendung ruft eine Methode auf, die auf einem anderen Gerät ausgeführt wird, und erhält deren Rückgabewert, wobei an der Aufrufstelle nicht erkennbar ist, dass eine Übertragung stattgefunden hat, was als Aufruftransparenz bezeichnet wird.

Die dort betrachteten Rahmenwerke setzen dafür Eigenschaften voraus, die diese Verbindung nicht bietet. Sie erwarten einen Transport, der Nachrichten beliebiger Länge zuverlässig befördert, und einen Rückkanal, über den der Server sein Ergebnis von sich aus zustellt. Beides ist hier nicht gegeben, weshalb die folgenden Abschnitte einen eigenen Aufrufmechanismus entwickeln, der mit den Mitteln der Transportschicht auskommt.

Dabei ist zu klären, welche Angaben eine Aufrufnachricht mitführen muss, wie eine Antwort ihrer Anfrage zugeordnet wird, wie der Server den zuständigen Empfänger auswählt und auf welchem Weg ein Ergebnis zurückgelangt.

5.3.1 Aufbau einer Aufrufnachricht

Ein **RPC** muss so viel mitführen, dass der Server den Aufruf ohne weitere Rückfrage durchführen kann, wofür drei Angaben nötig sind. Die Methodenkennung legt fest, was ausgeführt wird, die Argumente liefern die Werte, mit denen es ausgeführt wird, und der Scope bestimmt, welcher Empfänger die Methode ausführt. Wie das funktioniert, beschreibt der übernächste Abschnitt.

Angaben über Typen oder Signaturen werden nicht mitgeführt, denn nach Abschnitt 5.2.6 kommt das Format ohne Schema aus und überträgt nur Werte, keine Beschreibungen von Werten. Ob eine Methode existiert oder bestimmte Argumente erwartet, stellt sich somit erst auf dem Server und damit zur Laufzeit heraus.

Das Ergebnis nimmt denselben Weg zurück und ist ebenfalls eine Nachricht, die entweder den Rückgabewert oder eine Fehlermeldung trägt. Welcher der beiden Fälle vorliegt, ist am Schlüssel zu erkennen, unter dem der Inhalt steht. Das folgt aus der Zusicherung in Abschnitt 5.2.7, denn der Client wartet auf genau eine Antwort und muss auch im Fehlerfall eine erhalten.

Diese Angaben werden mit jedem Aufruf übertragen, und nach Formel 3 wirkt sich ihre Länge direkt auf die Anzahl der Blöcke aus. Zwei Festlegungen halten sie deshalb kurz. Die Argumente werden in der Reihenfolge der Parameter übertragen, ohne ihre Namen. Die Methode wird über eine Kennung fester Länge bezeichnet, die beide Seiten auf dieselbe Weise aus ihrem Namen berechnen, sodass keine Tabelle zwischen Namen und Kennungen gepflegt werden muss.

Beide Festlegungen setzen voraus, dass Client und Server dieselbe Schnittstelle kennen. Sie erreichen damit dasselbe wie ein schemagebundenes Format, das Bezeichner einmal vorab vereinbart und in der Nachricht nur noch Werte überträgt. Als Vereinbarung dient hier die gemeinsame Schnittstelle der Objekttypen aus Kapitel 4. Es entsteht kein eigener Übersetzungsschritt, und das Serialisierungsformat bleibt schemafrei.

Fehler in dieser Vereinbarung fallen unterschiedlich auf. Eine unbekannte Methodenkennung erkennt der Server und antwortet mit einer Fehlermeldung, während eine abweichende Reihenfolge der Argumente nur auffällt, wenn sich dabei ihre Anzahl ändert. Vertauschte Werte gleicher Art werden dagegen ohne Fehler ausgeführt.

5.3.2 Zuordnung von Anfrage und Antwort

Jede Antwort muss der Anfrage zugeordnet werden, zu der sie gehört. Der Frame führt nach Abschnitt 5.2.2 bereits eine Kennung je Nachricht mit, und ein Aufruf entspricht genau einer Nachricht. Diese Kennung identifiziert also zugleich den Aufruf, und eine zweite Angabe in der Nutzlast ist nicht nötig.

Die Kennung stammt damit aus dem Header, und ihr Wertebereich folgt aus dessen Breite. Sie wiederholt sich nach einer festen Anzahl von Aufrufen, was zulässig ist, solange zwischen zwei gleichen Kennungen kein älteres Ergebnis mehr unterwegs sein kann. Eine eigene Kennung der Kommunikationsschicht wäre von der Headerbreite unabhängig, müsste aber in jeder Nachricht zusätzlich übertragen werden und würde nach Formel 3 die Anzahl der Übertragungen erhöhen. Die Übernahme der Kennung aus dem Header ist also eine Abwägung zwischen doppelten Steuerinformationen und einer strengen Trennung der Schichten.

5.3.3 Verteilung eingehender Aufrufe

Der Server erhält eine Nachricht mit Methodenkennung, Argumenten und Scope und muss daraus bestimmen, wer den Aufruf ausführt. Die Adressierung ist zweistufig und mit dem Routing in [REST](#)-Frameworks vergleichbar. Dort ist ein Controller für einen Ressourcentyp zuständig, eine Methode darin für die Operation, und die Kennung der einzelnen Ressource kommt als Parameter an. Hier benennt der Scope die Art des Empfängers, der Methodenname die Operation, und die Referenz des einzelnen Objekts reist als Argument mit.

Der Unterschied liegt im Satz der Operationen. [REST](#) beschränkt ihn auf wenige festgelegte Verben, während hier jede Methode des Objektmodells aufrufbar ist. Die Referenz ist außerdem kein Pfad, sondern eine Kennung, die der Server beim Erzeugen des Objekts vergibt. Wie sie gebildet wird, gehört zum Objektmodell und wird in Abschnitt 6.6 gezeigt.

Die zweistufige Adressierung hat einen weiteren Vorteil, da die Kennungen der Methoden nur innerhalb eines Scopes eindeutig sein müssen. Gleichnamige Methoden können somit auf verschiedenen Arten von Empfängern nebeneinander bestehen, und eine zufällige Übereinstimmung zweier Kennungen stört nur, wenn sie im selben Scope auftritt. Der Server prüft das beim Registrieren eines Empfängers und nicht erst beim Aufruf.

Verteilung und Ausführung bleiben dabei getrennt, denn die Verteilung wählt anhand des Scopes den Empfänger aus, während die Ausführung beim Empfänger selbst liegt. Ein weiterer Empfänger kann somit registriert werden, ohne die Verteilung oder die darunterliegenden Schichten zu ändern, wodurch sich das Objektmodell erweitern lässt.

Findet sich für einen Scope kein Empfänger, muss der Aufruf mit einer Fehlermeldung beantwortet werden. Nach Abschnitt 5.2.7 wartet der Client auf genau eine Antwort, und ein stillschweigendes Verwerfen würde ihn bis zur Obergrenze warten lassen.

5.3.4 Anfragegetriebener Rückkanal

Aufgrund der eingeschränkten Kommunikationsrichtung aus Abschnitt 3.4 erfährt der Client von einem Ergebnis erst, wenn er selbst nachfragt. Der Server kann es deshalb nicht zustellen, sondern nur bereithalten, bis es abgeholt wird. Hinzu kommt, dass die Ausführung noch nicht abgeschlossen ist, wenn der letzte Block des Aufrufs bestätigt wurde. Der Client muss also zunächst feststellen, ob ein Ergebnis vorliegt, und es anschließend abholen.

Ereignisse folgen demselben Muster, unterscheiden sich jedoch in einem Punkt. Ein Ergebnis erwartet der Client, weil er den zugehörigen Aufruf abgesetzt hat, ein Ereignis dagegen nicht, sodass er regelmäßig nachfragen muss, ohne zu wissen, ob es etwas abzuholen gibt.

Die Unterscheidung zwischen einem Abholen durch den Empfänger und einem Zustellen durch die Quelle ist in verteilten Systemen geläufig und unter den Begriffen Pull und Push eingeführt [52]. Eine Wahl besteht hier nicht, denn auch ein ohne Anfrage gesendeter Wert wird auf dem Client erst beim nächsten Lesen sichtbar. Die zeitliche Auflösung hängt damit vom Abfragetakt ab, denn ein Ereignis, das zwischen zwei Abfragen auftritt, wird erst bei

der nächsten bemerkt. Häufigeres Abfragen verkürzt zwar diese Verzögerung, kostet jedoch je Abfrage eine Übertragung und damit Zeit, die für Aufrufe fehlt, während selteneres Abfragen Übertragungen spart und die Verzögerung verlängert. Beide Ziele lassen sich also nicht zugleich erreichen.

Zwischen zwei Abfragen können mehrere Ereignisse auftreten, die der Server puffern muss, wobei der Puffer nach K5 begrenzt ist. Läuft er über, gehen Ereignisse verloren, weshalb für diesen Fall ein Verhalten festgelegt sein muss, da ein stilles Verwerfen für die Anwendung nicht erkennbar wäre.

Am anfragegetriebenen Rückkanal wirkt sich die eingeschränkte Kommunikationsrichtung direkt auf die Anwendung aus. Ob die erreichbare Antwortzeit die Anforderung K4 einhält, prüft Kapitel 7. Eine Möglichkeit, diesen Weg für reine Darstellungsreaktionen zu umgehen, zeigt Abschnitt 6.7.

6 Referenzimplementierung

Dieses Kapitel bildet den Entwurf aus Kapitel 4 und Kapitel 5 auf eine konkrete Hardware, zwei konkrete Laufzeitumgebungen und einen konkreten Bus ab. Hier stehen die Zahlen, die der Entwurf offengelassen hat, also die Blockgröße, die Breite der Felder im Header und das gewählte Serialisierungsformat. Der Aufbau folgt den Schichten von unten nach oben und schließt mit dem Ablauf eines vollständigen Aufrufs sowie der Nebenläufigkeit.

6.1 Laufzeitumgebung

Client und Server laufen beide unter MicroPython, allerdings in sehr unterschiedlichen Ausprägungen.

Auf dem Hub läuft Pybricks [46], eine Firmware, die die Software von LEGO ersetzt und eine stark reduzierte Variante von MicroPython bereitstellt. Für die Umsetzung sind vier Einschränkungen von Bedeutung. Ganzzahlen sind auf $2^{30} - 1$ begrenzt, da die Firmware ohne Unterstützung für lange Ganzzahlen übersetzt ist, und schon eine größere Konstante im Quelltext verhindert das Laden des Moduls. Threads stehen nicht zur Verfügung, ebenso wenig einige Teile der Standardbibliothek wie `memoryview` oder das Modul `warnings`. Außerdem nimmt der Hub nur einzelne Dateien an, keine Pakete mit Unterverzeichnissen.

Grund für die alternative Firmware sind die Einschränkungen von LEGO. Der Hub ist ein geschlossenes System, das nur die von LEGO bereitgestellten Sensoren und Aktoren akzeptiert. Erst die Pybricks-Firmware [46] erlaubt die Anbindung externer Geräte, jedoch auch nur durch die bereitgestellte Schnittstelle LPF2 und PUPRemote [36], [43].

Die Module der Middleware liegen dagegen in einer Paketstruktur. Ein eigenes Werkzeug, der Bundler in `tools/bundler.py`, führt deshalb alle Module, die der Client benötigt, zu einer einzigen Datei `pybricks_bundle.py` zusammen. Er löst die Importe innerhalb des Projekts auf, sortiert die Module so, dass jedes nach seinen Abhängigkeiten steht, und ersetzt dabei `struct` durch das auf dem Hub vorhandene `ustruct`. Das Anwendungsprogramm auf dem Hub importiert anschließend nur noch diese Datei.

Auf dem ESP32 läuft MicroPython mit der Anbindung an `LVGL`, wobei diese Umgebung deutlich umfangreicher ausfällt und lange Ganzzahlen, Threads, `uasyncio` sowie einen Zufallszahlengenerator bietet. Knapp ist hier vor allem der Arbeitsspeicher, da `LVGL` einen großen Teil des Heaps für die Darstellung belegt. Welche Folgen das für die Nebenläufigkeit hat, beschreibt Abschnitt 6.9.

Der gemeinsame Code muss in beiden Umgebungen laufen und richtet sich deshalb nach der engeren, also nach Pybricks. Das betrifft vor allem die Transportschicht, den Codec und die Stellvertreter. Code, der nur auf dem Server läuft, darf die Möglichkeiten des ESP32 nutzen. Tabelle III ordnet die Module der Referenzimplementierung den Schichten aus Kapitel 4 zu.

Tabelle III: Zuordnung der Module zu den Schichten aus Kapitel 4

Schicht	Module
Anwendung und Anzeige	hub/main.py , display/server/driver/
Objektmodell	display/protocol/ , display/client/ mit remote.py , display/server/ , middleware/rpc/registry.py
Kommunikationsschicht	middleware/rpc/dispatcher.py , middleware/rpc/names.py , middleware/errors.py , Aufrufteil von transport/pupremote/client.py und server.py
Transportschicht	middleware/frame.py , message.py , codec.py , queue.py , constants.py , Übertragungsteil von client.py und server.py
Busanbindung	transport/base.py , transport/pupremote/binding.py , vendor/pupremote/

Zwei Zeilen der Tabelle verdienen eine Anmerkung. Die Kommunikationsschicht ist nicht vollständig busunabhängig abgelegt, denn ihr Aufrufteil liegt in denselben beiden Klassen, die auch die Rahmung ansteuern, und diese Klassen stehen im busspezifischen Paket. Die Trennung der Schichten aus Kapitel 4 ist an dieser Stelle also eine Eigenschaft des Entwurfs und nicht der Dateiablage. Eine Anbindung an einen anderen Bus müsste die beiden Klassen auftrennen, was den Aufwand gegenüber Abschnitt 7.2 erhöht, wo die darunterliegende Operation ersetzt und die Klassen selbst unverändert verwendet werden. Spätere Stände holen diese Trennung nach, wie Abschnitt 8.3.4 beschreibt.

6.2 Hardwareaufbau

Der Hardwareaufbau ist kein eigener Beitrag dieser Arbeit, sondern der Träger, auf dem die Software läuft. Er besteht deshalb aus handelsüblichen Baugruppen, die über eine einfache Adapterplatine verbunden werden. Eine eigene Schaltungsentwicklung, etwa für die Spannungsversorgung, findet nicht statt.

6.2.1 Aufbau und Komponenten

Der Aufbau setzt die Anforderungen D1 bis D3 und S1 bis S3 aus Abschnitt 3.2 mit handelsüblichen Baugruppen um. Wie weit er sie erfüllt, bewertet Kapitel 7. Das Displaymodul bietet die geforderte grafische Darstellung und die Berührungseingabe bei einer für Embedded-Anwendungen typischen Auflösung, das ESP32-Board die nötigen Schnittstellen, genügend Speicher für die Anzeigebibliothek und eine Anbindung an den Bus des Hubs.

Der Aufbau besteht aus drei Teilen. Das LMS-ESP32-Board [43] bildet den Mikrocontroller der Anzeigeeinheit, wird über das Kabel eines LEGO-Sensors direkt an einen Port des Hubs angeschlossen und übernimmt die Busanbindung. Das Displaymodul enthält das TFT-Display mit dem Controller ILI9341 [40], einen kapazitiven Touchcontroller sowie einen Steckplatz für eine SD-Karte. Die Adapterplatine verbindet schließlich beide Baugruppen und enthält dabei keine aktiven Bauteile, sondern lediglich Steckverbinder und Leiterbahnen.

Signalseitig benötigt die in Abschnitt 6.3 beschriebene Anbindung nur eine UART-Verbindung auf zwei Leitungen des ESP32, die das LMS-ESP32-Board bereits zum Anschluss des Hubs führt. Die Adapterplatine muss sich deshalb nur um die Signale des Displays kümmern. Versorgt wird der Aufbau über den Port des Hubs, wobei der ESP32 die 8-V-Versorgung anfordert, da das Display mehr Strom benötigt, als die Logikversorgung liefert. Wie Abschnitt 3.4 beschreibt, verkürzt das die zulässige Länge der Kommandonamen auf fünf Zeichen.

6.2.2 Signalzuordnung und Aufbau der Adapterplatine

Das Display wird über **SPI** angesteuert, der Touchcontroller über **I2C**, wobei Abbildung 9 im Anhang die Schaltung der Adapterplatine zeigt. Die SPI-Leitungen MOSI, MISO, SCK und CS sind an den ESP32 geführt, ebenso der Touchcontroller über SDA und SCL.

Die Hintergrundbeleuchtung ist nicht an den ESP32 geführt und leuchtet dauerhaft. Die Interrupt-Leitung des Touchcontrollers ist zwar verbunden, wird jedoch nicht genutzt, da der Controller abgefragt wird (Abschnitt 6.7). Gleiches gilt für die Auswahlleitung der SD-Karte, die ebenfalls an den ESP32 geführt ist, während die SD-Karte selbst in dieser Arbeit nicht verwendet wird.

Bei der Pinbelegung ist GPIO 12 zu beachten. Der Pin wird beim Start des ESP32 ausgelesen und legt die Spannung des Flash-Speichers fest. Solange nur das Display an MISO hängt, ist das unkritisch, da die Leitung beim Start kaum getrieben wird. Wird die SD-Karte genutzt, teilt sie sich diese Leitung und kann sie über ihren Pull-up-Widerstand beim Start auf High ziehen, sodass der ESP32 nicht mehr zuverlässig startet. Für eine spätere Nutzung der SD-Karte müsste MISO deshalb auf einen anderen Pin gelegt werden.

Die Platine wurde mit KiCad entworfen und extern gefertigt, wobei beide Baugruppen aufgesteckt werden und sich ohne Löten tauschen lassen. Eine integrierte Trägerplatine, die ESP32, Display und Versorgung auf einer Platine vereint, war ursprünglich als zweite Ausbaustufe geplant. Sie wurde aus Zeitgründen nicht umgesetzt und wird in Abschnitt 8.3.6 als Erweiterung aufgegriffen.

6.3 Busanbindung

Die Busanbindung wird nicht entworfen, sondern vorgefunden, denn beide Seiten verwenden PUPRemote [36] in Version 1.6, auf dem Hub als `PUPRemoteHub` und auf dem ESP32 als `PUPRemoteSensor`. Die Bibliothek setzt das **LPF2**-Protokoll um, über das der Hub mit angeschlossenen Sensoren kommuniziert.

Gegenüber dem Hub gibt sich der ESP32 als Ultraschallsensor aus, da der Hub nur Geräte mit bekannter Kennung annimmt. Beim Verbindungsaufbau handeln beide Seiten zunächst bei 2400 Baud die Eigenschaften des Geräts aus und wechseln anschließend auf 115200 Baud. Danach überwacht die Bibliothek die Verbindung selbstständig und baut sie nach einem Abbruch neu auf. Zusätzlich fordert der ESP32 die 8-V-Versorgung am Port an, da das Display mehr Strom benötigt, als die Logikversorgung liefert.

PUPRemote bildet benannte Kommandos auf die Modi des Busses ab und bringt dafür eine eigene Aufrufsemantik mit festen Formaten je Kommando mit, die hier jedoch nicht genutzt wird. Registriert ist stattdessen ein einziges Kommando `xfer`, dessen Format in beide Richtungen ein Block von 16 Byte ist, siehe Listing 1. Für PUPRemote besitzt dieser Block keine Struktur, sodass alles Weitere in den Schichten darüber liegt.

```
COMMAND = "xfer"

def register_interceptor(self):
    self.remote.add_command(
        self.COMMAND,
        from_hub_fmt=f"{{FRAME_SIZE}}s",
        to_hub_fmt=f"{{FRAME_SIZE}}s",
    )
```

Listing 1: Registrierung des einzigen Kommandos

Ein Austausch im Sinne von Abschnitt 5.2.1 ist auf dem Hub ein Aufruf von `remote.call`. Dabei wird der Block in den Modus des Kommandos geschrieben und anschließend der Inhalt desselben Modus zurückgelesen. Zwischen beidem wartet die Bibliothek eine Zeit, die sich je Aufruf übergeben lässt. Ohne Angabe sieht die Bibliothek 0 ms vor. Mit diesem Wert lief der Austausch während der Entwicklung nicht zuverlässig, weshalb er in der eingesetzten Kopie

der Bibliothek auf 100 ms angehoben wurde. Die Middleware übergibt keinen eigenen Wert und erhält damit diese 100 ms, was sich nach Abschnitt 7.7 unmittelbar auf die Dauer jedes Aufrufs auswirkt. Ist die Wartezeit zu kurz, hat der ESP32 den Block bis zum Lesen noch nicht verarbeitet, und der Lesevorgang liefert die vorige Antwort. Solche veralteten Antworten erkennen die darüberliegenden Schichten an Position und Art des Frames, wie Abschnitt 6.9 ausführt.

Auf dem ESP32 ruft ein Task der Ereignisschleife etwa jede Millisekunde `process` auf, wobei je Durchlauf höchstens ein Kommando bearbeitet wird. Für jedes eintreffende Kommando ruft PUPRemote den registrierten Rückruf auf. Die Bibliothek findet ihn, indem sie den Kommandonamen im Namensraum des Hauptprogramms auswertet. Das Hauptprogramm muss den Rückruf deshalb unter dem Namen `xfer` bereitstellen.

Zwei Grenzen der Busanbindung wirken sich auf den gesamten Entwurf aus. Die Blockgröße ist mit der Firmware des Hubs auf 16 Byte begrenzt, darüber treten Prüfsummenfehler auf. Außerdem dürfen Kommandonamen bei angeforderter 8-V-Versorgung höchstens fünf Zeichen lang sein, gegenüber elf ohne diese Anforderung, weil die Bibliothek dem Namen in diesem Fall sieben Byte anhängt, mit denen sie die Versorgung ankündigt. Oberhalb dieser Schicht ist davon nur noch die Blockgröße sichtbar, und zwar als Zahl, aus der Abschnitt 6.4 die Nutzlast ableitet.

6.4 Transportschicht

Die Transportschicht setzt den Entwurf aus Abschnitt 5.2.2 in konkrete Feldbreiten um. Die folgenden Abschnitte beschreiben den Aufbau eines Frames, die Zerlegung einer Nachricht in Frames und die Serialisierung der Nutzlast.

6.4.1 Frame

Die Blockgröße ist durch die Busanbindung vorgegeben, da oberhalb von 16 Byte je Modus nach Abschnitt 3.4 Prüfsummenfehler auftreten. Damit steht die in Abschnitt 5.2.3 eingeführte MTU fest, und alle weiteren Größen leiten sich daraus ab.

Der Header belegt drei Byte.

Tabelle IV: Aufbau des Frame-Headers

Byte	Bits	Inhalt
0	8	Kennung der Nachricht
1	8	Position innerhalb der Nachricht
2	4 obere	Art des Frames
2	4 untere	Länge der Nutzlast in Byte

Tabelle IV zeigt den Aufbau des Headers, der für die Nutzlast 13 Byte je Frame übrig lässt.

Diese Aufteilung erfüllt die Bedingungen aus Abschnitt 5.2.3. Bei einem Header von drei Byte ergibt Formel 1 eine Nutzlast von 13 Byte, und Formel 2 verlangt dafür ein vier Bit breites Längenfild. Die obere Hälfte des dritten Bytes enthält die Art des Frames, die untere Hälfte die Länge der Nutzlast, sodass das Längenfild den Header nicht vergrößert. Ein Header von zwei Byte wäre nur möglich, wenn Kennung oder Position schmaler ausfielen, was die Anzahl unterscheidbarer Nachrichten oder die maximale Nachrichtenlänge verringern würde.

Listing 2 zeigt die Umwandlung von Bytes in einen Frame, bei der auch eine zu kurze Eingabe abgefangen und das dritte Byte in Art und Länge aufgeteilt wird.

```

@classmethod
def from_bytes(cls, data):
    if data is None or len(data) < cls.HEADER_SIZE:
        return cls.create_error(payload=b"short")

    g_id, frame_nr, b2 = unpack(cls._header_fmt, bytes(data[:cls.HEADER_SIZE]))
    length = b2 & 0x0F
    start = cls.HEADER_SIZE

    return cls(g_id, frame_nr, b2 >> 4, data[start:start + length])

```

Listing 2: Funktion zur Erzeugung eines Frames aus Bytes

Gültig bleibt die Aufteilung, solange die Nutzlast 15 Byte nicht überschreitet, da das Längenfeld keine größeren Werte darstellen kann. Bei einem Header von drei Byte entspricht das einer Blockgröße von höchstens 18 Byte. Größere Blöcke erfordern ein breiteres Längenfeld. Bis 34 Byte genügt ein Bit mehr, das sich dem Feld für die Art des Frames entnehmen ließe, solange keine neunte Art hinzukommt, wie Abschnitt 8.3.2 ausführt. Darüber wächst der Header auf vier Byte. Listing 3 zeigt die umgekehrte Umwandlung eines Frames in Bytes, bei der eine zu lange Nutzlast abgewiesen wird.

```

def to_bytes(self):
    n = len(self.payload)
    if n > MAX_PAYLOAD:
        raise ValueError("payload grew beyond {} bytes: {}".format(MAX_PAYLOAD, n))

    b2 = (self.opcode << 4) | n

    return pack(self._header_fmt, self.g_id, self.frame_nr, b2) + self.payload

```

Listing 3: Funktion zur Umwandlung eines Frames in Bytes

Die Position hat durch ihre Breite von acht Bit einen Wertebereich von 0 bis 255. Der Wert 0xFF ist für Frames reserviert, die sich auf keine Position innerhalb einer Nachricht beziehen. Für Nutzlastframes stehen damit 255 Werte zur Verfügung, was nach Formel 5 die Länge einer Nachricht auf $13 \cdot 255 = 3315$ Byte begrenzt. Dieser Wert ist zugleich die nach K5 geforderte Obergrenze des Puffers.

Für die Art des Frames sind von 16 darstellbaren Werten acht belegt (siehe Tabelle V). Acht Werte kämen gerade noch mit drei Bit aus, allerdings bliebe dann kein Wert für eine weitere Art frei. Das eingesparte Bit würde außerdem nichts bringen, denn im Längenfeld könnte es Werte bis 31 darstellen, obwohl die Nutzlast 13 Byte nie überschreitet. In der Position würde es über die Bytegrenze hinausreichen und Bitoperationen über zwei Byte erfordern. Die Breite von vier Bit ist daher eine pragmatische Entscheidung.

Tabelle V: Werte für die Art des Frames und ihre Bedeutung

Wert	Name	Bedeutung
0x0	DATA	Nutzlastframe, weitere folgen
0x1	DATA_LAST	Nutzlastframe, letzter der Nachricht
0x2	ACK	Bestätigung eines empfangenen Frames
0x3	NACK	Ergebnis noch nicht bereit oder nichts weiter vorhanden
0x4	ERR	Fehlermeldung, kann eine kurze Beschreibung tragen
0x5	READY	Anfrage, ob ein Ergebnis bereitliegt
0x6	NEXT	Anfrage nach dem Ergebnisframe an der angegebenen Position
0x7	DONE	Antwort auf READY, das Ergebnis liegt zur Abholung bereit

6.4.2 Message

Ein `Message`-Objekt enthält die Kennung und die serialisierte Nutzlast einer Nachricht. Es dient beiden Seiten als Container für die Nachricht, die über den Bus übertragen wird. Beim Senden wird die Nutzlast gesetzt, und das Objekt liefert daraus die Frames. Beim Empfangen werden Frames gesammelt, und daraus wird die Nutzlast rekonstruiert.

Die Zerlegung ist als Funktion über Position und Nutzlast umgesetzt. In einer früheren Version lieferte ein Generator die Frames mit fortlaufendem Zustand. Diese Variante wurde verworfen, da sie das Wiederholen einzelner Frames erschwerte.

Listing 4 berechnet den zugehörigen Abschnitt direkt aus dem Index, ohne einen Fortschritt mitzuführen. Dieselbe Position liefert damit immer denselben Frame, wie es Abschnitt 5.2.5 fordert. Der Rückkanal ist darauf angewiesen, da er Ergebnisframes einzeln über ihre Position anfordert und dieselbe Position mehrfach anfragen kann.

```
def frame_at(self, index: int):
    if index < 0 or index >= MAX_FRAMES:
        return None

    start = index * PAYLOAD_SIZE
    if start >= len(self.payload) and not (index == 0 and not self.payload):
        return None

    chunk = self.payload[start:start + PAYLOAD_SIZE]
    last = start + PAYLOAD_SIZE >= len(self.payload)

    return Frame(
        g_id=self.g_id,
        frame_nr=index,
        opcode=OP_DATA_LAST if last else OP_DATA,
        payload=chunk,
    )
```

Listing 4: Funktion zur Erzeugung von Frames aus einer Nachricht

Für das Einsammeln eingehender Frames gibt es zwei Wege, die sich in Speicherbedarf und Zusicherung unterscheiden. `collect_frame` behält jeden Frame, bis die Nachricht vollständig ist, und sortiert die Frames nach ihrer Position. Diese Variante ist von der Reihenfolge der Ankunft unabhängig, hält aber alle Frames einer Nachricht gleichzeitig im Speicher. `append_frame` hängt die Nutzlast direkt an und verwirft den Frame, was Speicher spart, aber die richtige Reihenfolge voraussetzt. `get_payload` liefert in beiden Fällen die vollständige Nutzlast.

Der Rückkanal verwendet den sammelnden Weg, der Hinweg den anhängenden. Die in Abschnitt 5.2.5 beschriebene Unabhängigkeit von der Reihenfolge ist damit nur auf dem Rückkanal umgesetzt. Für den hier betrachteten Bus genügt das, da er sequenziell arbeitet. Bei einem Übertragungsweg, der umsortiert, müsste auch der Hinweg auf den sammelnden Weg wechseln.

6.4.3 Serialisierung

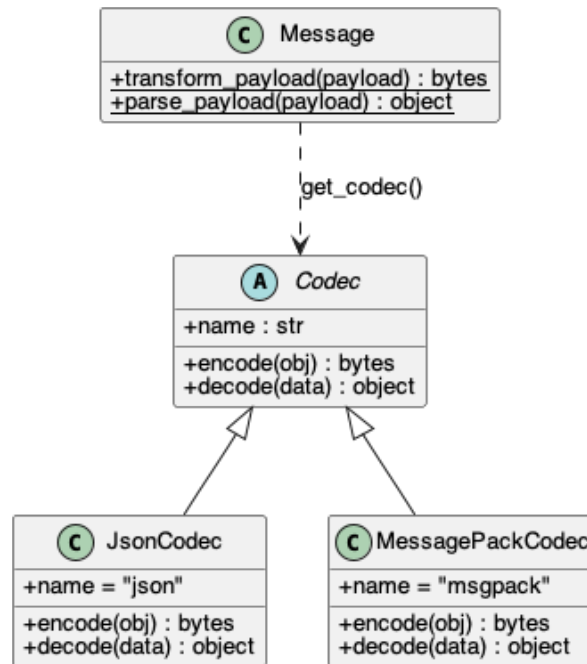


Abbildung 3: Klassendiagramm des Codecs

Die Wahl des Formats ist an einer Stelle gebündelt, an der ein `Codec` zwei Operationen festlegt: `encode` wandelt ein Python-Objekt in Bytes um, `decode` wandelt Bytes wieder in ein Python-Objekt um. Abbildung 3 zeigt die Schnittstelle des Codecs, ihre zwei Umsetzungen in der Referenzimplementierung sowie die Stellen, an denen sie verwendet werden. Eine Umsetzung muss dabei lediglich eine Bedingung erfüllen, nach der ein Wert beim Kodieren und anschließenden Dekodieren unverändert zurückkommen muss. Formal lässt sich das als

$$\text{decode} \circ \text{encode} = \text{id}_D \quad (6)$$

ausdrücken. Das Zeichen $\circ$ bezeichnet die Hintereinanderausführung, wobei die rechte Funktion zuerst angewendet wird. id_D ist die Identität auf der Menge D , also die Funktion, die jeden Wert unverändert zurückgibt. Formel 6 besagt damit, dass Kodieren und anschließendes Dekodieren für jeden Wert aus D dasselbe Ergebnis liefert wie gar keine Umwandlung.

D umfasst dabei nur die Werte, die das jeweilige Format unterstützt. Für die hier eingesetzte MessagePack-Umsetzung sind das Wahrheitswerte, Ganzzahlen im Bereich von 16 Bit, Zeichenketten, Bytefolgen sowie Listen und Abbildungen über diesen Typen. Außerhalb von D gilt die Eigenschaft nicht, ein Tupel etwa kommt als Liste zurück. Die Umkehrung gilt ebenfalls nicht, da sich derselbe Wert auf mehrere Arten kodieren lässt und eine fremde Bytefolge deshalb nicht zwingend unverändert zurückkommt.

Die MessagePack-Umsetzung deckt bewusst nur den Teil des Formats ab, den das System tatsächlich austauscht. Fließkommazahlen fehlen ebenso wie die Erweiterungstypen des Formats. Übertragen werden Pixelkoordinaten, Objektreferenzen und kurze Texte, und jeder weggelassene Typ verkleinert den Decoder, der auf beiden Seiten läuft, was dem dritten Kriterium aus Abschnitt 5.2.6 entspricht.

Enger als im Format vorgesehen ist auch der Wertebereich der Ganzzahlen. Er endet bei 16 Bit, obwohl MessagePack auch 32 und 64 Bit kennt. Der Grund liegt in der Laufzeitumgebung des Hubs, die ohne Unterstützung für lange Ganzzahlen übersetzt ist, sodass sich dort keine Konstante oberhalb von $2^{30} - 1$ darstellen lässt. Die Zweige des Encoders für größere Werte enthielten solche Konstanten und verhinderten das Laden des gesamten Moduls, unabhängig davon, ob je ein so großer Wert übertragen wird. Genau genommen reicht der Bereich von

–32768 bis 65535, da positive Werte ohne und negative mit Vorzeichen kodiert werden. Für die hier auftretenden Werte reicht das aus. Größere Werte weist der Encoder mit einem `OverflowError` ab, wie Tabelle IX zeigt.

Welcher Codec verwendet wird, legt eine einzige Zuweisung fest. Möglich ist dieser Wechsel nur, weil die Rahmung die Länge der Nutzlast ausdrücklich mitführt. Die binäre Kodierung erzeugt Nullbytes, und ohne Längenfeld würde das Abschneiden von Füllbytes Teile der Nutzlast entfernen.

Der JSON-Codec ist dabei nicht nur eine Vergleichsgröße, sondern war die erste Umsetzung. Zwei praktische Gründe sprachen dafür. Das Format steht in beiden Laufzeitumgebungen ohne zusätzliche Abhängigkeit zur Verfügung, und die Nutzlast ist im Klartext lesbar. Auf einem Bus, dessen Datenverkehr sich nur mit erheblichem Aufwand mitschneiden lässt, ist das bei der Fehlersuche ein deutlicher Vorteil, und solange die übrigen Schichten noch entstanden, wog die Lesbarkeit schwerer als eine kompakte Kodierung.

Mit zunehmender Reife änderte sich diese Gewichtung, denn bei dreizehn Byte Nutzlast je Frame wirkt sich jedes zusätzliche Zeichen nach Formel 3 unmittelbar auf die Anzahl der Übertragungen aus. Der Wechsel auf `MessagePack` betraf allein die Stelle, an der kodiert und dekodiert wird, während der entfernte Aufruf und das Objektmodell unverändert blieben. Der JSON-Codec ist dabei erhalten geblieben, weil ein mitgeschnittener Frame seine Nutzlast damit im Klartext zeigt. Wie sich die beiden Formate auf die Anzahl der Übertragungen auswirken, vergleicht Kapitel 7.

6.5 Kommunikationsschicht

Die Kommunikationsschicht ist auf beide Seiten verteilt. Auf dem Client, also dem Hub, bildet `PyBricksInterceptor.call()` einen Methodenaufruf auf eine Nachricht ab und wartet auf das Ergebnis, während auf dem Server, also dem ESP32, `MiddlewareInterceptor` die Nachricht entgegennimmt, sie der Verteilung übergibt und das Ergebnis zur Abholung bereitlegt. Beide Klassen übernehmen zugleich Aufgaben der Transportschicht, wie Tabelle III zeigt, weshalb die folgenden Abschnitte ausschließlich den Teil beschreiben, der zur Kommunikationsschicht gehört.

Die Umsetzung folgt dem Entwurf aus Abschnitt 5.3 und legt zwei Punkte fest, die dort nur als Prinzip beschrieben wurden. Erstens werden Argumente nach ihrer Position übertragen, wobei objektgebundene Methoden die Referenz des Objekts als erstes Argument erhalten. Zweitens werden Methoden über eine Kennung von 16 Bit angesprochen, die beide Seiten aus dem Methodennamen berechnen.

6.5.1 Aufrufnachricht

Die Nutzlast einer Nachricht tritt in drei Ausprägungen auf, je nach Richtung und Ausgang des Aufrufs.

Tabelle VI: Aufbau der Nutzlast je Ausprägung

Ausprägung	Schlüssel	Inhalt
<code>CommandPayload</code>	<code>s</code> , <code>c</code> , <code>a</code>	Scope, Methodenkenung, Liste der Argumente
<code>DataPayload</code>	<code>d</code>	Rückgabewert
<code>ErrorPayload</code>	<code>e</code> , <code>k</code>	Fehlerbeschreibung, optional die Fehlerart

Tabelle VI zeigt die drei Ausprägungen, deren Schlüssel jeweils einbuchstabig sind. Die Argumente stehen als Liste in der Reihenfolge der Parameter, wie sie die gemeinsamen Schnittstellen in `display/protocol` festlegen. Die Methode wird durch eine Kennung von 16 Bit bezeichnet, die Listing 5 aus ihrem Namen berechnet.

```
def name_hash(name):
    h = 5381
    for ch in name:
        h = ((h * 33) ^ ord(ch)) & 0xFFFF
    return h
```

Listing 5: Berechnung der Methodenkennung aus dem Methodennamen

Das Verfahren ist eine auf 16 Bit begrenzte Variante des bekannten djb2-Hashs [53]. Die eingebaute Funktion `hash` ist dafür nicht verwendbar, da CPython ihr Ergebnis je Prozess zufällig verändert und MicroPython es anders berechnet. Die Maskierung nach jedem Schritt hält alle Zwischenwerte unter 2^{30} , was auf dem Hub aus demselben Grund nötig ist wie die Begrenzung der Ganzzahlen in Abschnitt 6.4. Ein gebräuchlicher 32-Bit-Hash wie FNV würde diese Grenze überschreiten.

Die Breite von 16 Bit stellt eine Abwägung dar, denn MessagePack kodiert Werte bis 65535 in drei Byte, gegenüber 13 Byte für den Namen `create_label`. Eine Kennung von acht Bit wäre zwar ein Byte kürzer, bei zehn Methoden in einem Scope läge die Wahrscheinlichkeit einer Kollision dann jedoch bei etwa 16,3 %. Eine Kollision liegt vor, wenn zwei verschiedene Methodennamen denselben Hashwert ergeben, wobei die Wahrscheinlichkeit dafür mit der Anzahl der Methoden pro Scope steigt. Die folgende Gleichung

$$PKollision = 1 - \prod_{i=0}^{n-1} \left(1 - \frac{i}{N}\right) \quad (7)$$

schätzt die Wahrscheinlichkeit ab, dass bei n Methoden in einem Scope und N möglichen Kennungen mindestens zwei denselben Hashwert liefern. Sie ist nur eine Näherung, da sie gleichverteilte Hashwerte annimmt und djb2 nicht gleichverteilt ist. Nach Formel 7 liegt die Wahrscheinlichkeit einer Kollision bei 16 Bit und zehn Methoden bei etwa 0,069 %, bei 20 Methoden bei etwa 0,29 % und bei 50 Methoden bei 1,852 %.

Kollisionen sind somit möglich, werden jedoch beim Start des Servers erkannt. Der Server trägt dazu jede Methode in ein Dictionary ein, wobei der Hashwert als Schlüssel dient und eine zweite Methode mit demselben Hashwert zu einer Ausnahme führt, die den Start verhindert. Listing 6 zeigt die Funktion, die diese Indizierung vornimmt und in der Basisklasse `RPCDispatcher` liegt, von der alle Empfänger erben. Die genaue Struktur beschreibt Abschnitt 6.6.

```
def _index_methods(self):
    index = {}
    for name in dir(self):
        if name.startswith("_") or name in self.NOT_REMOTE:
            continue
        if not callable(getattr(self, name)):
            continue

        h = name_hash(name)
        if h in index:
            raise ValueError("hash collision in scope '{}': {} and {}".format(
                self.scope, index[h], name))
        index[h] = name
    return index
```

Listing 6: Funktion zur Registrierung einer Methode

Ergebnis und Fehler unterscheiden sich allein im Schlüssel. Der Client prüft, ob `e` vorhanden ist, und löst in diesem Fall eine Ausnahme aus. Welche Ausnahme das ist, bestimmt die Fehlerart unter `k`, wie Abschnitt 6.6 beschreibt. Antworten enthalten keinen Scope, denn der Server braucht ihn zur Auswahl des Empfängers, während eine Antwort immer an den einen Aufrufer zurückgeht, der auf sie wartet.

6.5.2 Verteilung eingehender Aufrufe

Die Zuordnung eines Aufrufs zur ausführenden Methode erfolgt in zwei Stufen. Zunächst wird anhand des Scopes ein Dispatcher ausgewählt, wobei der Scope die Art des Empfängers benennt, der den Aufruf ausführt, und im `CommandPayload` unter dem Schlüssel `s` mitgeführt wird. Die Dispatcher sind in einem `RPCDispatcherPool` registriert, wobei jeder einen eigenen Scope bedient. Die Registrierung erfolgt beim Start des Servers, und der Versuch, einen Scope ein zweites Mal zu belegen, löst eine Ausnahme aus.

Anschließend schlägt der ausgewählte Dispatcher die Methodenkennung aus `c` nach und ruft die Methode mit den Argumenten aus `a` auf. Dass diese Kennungen innerhalb eines Scopes eindeutig sind, prüft der Dispatcher beim Aufbau seines Index, wie Listing 6 zeigt.

Findet sich eine Kennung nicht im Index, löst der Dispatcher eine Ausnahme aus. Sie wird in eine Fehlermeldung überführt und erreicht den Client, womit die Zusicherung aus Abschnitt 5.2.7 auch für Aufrufe gilt, die es nicht gibt.

Aufruflbar ist damit jede öffentliche Methode eines Adapters. Das ist enger als ein Zugriff über den bloßen Namen, bei dem auch Attribute wie die Registratur erreichbar wären, aber weiter gefasst als die gemeinsame Schnittstelle in `display/protocol`. Der Adapter für Screens etwa erbt die Rückrufe der Anzeigebibliothek, die ebenfalls im Index stehen. Damit wächst auch n in Formel 7 über die Zahl der Methoden der Schnittstelle hinaus, und mit ihm die Wahrscheinlichkeit einer Kollision, die der Start dann meldet. Für die betrachtete Punkt-zu-Punkt-Verbindung, in die kein Dritter Nachrichten einspeisen kann, ist das hinnehmbar. Bei mehreren Teilnehmern müsste der Index auf die Methoden der Schnittstelle beschränkt werden.

6.5.3 Rückkanal

Nach der Bestätigung des letzten Frames ist ein Aufruf zwar vollständig übertragen, aber noch nicht ausgeführt. Der Server muss das Ergebnis erst berechnen und legt es anschließend in einem Ausgangspuffer ab, bis der Client es abholt.

Das Abholen erfolgt über denselben anfragegetriebenen Kanal wie die Übertragung der Aufrufe. Der Client fragt in einem festen Intervall mit `READY` an, ob ein Ergebnis bereitliegt. Der Server antwortet mit `DONE`, wenn das der Fall ist, und sonst mit `NACK`. Liegt das Ergebnis vor, holt der Client es mit `NEXT` Frame für Frame ab, wobei jede Anfrage die Position des nächsten Frames angibt. Der Server beantwortet jede Anfrage mit `frame_at(index)`, sodass eine wiederholte Anfrage denselben Frame liefert. Listing 7 zeigt die Beantwortung beider Anfragen.

```
if frame.opcode == OP_READY:
    if self.__is_result_ready__(frame.g_id):
        return Frame.create_done(g_id=frame.g_id).to_bytes()
    return Frame.create_no_ack(g_id=frame.g_id).to_bytes()

if frame.opcode == OP_NEXT:
    msg = self.outgoing.get(frame.g_id)
    if msg is None:
        return Frame.create_error(g_id=frame.g_id, payload=b"nomsg").to_bytes()

    res_frame = msg.frame_at(frame.frame_nr)
    if res_frame is None:
        return Frame.create_no_ack(g_id=frame.g_id, frame_nr=frame.frame_nr).to_bytes()

    return res_frame.to_bytes()

return Frame.create_error(g_id=frame.g_id, payload=b"badop").to_bytes()
```

Listing 7: Beantwortung der beiden Anfragen auf dem Server

Die Grenzen dieses Ablaufs sind als Konstanten festgelegt.

Tabelle VII: Grenzen des Rückkanals

Konstante	Wert	Bedeutung
RESULT_POLL_MS	50 ms	Abstand zwischen zwei READY-Anfragen, siehe Kapitel 7
MAX_RESULT_WAIT_MS	5000 ms	Wartezeit, nach der ein Aufruf als gescheitert gilt
MAX_FRAME_ATTEMPTS	8	Versuche je Frame beim Senden und je Position beim Abholen
MAX_KEPT_RESULTS	4	Ergebnisse, die der Server zur Abholung bereithält
MAX_EVENTS	10	gepufferte Ereignisse je Objekt

Der Server behält nur die jüngsten Ergebnisse und verwirft ältere, wodurch der Speicherbedarf nach K5 begrenzt bleibt. Da der Client streng nacheinander aufruft, fragt er ohnehin nur nach dem Ergebnis seines letzten Aufrufs. Ereignisse nutzen keinen eigenen Mechanismus. Der Server legt sie je Objekt in der Registratur ab und hält dort die letzten zehn vor, wobei bei Überlauf das älteste Ereignis verworfen und gezählt wird. Abgefragt werden sie über einen gewöhnlichen Aufruf wie `is_button_pressed`, der den Puffer dabei leert und die Anzahl der verworfenen Ereignisse mitliefert. Jede Abfrage kostet einen vollständigen Aufruf mit Übertragung, Wartezeit und Abholung. Ein Verlust ist für die Anwendung damit erkennbar, wie K3 in Abschnitt 3.3 es verlangt.

Wie viel Zeit ein Aufruf im Warten auf das Ergebnis verbringt, misst Kapitel 7. Reaktionen, die ohne diesen Weg auskommen, weil der Server sie selbst ausführt, behandelt Abschnitt 6.7.

6.6 Objektmodell

Das Objektmodell bildet die grafischen Elemente der Anzeige auf Objekte ab, die die Anwendung auf dem Hub wie lokale Objekte benutzt. Bildschirme, Beschriftungen und Schaltflächen heißen in der Umsetzung Screens, Labels und Buttons. Je Objekttyp gibt es eine Basisklasse in `display/protocol`. Sie wird auf dem Client von einem Stellvertreter erfüllt, der jeden Methodenaufruf weiterleitet, und auf dem Server von einem Adapter, der ihn ausführt. Die Adapter erben zusätzlich von `RPCDispatcher` und werden dadurch über ihren Scope erreichbar, wie Abschnitt 6.5.2 beschreibt. Jeder Stellvertreter hält ein `RemoteObject`, das die Referenz des Objekts führt und sie jedem Aufruf voranstellt. Dass die Stellvertreter es halten und nicht von ihm erben, liegt an Pybricks. Die Laufzeitumgebung unterstützt keine Mehrfachvererbung, und die eine mögliche Basisklasse belegt bereits die gemeinsame Schnittstelle des Objekttyps. Abbildung 4 zeigt diese Beziehungen beispielhaft.

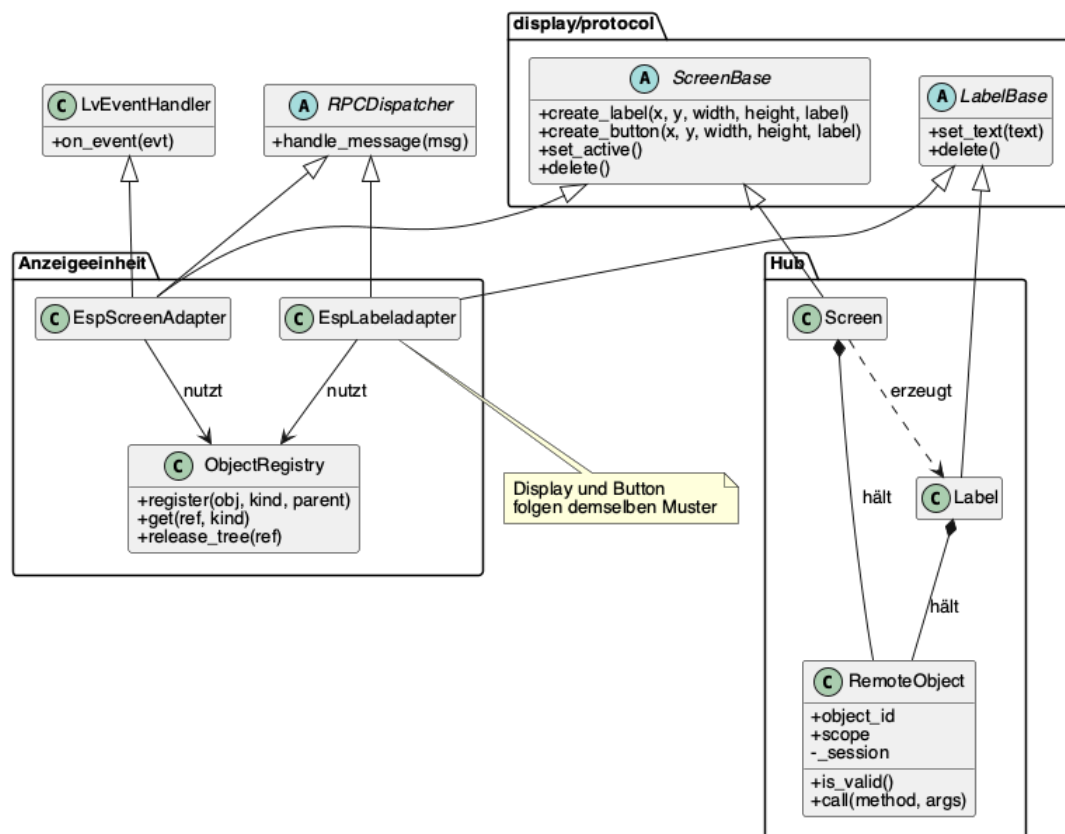


Abbildung 4: Klassen des Objektmodells auf beiden Seiten

6.6.1 Referenzen

Eine Referenz ist eine Ganzzahl von 16 Bit, die der Server beim Erzeugen eines Objekts vergibt und die aus zwei Teilen besteht. Die unteren zehn Bit bezeichnen einen Slot, also die Position des Objekts in der Tabelle der Registratur, während die oberen sechs Bit die Generation dieses Slots enthalten, einen Zähler, der bei jeder Freigabe um eins steigt. Dieses Verfahren ist als Generational Handle bekannt und wird unter anderem in der Spieleentwicklung für Referenzen auf wiederverwendete Plätze eingesetzt [54].

```
def _slot(self, ref):
    # gekürzt: die Prüfung, ob ref eine nicht negative Ganzzahl ist
    index = ref & SLOT_MASK
    if index >= len(self._slots):
        return None
    slot = self._slots[index]
    if slot[2] is None or slot[0] != ref >> SLOT_BITS:
        return None
    return slot
```

Listing 8: Auflösung einer Referenz in der Registratur

Ein Slot speichert neben dem Objekt und seiner Generation auch die Art des Objekts und die Referenz seines Parents. Die Art ist `screen`, `label` oder `button`, und jede Methode eines Adapters gibt beim Nachschlagen an, welche Art sie erwartet. Übergibt die Anwendung etwa die Referenz eines Screens an `set_text`, wird das erkannt, bevor auf das Objekt zugegriffen wird.

Zehn Bit erlauben 1024 gleichzeitig bestehende Objekte. Wie viele Objekte der Speicher des ESP32 tatsächlich trägt, wurde nicht gemessen und hängt mit der Lücke beim Speicherbedarf des Servers zusammen, die Kapitel 7 benennt. Die Breite von 16 Bit folgt aus Abschnitt 6.4, da der Codec Ganzzahlen nur bis zu dieser Größe überträgt und MessagePack eine Referenz

damit in höchstens drei Byte kodiert. Eine Referenz in Textform wie `obj_12` belegt dagegen sieben Byte.

6.6.2 Lebenszyklus

Objekte entstehen immer innerhalb eines übergeordneten Objekts, wobei Screens vom Display und Labels sowie Buttons von einem Screen erzeugt werden. Ein Button besteht aus dem Server aus zwei Objekten, dem Button selbst und dem Label für seine Beschriftung. Beide werden getrennt registriert, und der Stellvertreter auf dem Client erhält beide Referenzen. So lässt sich die Beschriftung über den Scope `label` ändern, ohne dass der Button-Adapter dafür eine eigene Methode braucht.

Die Registratur bildet dabei den Objektbaum der Anzeigebibliothek nach. Jeder Eintrag kennt seinen Parent, und `release_tree` gibt ein Objekt zusammen mit allen darin erzeugten Objekten frei. Das ist nötig, weil LVGL beim Löschen eines Objekts auch dessen Kinder löscht. Ohne den Baum würden deren Einträge in der Registratur bestehen bleiben und auf nicht mehr vorhandene Objekte verweisen.

Gelöscht wird über die Methode `delete()`, die jeder Objekttyp anbietet. Der Adapter löscht dabei zuerst das LVGL-Objekt und gibt anschließend die Einträge frei, wobei auch die Ereignisse des Objekts und alle Bindings entfallen, an denen es beteiligt ist. Den gerade angezeigten Screen kann die Anwendung hingegen nicht löschen, da LVGL nicht festlegt, was angezeigt wird, wenn der aktive Screen fehlt. Ein solcher Aufruf wird deshalb mit einem Fehler beantwortet.

`clear()` löscht alle Objekte auf einmal. Die Registratur allein zu leeren genügt dafür nicht, da LVGL die Objekte in einem eigenen Baum hält und sie nicht freigibt, wenn die letzte Python-Referenz entfällt. Der Server lädt daher zuerst einen leeren Screen und löscht anschließend jeden Screen der Registratur mit allem, was darauf liegt. Die Slots bleiben dabei erhalten und werden einzeln freigegeben, damit ihre Generationen weiterzählen.

6.6.3 Veraltete Referenzen

Nach dem Löschen eines Objekts kann dessen Stellvertreter auf dem Client weiter bestehen, wobei seine Referenz dann veraltet ist. Da freie Slots wiederverwendet werden, kann derselbe Slot inzwischen ein neues Objekt enthalten. Ohne Generation würde ein Aufruf über den alten Stellvertreter unbemerkt das neue Objekt verändern. Mit Generation passt die Referenz nicht mehr zum Slot, und der Aufruf wird abgewiesen.

Die Prüfung geschieht an zwei Stellen. Auf dem Server prüft die Registratur jede Referenz wie in Listing 8. Auf dem Client führt der `PyBricksInterceptor` eine Session, die bei `clear()` und bei einem Verbindungsabbruch endet. Jeder Stellvertreter merkt sich die Session, in der er entstanden ist. Gehört er zu einer früheren Session, löst er den Fehler aus, ohne eine Übertragung zu starten. Nach einem Verbindungsabbruch ist das die einzige verlässliche Prüfung, denn der ESP32 kann in der Zwischenzeit neu gestartet sein und seine Registratur von vorn aufbauen. Damit eine alte Referenz auch in diesem Fall nicht zufällig passt, beginnen die Generationen nach einem Start bei einem zufälligen Wert.

Beide Prüfungen melden denselben Fehler, wobei die Fehlernutzlast neben der Beschreibung unter `e` eine Fehlerart unter `k` trägt und der Client anhand dieser Art die passende Ausnahme auslöst. `StaleReferenceError` steht dabei für ein nicht mehr vorhandenes Objekt, `WrongKindError` für eine Referenz der falschen Art, während alle übrigen Fehler als `RemoteError` ankommen. Alle drei erben von `RuntimeError`, wodurch die Anwendung gezielt auf eine veraltete Referenz reagieren und beispielsweise die Oberfläche neu aufbauen kann, ohne den Fehlertext auswerten zu müssen.

Ganz ausgeschlossen sind Verwechslungen allerdings nicht. Nach 64 Freigaben desselben Slots wiederholt sich die Generation, und ein Stellvertreter, der die ganze Zeit bestanden hat, passt wieder. Löscht die Anwendung einen Screen, bleiben außerdem die Stellvertreter der Elemente darauf auf dem Client gültig, da der Client sie nicht kennt. Ihr nächster Aufruf erreicht den Server und wird erst dort abgewiesen.

6.6.4 Erweiterung um einen Objekttyp

Wie weit das Objektmodell erweiterbar ist, zeigt sich an einem zusätzlichen Element. Für einen Fortschrittsbalken sind vier Schritte nötig. Zunächst entsteht in `display/protocol` eine Basisklasse `ProgressBarBase` mit den Methoden, die beide Seiten teilen, etwa `set_value` und `delete`. Anschließend wird auf dem Client ein Stellvertreter ergänzt, der wie die übrigen ein `RemoteObject` hält, und auf dem Server ein Adapter, der von `RPCDispatcher` erbt und den Scope `progress` belegt. Der Adapter erzeugt das `LVGL`-Objekt und trägt es mit der Art `progress` in die Registratur ein.

Der vierte Schritt betrifft eine bestehende Klasse, denn ein Fortschrittsbalken wird auf einem Screen erzeugt. Der Screen-Adapter und der Screen-Stellvertreter brauchen deshalb je eine zusätzliche Methode `create_progress_bar`. Transport- und Kommunikationsschicht bleiben dagegen unberührt, da ein neuer Scope lediglich registriert und eine neue Methodenkennung beim Start indiziert wird.

Die Grenze der Erweiterbarkeit liegt damit im Objektmodell selbst. Ein neuer Objekttyp kostet drei neue Klassen und einen Eingriff in das erzeugende Objekt. Soll das neue Element außerdem Ereignisse liefern, kommt ein Rückruf bei der Anzeigebibliothek hinzu, wie ihn bisher nur Buttons anmelden.

6.7 Ansteuerung der Anzeige

Die Ansteuerung der Anzeige bildet auf dem Server die oberste Schicht aus Kapitel 4. Sie ist die einzige Stelle mit Kenntnis der Anzeigehardware und liegt in `display/server/driver`. Die Darstellung übernimmt `LVGL` in der MicroPython-Anbindung [55], die auch den Treiber für den Displaycontroller ILI9341 mitbringt. Das Display ist über `SPI` angebunden, der kapazitive Touchcontroller über `I2C`.

Aufgebaut wird die Anzeige vom `ESPDisplayAdapter`, der die `I2C`-Verbindung öffnet, den Touchcontroller erzeugt und ihn dem Displaytreiber übergibt, welcher ihn als Zeigegerät bei `LVGL` anmeldet. Anschließend reicht er das `LVGL`-Modul an die Adapter für Screens, Labels und Buttons weiter. Ausschließlich diese Adapter rufen `LVGL` auf, während die Schichten darunter die Bibliothek nicht kennen.

Das Zeichnen ist von der Ausführung der Aufrufe getrennt. Der Treiber startet beim Initialisieren eine Ereignisschleife, die in festem Takt über einen Hardware-Timer den Task-Handler von `LVGL` einplant, welcher geänderte Bereiche neu zeichnet und die Eingabegeräte abfragt. Gemessen liegt dieser Takt bei 40 ms, siehe Abschnitt 7.4. Ein Adapter ändert somit lediglich den Zustand eines Objekts, beispielsweise den Text eines Labels, wobei die Änderung erst beim nächsten Durchlauf des Task-Handlers sichtbar wird. Ein Aufruf kann deshalb bereits abgeschlossen sein, bevor die Änderung auf dem Display erscheint.

Der Touchcontroller wird regelmäßig abgefragt, seine Interrupt-Leitung dagegen nicht genutzt. `LVGL` ruft dazu bei jedem Durchlauf die Funktion `touch_read_cb` auf, die das Registerabbild des Controllers liest, wobei jede Abfrage eine `I2C`-Transaktion kostet, auch wenn keine Berührung vorliegt. Die Funktion darf zudem keine Ausnahme weitergeben, weshalb ein fehlgeschlagener Lesevorgang als fehlende Berührung gemeldet wird, wie Abschnitt 6.9 begründet.

Erkennt `LVGL` eine Berührung auf einem Button, löst es ein Ereignis aus. Der Screen-Adapter registriert dafür beim Erzeugen eines Buttons die Methode `on_event` für die Ereignisse `PRESSED` und `RELEASED`. `on_event` ermittelt über die Registratur die Referenz des Buttons, übersetzt den Ereigniscode in einen Namen wie `press` und ergänzt einen Zeitstempel. Das Ergebnis wird im Ereignispuffer des Objekts abgelegt, aus dem der Client es wie in Abschnitt 6.5 beschrieben abfragt.

Der Puffer beantwortet zwei verschiedene Fragen, und beide werden in einem Aufruf beantwortet. Die erste ist der aktuelle Zustand, also ob ein Button gerade gedrückt ist. Dafür hält die Registratur das jüngste Ereignis getrennt vor, das eine Abfrage überdauert. Die zweite ist, was seit der letzten Abfrage geschehen ist. Dafür wird der Puffer beim Abfragen geleert und die

Anzahl der Betätigungen gezählt. Ohne diese Trennung ginge eine kurze Berührung verloren, die vor der nächsten Abfrage schon wieder beendet ist, weil dann nur noch das Loslassen im Puffer stünde. Da eine Abfrage einen vollständigen Aufruf kostet, ist dieser Fall der Normalfall und nicht die Ausnahme.

Vor dem Ablegen prüft `on_event`, ob für das Ereignis ein Binding besteht. Ein Binding legt fest, dass ein Ereignis auf einem Objekt eine Aktion auf einem anderen auslöst. Der Client legt es einmal mit einem gewöhnlichen Aufruf an. Danach führt der Server die Aktion direkt im Rückruf aus, ohne dass eine Übertragung nötig ist. Das Ereignis wird unabhängig davon gepuffert und steht der nächsten Abfrage zur Verfügung. Der Client erfährt von der ausgeführten Aktion selbst jedoch nichts. Welcher Screen gerade angezeigt wird, weiß er nur, solange er die Ereignisse regelmäßig abfragt und die Bindings selbst nachhält.

Tabelle VIII: Aktionen, die an ein Ereignis gebunden werden können

Wert	Name	Wirkung auf das Ziel
0x1	ACT_SHOW_SCREEN	Screen laden und anzeigen
0x2	ACT_SHOW	Objekt sichtbar machen
0x3	ACT_HIDE	Objekt verbergen
0x4	ACT_TOGGLE	Sichtbarkeit umkehren

Die Menge der Aktionen ist absichtlich klein und fest vorgegeben, da sie weder Bedingungen noch Zustand oder Berechnungen enthält. Bindings bleiben somit auf Reaktionen beschränkt, die lediglich die Darstellung betreffen, beispielsweise den Wechsel zwischen zwei Screens, während die Logik der Anwendung weiterhin auf dem Client liegt. Unbekannte Aktionen weist `bind` ebenso zurück wie Referenzen, die veraltet sind oder die falsche Art haben (Abschnitt 6.6).

6.8 Ablauf eines vollständigen Aufrufs

Nachdem die vorigen Abschnitte die Schichten einzeln beschrieben haben, verfolgt dieser Abschnitt einen einzelnen Aufruf durch alle Schichten. Als Beispiel dient folgende Zeile

```
counter_label.set_text("Counter: {}".format(counter))
```

aus dem Beispielprogramm des Hubs (siehe Listing 9). Sie ändert den Text eines Labels und liefert keinen Rückgabewert, wobei der Einfachheit halber `counter = 42` angenommen wird.

Aufruf auf dem Client. `counter_label` ist ein Stellvertreter vom Typ `Label`, dessen Methode `set_text` zunächst prüft, ob er zur aktuellen Session gehört, und den Aufruf anschließend an `PyBricksInterceptor.call()` übergibt. Die Referenz des Labels wird dabei den Argumenten vorangestellt. Die Kommunikationsschicht bildet daraus einen `CommandPayload` mit dem Scope `label`, der Methodenkennung von `set_text` und der Argumentliste aus Referenz und Text, wobei die Nachricht die nächste freie Kennung erhält.

Zerlegung. `MessagePack` kodiert diese Nutzlast in 32 Byte, die bei 13 Byte Nutzlast je Frame nach Formel 3 in drei Frames zerfallen. Die ersten beiden tragen `DATA`, der letzte `DATA_LAST` mit den verbleibenden sechs Byte.

Übertragung. Der Client sendet jeden Frame einzeln und wartet auf dessen Bestätigung. Jeder Austausch ist ein Aufruf von `remote.call` und damit ein Round Trip über den Bus. Stimmen Position oder Art der Antwort nicht, war die Antwort veraltet oder der Frame ist verloren gegangen, und der Client wiederholt seine letzte Übertragung. Der Server legt jeden neuen Frame ab und bestätigt ihn sofort. Mit dem letzten Frame setzt er die Nutzlast zusammen und startet die Ausführung als eigenen Task der Ereignisschleife. Der letzte Frame wird bestätigt, bevor der Aufruf ausgeführt wird.

Ausführung. Der Task dekodiert die Nutzlast, wählt über den Scope den Label-Adapter aus und schlägt dort die Methodenkennung nach. `set_text` löst die Referenz in der Registratur auf, prüft dabei Generation und Art und setzt den Text des `LVGL`-Objekts. Sichtbar wird der

neue Text erst beim nächsten Durchlauf des Task-Handlers, wie in Abschnitt 6.7 beschrieben wurde. Das Ergebnis `None` wird als `DataPayload` kodiert, das der Server anschließend unter der Kennung im Ausgangspuffer ablegt.

Ein sofortiges Neuzeichnen im Adapter wäre möglich, würde aber die Ereignisschleife für die Dauer der Übertragung zum Display blockieren und damit auch die Bearbeitung weiterer Frames verzögern.

Abholen. Der Client fragt mit `READY` nach, ob das Ergebnis bereitliegt. Ob ein Ergebnis schon beim ersten Versuch vorliegt, hängt davon ab, ob die Ereignisschleife des Servers den Task bis dahin ausgeführt hat. Antwortet der Server mit `NACK`, wartet der Client 50 ms und fragt erneut. Auf `DONE` folgt eine Anfrage `NEXT` für Position 0, die den einzigen Ergebnisframe liefert, gekennzeichnet mit `DATA_LAST`, womit der Abholprozess endet.

Rückgabe. Der Client dekodiert die Nutzlast und gibt den Rückgabewert `None` an den Aufrufer zurück, falls das Ergebnis nicht als Fehler gekennzeichnet ist. Andernfalls löst er die passende Ausnahme aus, wie in Abschnitt 6.6 beschrieben.

Dieser Aufruf kostet damit mindestens fünf Round Trips. Nur drei davon transportieren den eigentlichen Aufruf mit seiner Nutzlast, während mindestens zwei weitere nur Steuerinformationen tragen und das Ergebnis abholen. Wie viele Round Trips in der Praxis nötig sind, misst Kapitel 7.

Das Aufrufschema folgt somit der in Abschnitt 5.2.1 beschriebenen Abfolge. Das Sequenzdiagramm Abbildung 2 zeigt die generischen Abläufe, die hier mit einem konkreten Beispiel gefüllt wurden. Die Abbildung zeigt auch die beiden Round Trips, die der Client für das Abholen des Ergebnisses benötigt.

6.8.1 Zustände eines Aufrufs

Der beschriebene Ablauf lässt sich für beide Seiten als Zustandsautomat angeben. Das ist genauer als eine Beschreibung im Fließtext, weil sich damit auch die Fälle festhalten lassen, die im Beispiel nicht vorkommen, also ausbleibende Bestätigungen, veraltete Antworten und abgebrochene Übertragungen. Eine Kante nennt jeweils den eintreffenden Frame und, nach dem Schrägstrich, die Reaktion darauf.

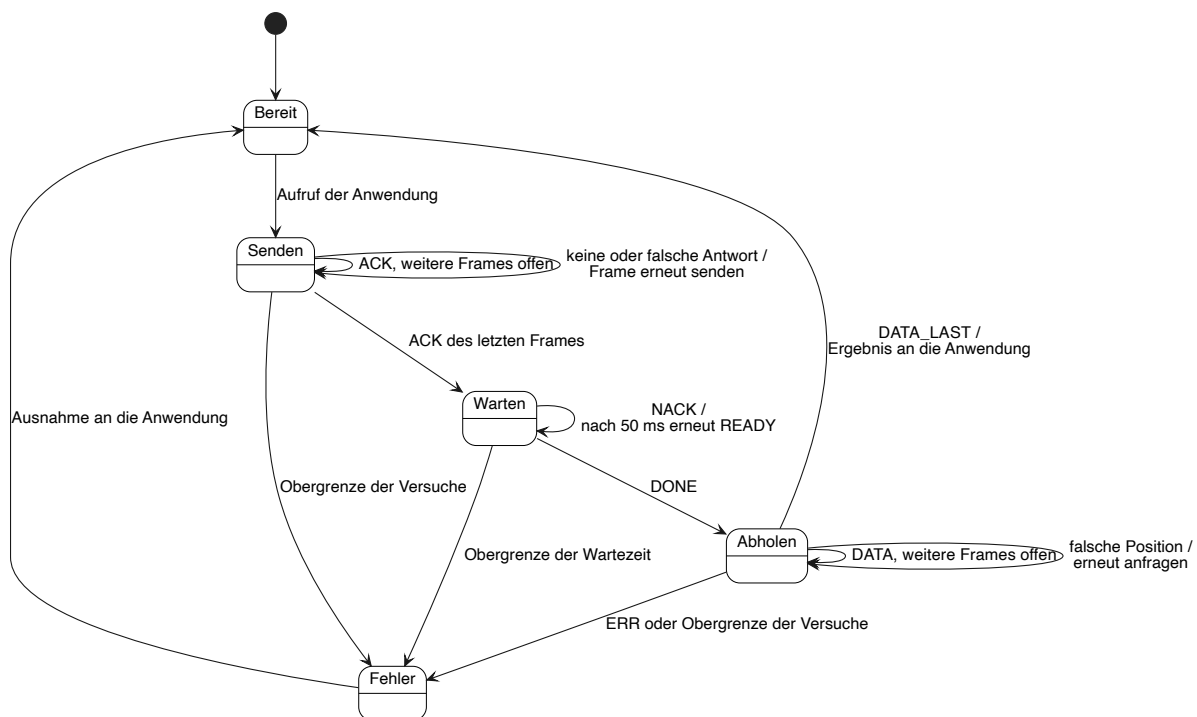


Abbildung 5: Zustände eines Aufrufs auf dem Client

Abbildung 5 zeigt den Client. Zwischen dem Verlassen von `Senden` und dem Eintreffen von `DONE` ist die Anwendung blockiert, und es ist stets nur ein Aufruf unterwegs. Jeder der drei Zustände `Senden`, `Warten` und `Abholen` besitzt eine eigene Obergrenze, nach deren Überschreiten der Aufruf in `Fehler` übergeht und die Anwendung eine Ausnahme erhält. Damit ist die Zusicherung aus Abschnitt 5.2.7 auch dann eingehalten, wenn die Gegenseite gar nicht mehr antwortet.

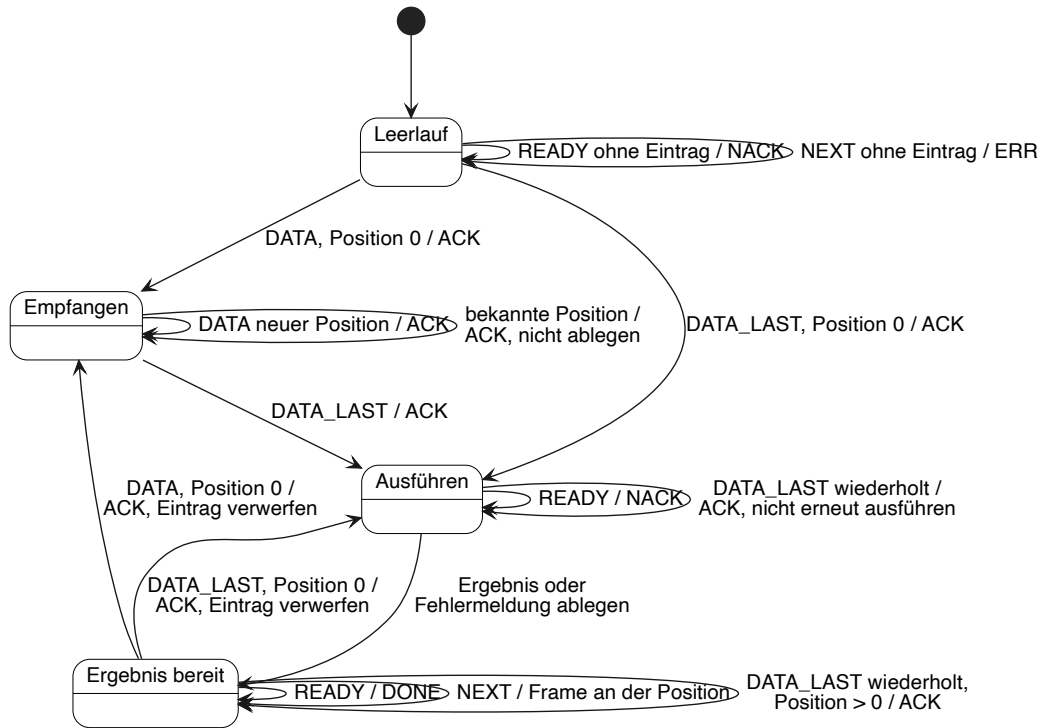


Abbildung 6: Zustände eines Aufrufs auf dem Server, je Nachrichtenennung

Abbildung 6 zeigt den Server. Zwei Kanten sind dort die eigentlichen Zusicherungen. Eine bereits bekannte Position wird bestätigt, aber nicht erneut abgelegt, und ein wiederholter letzter Frame wird bestätigt, ohne den Aufruf ein zweites Mal auszuführen. Zusammen ergibt das die in Abschnitt 5.2.7 beschriebene At-most-once-Semantik. Der Übergang von `Ergebnis bereit` zurück nach `Empfangen` ist der Punkt, an dem eine wiederverwendete Kennung ihren alten Eintrag verwirft, was Abschnitt 5.3.2 begründet.

Der Automat zeigt zugleich eine Lücke dieses Stands. Ein Frame an Position 0, der zugleich der letzte ist, führt aus `Ergebnis bereit` erneut nach `Ausführen`, da der Server die Kennung nicht mit der zuletzt empfangenen vergleicht. Wird eine Nachricht aus einem einzigen Frame nach einer veralteten Antwort wiederholt, führt er den Aufruf deshalb ein zweites Mal aus. Die Anwendung kann diesen Fall nicht auslösen, da mit der eingesetzten Kodierung schon der kürzeste Aufruf 18 Byte lang ist und zwei Frames belegt. Bei einem kürzeren Scope-Namen oder einem anderen Codec wäre er aber erreichbar. Eine Wiederholung des letzten Frames an einer späteren Position ist dagegen unkritisch, da diese Position bereits bekannt ist. Spätere Stände schließen die Lücke mit dem Vergleich der Kennung, den Abschnitt 5.2.5 beschreibt.

6.9 Nebenläufigkeit und Fehlerbehandlung

Die bisherigen Abschnitte beschreiben den Ablauf eines Aufrufs, der gelingt. Dieser Abschnitt behandelt, wie die Aufgaben auf dem Server nebeneinander laufen und wie Fehler behandelt werden.

6.9.1 Nebenläufigkeit

Auf dem Client gibt es keine Möglichkeit zur Nebenläufigkeit, sodass ein Aufruf die Anwendung blockiert, bis das Ergebnis vorliegt oder eine Obergrenze erreicht ist. Da immer nur ein Aufruf gleichzeitig unterwegs ist, braucht der Client weder Sperren noch Puffer für mehrere offene Aufrufe.

Auf dem Server laufen dagegen drei Tasks nebenläufig. Ein Task ruft etwa jede Millisekunde `process` von PUPRemote auf und beantwortet damit die Anfragen des Clients. `LVGL` zeichnet außerdem in einem festen Takt die Anzeige neu und fragt den Touchcontroller ab. Schließlich läuft jeder empfangene Aufruf in einem eigenen Task. Alle drei teilen sich eine einzige Ereignisschleife von `uasyncio` und damit einen Thread, weshalb sie sich nur an festen Stellen abwechseln, beispielsweise beim Warten eines der anderen Tasks. Dieses Vorgehen ist für speicherbeschränkte Geräte üblich, weil es den Stack je Aufgabe einspart, den ein eigener Thread benötigt [56].

Die Ausführung von Aufrufen lief in einer früheren Version in einem eigenen Thread. Das führte zu zwei Fehlern. Der Stack eines Threads wird vom Heap genommen, den `LVGL` bereits zum großen Teil belegt. Nach einigen Dutzend Aufrufen ließ sich deshalb kein neuer Thread mehr anlegen. Außerdem ist `LVGL` nicht reentrant [57], und der Task-Handler der Bibliothek lief teilweise im Thread mit seinem kleineren Stack. Dort lief der Stack im Rückruf des Touchcontrollers über. Mit einer gemeinsamen Ereignisschleife treten beide Fehler nicht mehr auf, da nie zwei Stellen gleichzeitig auf `LVGL` zugreifen.

Der Preis dafür ist, dass die Ereignisschleife während der Ausführung eines Aufrufs belegt ist. Anfragen des Clients werden in dieser Zeit nicht beantwortet, und die Anzeige wird nicht neu gezeichnet. Da der Client ohnehin auf das Ergebnis wartet, ist das hier unkritisch.

6.9.2 Fehlerbehandlung

Fehler werden in der Schicht behandelt, in der sie auftreten, und gelangen nach oben nur, wenn die Schicht sie nicht selbst behandeln kann. Tabelle IX fasst die Fälle zusammen.

Tabelle IX: Fehlerfälle und ihre Behandlung

Fehler	Schicht	Behandlung
Veraltete Antwort oder verlorene Bestätigung	Transport	Frame erneut senden, höchstens acht Versuche
Wiederholter Frame auf dem Server	Transport	bestätigen, aber nicht erneut ablegen, außer bei Nachrichten aus einem Frame
Abgebrochene Übertragung	Transport	Reste verwerfen, sobald Frame Nr. 0 einer neuen Nachricht eintrifft
Nachricht länger als 255 Frames	Transport	Aufruf scheitert auf dem Client, bevor etwas gesendet wird
Ganzzahl außerhalb von 16 Bit im Argument	Transport	<code>OverflowError</code> auf dem Client, bevor etwas gesendet wird
Ganzzahl außerhalb von 16 Bit im Ergebnis	Transport	Aufruf ist ausgeführt, Fehlermeldung an den Client, <code>RemoteError</code>
Ergebnis bleibt aus	Kommunikation	nach 5000 ms Ausnahme auf dem Client
Unbekannter Scope oder Methodenbezeichnung	Kommunikation	Fehlermeldung an den Client, <code>RemoteError</code>
Veraltete Referenz oder falsche Art	Objektmodell	Fehlermeldung mit Fehlerart <code>StaleReferenceError</code> oder <code>WrongKindError</code>
Ausnahme in einer Methode des Adapters	Objektmodell	Fehlermeldung an den Client, <code>RemoteError</code>
Verbindungsabbruch	Busanbindung	Session beenden, <code>OSError</code> an die Anwendung
Fehler beim Lesen des Touchcontrollers	Anzeige	als fehlende Berührung werten

Abschnitt 6.8.1 zeigt die folgenden Fälle als Zustandsübergänge, dieser Abschnitt ordnet sie den Schichten zu.

Die Transportschicht muss mit veralteten Antworten rechnen. Da der Client direkt nach dem Schreiben auch liest, kann er die Antwort auf die vorherige Anfrage erhalten, wenn der Server den Frame bis dahin nicht verarbeitet hat. Wie selten das im Betrieb vorkommt, zeigt Kapitel 7. Der Client erkennt sie an Position und Art des Frames und sendet diesen anschließend erneut.

Die Kennung wertet er dabei nicht aus, obwohl Abschnitt 5.2.2 sie als das Mittel benennt, an dem eine veraltete Antwort erkennbar wird. Sie adressiert in der Umsetzung nur die Anfrage nach dem Ergebnis, das der Client unter der Kennung seines Aufrufs anfordert. Für die Bestätigungen genügen Position und Art, solange der Client streng nacheinander sendet und auf jede Bestätigung wartet, denn eine veraltete Antwort gehört dann zum vorangegangenen Frame und trägt dessen Position. Offen bleibt der Fall, in dem beide Frames dieselbe Position tragen, etwa Frame 0 zweier aufeinanderfolgender Aufrufe. Solange eine Bestätigung nichts weiter transportiert, hat das keine Folgen. Abschnitt 8.3.1 greift die Prüfung dort auf, wo sie nötig wird. Der Server bestätigt eine Wiederholung, legt den Frame aber kein zweites Mal ab. Das gilt auch für den letzten Frame einer Nachricht, sofern sie mehr als einen Frame umfasst, wie Abschnitt 6.8.1 ausführt. Würde dieser bei einer Wiederholung die Ausführung erneut starten, lief derselbe Aufruf zweimal, beim zweiten Mal mit einer leeren Nachricht. Erst wenn eine Obergrenze aus Tabelle VII erreicht ist, gibt der Client auf und löst eine Ausnahme aus, die den betroffenen Frame und den Aufruf nennt.

Für den Server gilt die Zusicherung aus Abschnitt 5.2.7, nach der jeder Aufruf ein Ergebnis oder eine Fehlermeldung hinterlässt. Jede Ausnahme bei der Ausführung wird abgefangen und als `ErrorPayload` im Ausgangspuffer abgelegt. Lässt sich selbst das Ergebnis nicht kodieren, legt er eine kurze Fehlermeldung ab. Anfragen, die der Server nicht zuordnen kann, beantwortet

er mit einem Frame der Art `ERR`, etwa wenn nach dem Ergebnis einer unbekannten Nachricht gefragt wird.

Ein Verbindungsabbruch zeigt sich auf dem Client als `OSError: [Errno 19] ENODEV:` von `PUPRemote`. Der Client beendet dann die Session, da der Server in der Zwischenzeit neu gestartet sein kann, und reicht den Fehler an die Anwendung weiter. Ob sie es erneut versucht oder die Oberfläche neu aufbaut, entscheidet die Anwendung. Alle Stellvertreter aus der alten Session melden danach `StaleReferenceError`, wie Abschnitt 6.6 beschreibt.

Eine Stelle darf gar keinen Fehler weitergeben, nämlich der Rückruf, mit dem `LVGL` den Touchcontroller abfragt. Eine Ausnahme dort beendet die Ereignisschleife von `LVGL` dauerhaft, sodass die Anzeige nicht mehr neu gezeichnet würde, obwohl weiterhin Aufrufe ankommen. Ein fehlgeschlagener Lesevorgang wird deshalb als nicht vorhandene Berührung gewertet. Die Anwendung kann das nicht erkennen, da sie keinen Zugriff auf den Touchcontroller hat und nur die Ereignisse abfragen kann, die `LVGL` aus der Abfrage erzeugt.

7 Evaluation

Dieses Kapitel bewertet die Referenzimplementierung anhand der drei in Kapitel 1 genannten Kriterien. Zunächst wird geprüft, ob sich eine interaktive Oberfläche über die entwickelte Abstraktion umsetzen lässt, bevor der Übertragungsaufwand je Aufruf und die Reaktionszeit auf eine Berührung gemessen werden. Daran schließen sich der Speicherbedarf, eine kurze Bewertung des Hardwareaufbaus sowie die Diskussion der Ergebnisse an.

Gemessen wird mit zwei eigenen Programmen. Das erste läuft auf dem Client und ruft `set_text` mit Texten wachsender Länge auf. Je Aufruf gibt es eine Zeile mit der Länge der Nutzlast, der Anzahl der Frames, der Anzahl der Round Trips, den Wiederholungen sowie den Zeiten für Senden, Warten und Abholen aus. Je Textlänge werden 20 Aufrufe gemessen. Der gesamten Reihe gehen drei Aufrufe zum Aufwärmen voraus. Das zweite Programm misst die Reaktion auf eine Berührung und wird dabei durch Zeitstempel auf dem Server ergänzt.

Angegeben ist jeweils der Median, da die Verteilungen durch feste Takte nicht symmetrisch sind. Beide Seiten laufen auf dem Logging-Level INFO, da die Ausgabe synchron über die serielle Verbindung erfolgt und andernfalls die Messung bestimmen würde. Vor jedem Lauf wird zudem der Server neu gestartet. Da Client und Server keine gemeinsame Zeitbasis besitzen, wird auf jedem Gerät für sich gemessen und erst in der Auswertung zusammengerechnet.

7.1 Funktionaler Nachweis

Der funktionale Nachweis ergibt sich aus dem Beispielprogramm in Listing 9 und aus den Messläufen. Das Beispielprogramm baut zwei Screens mit Labels und Buttons auf, verknüpft Buttons über Bindings mit einem Screenwechsel, schreibt fortlaufend einen Zähler und fragt die Zustände der Buttons ab. Damit sind alle Bestandteile des Objektmodells im Einsatz, also das Erzeugen von Objekten, das Ansprechen über Referenzen, das Ändern von Zuständen, lokale Reaktionen und der Rückfluss von Ereignissen.

Die Messläufe ergänzen das um die Dauer, denn in vier Läufen mit zusammen 640 Aufrufen trat keine einzige Wiederholung auf und kein Aufruf schlug fehl. Die Läufe sind voneinander getrennt, vor jedem wurde der Server neu gestartet. Die Zusicherung aus Abschnitt 5.2.7, dass jeder Aufruf ein Ergebnis oder eine Fehlermeldung hinterlässt, wurde in diesen Läufen somit nie in Anspruch genommen.

Tabelle X ordnet die Kommunikationsanforderungen aus Abschnitt 3.3 dem erreichten Stand zu.

Tabelle X: Erfüllung der Kommunikationsanforderungen

Nr.	Anforderung	Stand	Begründung
K1	Strukturierte Nachrichten unbestimmter Länge	erfüllt	Nachrichten bis 3315 Byte, gemessen bis 164 Byte über 13 Frames
K2	Aufrufe mit zuordenbarem Ergebnis	erfüllt	das Ergebnis wird unter der Kennung des Aufrufs angefordert und unter dieser Kennung herausgegeben, siehe Listing 7
K3	Rückfluss von Ereignissen	erfüllt	der Puffer wird beim Abfragen geleert, siehe Abschnitt 7.4, der Überlauf wird gemeldet, siehe Abschnitt 7.2
K4	Antwortzeit unter 100 ms	teilweise	über ein Binding höchstens 59 ms, über den Client rund 1490 ms
K5	Begrenzter Speicherbedarf	teilweise	feste Obergrenzen für Nachrichtenlänge, Ergebnisse und Ereignisse, siehe Tabelle VII, auf dem Server nicht gemessen
K6	Unabhängigkeit vom Übertragungsweg	teilweise	zweiter Übertragungsweg in der Testumgebung, siehe Abschnitt 7.2, für einen anderen Bus zwei Klassen aufzutrennen, siehe Tabelle III, nachholt in Abschnitt 8.3.4

K3 und K4 werden in Abschnitt 7.4 belegt und in Abschnitt 7.7 eingeordnet.

7.2 Nachweis durch die Testumgebung

Drei Eigenschaften lassen sich am laufenden Aufbau nur schwer zeigen, weil sie eine zweite Verbindung, einen gezielt herbeigeführten Fehler oder einen Überlauf voraussetzen. Für sie existiert eine Testumgebung, die unter CPython läuft und dieselben Adapter, Stellvertreter und Interceptoren verwendet wie Steuer- und Anzeigeeinheit. Ersetzt sind nur die Hardware, die Anzeigebibliothek und die beiden Laufzeitumgebungen.

Unabhängigkeit vom Übertragungsweg (K6). Die Testumgebung enthält eine zweite Busanbindung, also eine zweite Umsetzung genau der einen Operation, die die Interceptor-Schnittstelle aus Abschnitt 5.2.1 nach unten verlangt. Sie befördert einen Block nicht über den Bus, sondern übergibt ihn im selben Prozess an die Gegenseite und liefert deren Antwort zurück. Oberhalb davon ist nichts angepasst: derselbe Interceptor, dieselben Stellvertreter, dieselbe Rahmung, derselbe Codec, dasselbe Objektmodell. Damit läuft der Entwurf nachweislich über zwei verschiedene Übertragungswege. Was der Nachweis nicht leistet, ist eine Aussage über einen zweiten realen Bus, denn die Testverbindung verliert keine Blöcke und kennt keine Laufzeit. Hinzu kommt, dass die Testverbindung die busspezifischen Klassen unverändert weiterverwendet. Für einen anderen Bus wären sie nach Tabelle III aufzutrennen, da sie neben der Busanbindung auch den Aufrufteil der Kommunikationsschicht enthalten. K6 ist deshalb nur teilweise erfüllt. Den Umbau, der das nachholt, beschreibt Abschnitt 8.3.4.

Zuverlässigkeit. Fehler, die im Betrieb selten auftreten, werden dort gezielt erzeugt. Eine Betriebsart der Testverbindung liefert jede n -te Antwort als die vorangegangene und stellt damit genau die veralteten Antworten nach, die Abschnitt 5.2.2 beschreibt. Im Stand `stand-projektarbeit` nutzt sie allerdings noch kein Test. Der Nachweis, dass der Aufruf trotz veralteter Antworten gelingt, dass Wiederholungen auftreten und dass ein wiederholter letzter Frame die Ausführung kein zweites Mal auslöst, liegt erst in einem späteren Stand vor, in dem das Ergebnis nach Abschnitt 8.3.1 mit der Bestätigung zurückkommt. Eine zweite Betriebsart lässt jede Übertragung scheitern und bildet den Verbindungsabbruch nach. Geprüft wird dort, dass der Aufruf mit einem Fehler endet und die Session endet, sodass jeder bestehende Stellvertreter ungültig wird. Auf dem realen Bus wurden Fehler dagegen nicht gezielt herbeigeführt. Belegt ist dort nur die eine veraltete Antwort aus Abschnitt 7.4.

Ereignisverlust (K3). Der Überlauf des Ereignispuffers tritt in den Messungen nicht auf, weil er mehr Berührungen zwischen zwei Abfragen verlangt, als von Hand auszulösen sind. In der Testumgebung wird der Puffer über seine Grenze hinaus gefüllt. Geprüft wird, dass der Verlust gemeldet und der Zähler mit der Abfrage zurückgesetzt wird, dass also die von K3 in Abschnitt 3.3 geforderte Erkennbarkeit auch im Grenzfall gilt.

Die Testumgebung deckt damit die Stellen ab, an denen die Messungen nichts aussagen können, und die Messungen decken ab, was sich nur am echten Bus zeigt. Tabelle XVI im Anhang nennt die Ordner, in denen beide liegen.

7.3 Übertragungsaufwand

Gemessen wird in vier Konfigurationen, die jeweils eine Größe verändern. Die Nutzlast je Frame beträgt 13 Byte im heutigen Stand und 7 Byte in der Vergleichsfassung, während als Serialisierung MessagePack [23] und die JSON zum Einsatz kommen.

Die Vergleichsfassung ist nicht der alte Quelltext, sondern der heutige mit einer geänderten Konstante. Der frühere Header belegte neun Byte und ließ sieben für die Nutzlast, und genau diese sieben werden hier eingestellt. Gemessen wird damit nicht das alte Format, sondern dessen Kosten, denn nach Formel 3 hängt die Anzahl der Frames allein von der nutzbaren Nutzlast je Frame ab und nicht davon, wie der Header aufgebaut ist. Der Vergleich trennt somit die Wirkung der Rahmungsbreite von der Wirkung des Formats.

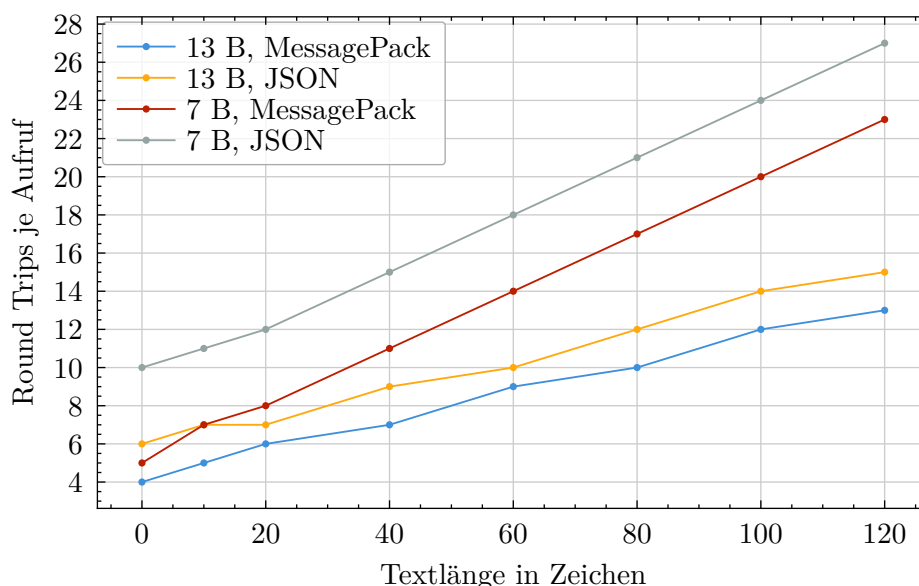


Abbildung 7: Round Trips je Aufruf über der Textlänge, Median aus je 20 Aufrufen

Tabelle XI fasst die vier Konfigurationen zusammen. Innerhalb einer Konfiguration und einer Textlänge ist die Anzahl der Round Trips über alle 20 Wiederholungen identisch, was aus Formel 3 folgt und durch die ausgebliebenen Wiederholungen bestätigt wird. Die Streuung liegt also allein in den Zeiten, nicht im Ablauf. Minimum und Maximum in Tabelle XI zeigen diese Streuung allerdings nicht, da sie über alle acht Textlängen gebildet sind und damit die Wirkung der Länge wiedergeben. Die Streuung je Round Trip bezieht der folgende Absatz.

Tabelle XI: Spanne der vier Konfigurationen über alle acht Textlängen, je 160 Aufrufe

Konfiguration	Round Trips	Median	Minimum	Maximum
MessagePack, 13 Byte	4 bis 13	920 ms	457 ms	1499 ms
MessagePack, 7 Byte	5 bis 23	1434 ms	571 ms	2642 ms
JSON, 13 Byte	6 bis 15	1085 ms	685 ms	1716 ms
JSON, 7 Byte	10 bis 27	1891 ms	1142 ms	3098 ms

Der wichtigste Befund ergibt sich aus dem Vergleich mit den gemessenen Zeiten. Ein Round Trip kostet in jeder der vier Konfigurationen und bei jeder der acht gemessenen Nachrichtenlängen 114,2 ms. Über alle 640 Aufrufe liegen das untere und das obere Quartil bei 114,1 ms und 114,2 ms, das Minimum bei 114,0 ms und die Standardabweichung bei 0,3 ms. Nur 20 der 640 Aufrufe liegen über 115 ms, der höchste bei 117,0 ms. Die Dauer eines Aufrufs ist somit das Produkt aus der Anzahl der Round Trips und einer festen Zeit je Round Trip, da sich weder die Länge der Nutzlast noch die Wahl des Formats darüber hinaus auswirken. Jeder Wert aus Abbildung 7 lässt sich deshalb unmittelbar in eine Dauer umrechnen, sodass der kürzeste gemessene Aufruf mit 4 Round Trips 457 ms benötigt und der längste mit 27 Round Trips im Median 3082 ms.

Die Anzahl der Round Trips folgt unmittelbar aus Formel 3. Sie setzt sich zusammen aus einem Round Trip je Frame der Nachricht, einer Anfrage `READY` und einem Round Trip je Frame des Ergebnisses. Für die gemessenen Aufrufe sind das $N + 2$ Round Trips, solange das Ergebnis in einen Frame passt. Das trifft auf drei der vier Konfigurationen zu, denn das Ergebnis belegt mit MessagePack vier und mit `JSON` als `{"d": null}` elf Byte. Nur bei `JSON` mit sieben Byte Nutzlast benötigt es zwei Frames, sodass dort $N + 3$ Round Trips anfallen.

Die beiden Einflussgrößen lassen sich daran trennen. Gegenüber dem heutigen Stand mit 13 Byte und MessagePack braucht die Fassung mit 7 Byte das 1,59-fache an Round Trips, die Fassung mit `JSON` das 1,21-fache und beides zusammen das 2,09-fache. Die Rahmung wiegt also deutlich schwerer als das Format. Der Grund ist in den Daten sichtbar. `JSON` erzeugt bei denselben Aufrufen eine um 22 bis 23 Byte längere Nutzlast, was nach Formel 3 nicht bei jeder Textlänge eine Blockgrenze überschreitet. Ein kleinerer Header verschiebt dagegen alle Blockgrenzen zugleich. Die Erwartung aus Abschnitt 5.2.4 wird damit für diesen Aufruf bestätigt. Allgemein gilt das nicht, denn der Mehraufwand von `JSON` besteht hier fast nur aus Schlüsseln und Klammern und ist deshalb nahezu fest. Bei Aufrufen mit vielen Argumenten oder Listen wächst er mit der Nachricht, und das Format gewönne an Gewicht.

Eine Grundlinie für diese Zahlen liefert PUPRemote selbst. Ein eigenes Kommando je Operation, etwa `set_text` mit fester Formatangabe für ein vorab festgelegtes Label, kommt mit einem einzigen Round Trip aus, also mit 114 ms statt der 457 ms des kürzesten gemessenen Aufrufs. Der Text ist dann aber auf die 16 Byte eines Modus begrenzt, jede Operation belegt einen der höchstens 16 Modi, und Objekte, die erst zur Laufzeit entstehen, lassen sich nicht ansprechen. Die drei zusätzlichen Round Trips sind damit der Preis der Allgemeinheit, die K1 und das Objektmodell verlangen. Mit dem Ergebnis in der Bestätigung nach Abschnitt 8.3.1 schrumpft er auf einen. Gemessen wurde die Grundlinie nicht, sie folgt aus der Dauer eines Round Trips.

Die Nutzlasteffizienz nach Formel 4 fällt dabei niedrig aus. Ein Aufruf mit 21 Byte Nutzlast belegt zwei Frames und somit 32 übertragene Byte, was einem Wert von 0,66 entspricht. Bei 142 Byte über elf Frames ergeben sich 0,807 und damit nahezu der theoretische Höchstwert, den ein Header von drei Byte zulässt, nämlich $13/16 = 0,8125$ bei vollständig gefülltem letztem Frame. Für die Dauer eines Aufrufs bleibt gleichwohl die Anzahl der Frames maßgeblich und nicht ihre Füllung.

Unabhängig von der Nachrichtenlänge kostet jeder Aufruf zwei zusätzliche Round Trips für das Abholen des Ergebnisses und somit 228 ms, was beim kürzesten Aufruf bereits die Hälfte der gesamten Dauer ausmacht. Auffällig ist zudem, dass das Abfrageintervall von 50 ms nie

zum Tragen kommt, da die gemessene Wartezeit in 595 der 640 Aufrufe genau 114 ms und im Höchstfall 129 ms beträgt, also stets einen einzigen Round Trip. Der Server ist folglich mit der Ausführung fertig, bevor der Client zum ersten Mal nachfragt.

7.4 Reaktionszeit einer Berührung

Die zweite Messung betrachtet den Weg von einer Berührung bis zur sichtbaren Reaktion und vergleicht dabei die beiden Wege aus Abschnitt 6.7. Über ein Binding führt der Server die Reaktion selbst aus, während über den Client das Ereignis abgefragt und mit einem zweiten Aufruf beantwortet wird. Auf dem Server werden dafür drei Zeitpunkte je Berührung festgehalten, nämlich die erste erkannte Berührung, das verarbeitete Ereignis und das abgeschlossene Neuzeichnen, auf dem Client hingegen die Zeit vom Beginn eines Abfragedurchlaufs bis zum Ende der Reaktion.

Der Touchcontroller wird von der Anzeigebibliothek alle 40 ms abgefragt, gemessen über 39 Berührungen mit Werten zwischen 38 und 42 ms. Eine Berührung wird deshalb im Mittel nach 20 ms und spätestens nach 42 ms bemerkt. Das gilt, solange die Ereignisschleife frei ist. Bearbeitet sie gerade einen Aufruf, verlängert sich der Abstand zwischen zwei Abfragen, in den Läufen über den Client bei 13 von 62 Berührungen auf 57 bis 89 ms. Für den Weg über den Client ist die Erkennung in Tabelle XII deshalb eher zu knapp angesetzt, an der Größenordnung ändert das nichts. Von dort bis zum verarbeiteten Ereignis vergehen 4 ms. Das Neuzeichnen dauert anschließend 6 ms, wenn ein Binding die Anzeige verändert hat, und 7 ms, wenn die Änderung von einem Aufruf des Clients stammt. Über ein Binding ist die Reaktion gemessen 13 ms nach der erkannten Berührung sichtbar, im Bereich von 10 bis 17 ms. Zusammen mit der Erkennung ergibt das im Mittel 33 ms und im ungünstigsten Fall 59 ms.

Auf dem Weg über den Client dauert allein die Reaktion 1161 ms, da sie aus zwei Aufrufen mit je 5 Round Trips besteht, nämlich dem Abfragen des Ereignisses und dem Ändern des Textes. Hinzu kommt die Zeit, bis überhaupt abgefragt wird, die nicht allein von der Pause in der Schleife der Anwendung abhängt, denn während eines laufenden Aufrufs kann nicht abgefragt werden. Der wirksame Abstand zwischen zwei Abfragen beträgt daher 572 ms zuzüglich der eingestellten Pause.

Tabelle XII: Zeit von der Berührung bis zur sichtbaren Reaktion

Teilstrecke	über ein Binding	über den Client	Herkunft
Erkennung durch die Anzeigebibliothek	im Mittel 20 ms	im Mittel 20 ms	abgeleitet
Verarbeitung des Ereignisses	4 ms	4 ms	gemessen
Warten auf die nächste Abfrage	entfällt	im Mittel 296 ms	abgeleitet
Abfrage und Reaktion	entfällt	1161 ms	gemessen
Neuzeichnen	6 ms	7 ms	gemessen
Summe im Mittel	33 ms	rund 1490 ms	berechnet
Summe im ungünstigsten Fall	59 ms	rund 1800 ms	berechnet

Die Spalte Herkunft unterscheidet, ob ein Wert unmittelbar gemessen, aus einer Messung abgeleitet oder aus den übrigen Zeilen berechnet wurde. Die gemessenen Werte sind Mediane. Die Verarbeitung des Ereignisses streut über 39 Berührungen von 4 bis 7 ms, das Neuzeichnen von 5 bis 11 ms über ein Binding und von 7 bis 8 ms über den Client. Die beiden Zeilen addieren sich nicht zu den Summen darunter, denn Mediane sind nicht additiv. Maßgeblich ist die je Berührung gemessene Zeit von der Erkennung bis zum fertigen Bild, deren Median bei 13 ms liegt und die zwischen 10 und 17 ms streut. Die Summen beruhen auf diesem Wert und nicht auf der Addition der Einzelmediane. Die Reaktion über den Client liegt in allen drei Läufen bei 1160 bis 1177 ms, mit einem einzelnen Ausreißer von 1277 ms, der weiter unten erklärt wird.

Abgeleitet sind die beiden Wartezeiten: die Erkennung ist der halbe Abtasttakt, das Warten auf die nächste Abfrage die halbe Summe aus Aufrufdauer und eingestellter Pause. Berechnet sind die Summen, die sich aus den Zeilen darüber ergeben.

Die Werte in Tabelle XII gelten für eine Pause von 20 ms in der Schleife der Anwendung. Gemessen wurde zusätzlich mit 0 ms und 1000 ms, wobei die Reaktion selbst in allen drei Fällen gleich lang ausfällt, da die Pause vor der Abfrage liegt. Bei einer Pause von 1000 ms steigt die Summe im Mittel auf rund 1980 ms, weil sich der Abstand zwischen zwei Abfragen entsprechend verlängert.

Dieser Abstand bestimmt zugleich, welche Berührungen überhaupt ankommen. In einer früheren Fassung lieferte eine Abfrage nur das jüngste Ereignis eines Objekts. Wurde ein Button gedrückt und wieder losgelassen, bevor die nächste Abfrage stattfand, stand im Puffer zuletzt das Loslassen, und die Berührung war vollständig verloren. Erkannt wurde sie nur, wenn sie zum Zeitpunkt der Abfrage noch andauerte, was bei einer Pause von 1000 ms rechnerisch ein Halten von etwa 1,5 Sekunden erforderte, nämlich eine Abfrage von 457 ms zuzüglich der Pause.

Diese Beobachtung war der Anlass für die in Abschnitt 6.7 beschriebene Trennung von Zustand und aufgelaufenen Ereignissen. Um sie zu prüfen, wurde die Messung wiederholt. Das Messprogramm auf dem Client zählt dabei nicht die Betätigungen, sondern läuft, bis zwanzig Abfragen eine Berührung melden. Wie oft der Button tatsächlich betätigt wurde, hält unabhängig davon die Anzeigeeinheit fest, die je erkannter Berührung eine Zeile ausgibt. Beide Zählungen lassen sich damit gegeneinander prüfen. Tabelle XIII stellt die Ergebnisse zusammen.

Tabelle XIII: Erkannte Berührungen je Variante, gezählt auf beiden Geräten

Pause	Abfragen mit Treffer	Berührungen laut Server	gezählte Be- tätigungen	verloren	Wiederholun- gen
0 ms	20	20	20	0	0
20 ms	20	22	22	0	1
1000 ms	20	20	20	0	0

In allen drei Läufen stimmt die Anzahl der auf dem Client gezählten Betätigungen mit der Anzahl der Berührungen überein, die die Anzeigeeinheit verzeichnet hat. Kein Ereignis ging also verloren, und zwar unabhängig von der Pause in der Schleife. Im Lauf mit 20 ms Pause wurde der Button 22-mal betätigt, gemeldet wurde das in nur 20 Abfragen, weil zweimal zwei Betätigungen in dasselbe Abfrageintervall fielen und die zugehörige Antwort beide mitführte. Genau das war in der früheren Fassung nicht möglich. Anforderung K3 ist damit erfüllt, und das Halten eines Buttons ist nicht mehr nötig.

Der Nachweis gilt für die gemessene Ereignisrate von Hand ausgelöster Betätigungen. Dass der Puffer auch bei höherer Rate meldet, was er verwirft, zeigt nicht diese Messung, sondern der Überlaufstest in Abschnitt 7.2.

Bezahlt wird das mit einem zusätzlichen Round Trip je Abfrage. Die Antwort trägt nun den Zustand, die Anzahl der Betätigungen und die Anzahl der verworfenen Ereignisse, passt damit nicht mehr in einen einzigen Frame und benötigt zwei Frames für den Rückweg. Eine Abfrage kostet dadurch 5 statt 4 Round Trips, also 572 ms statt 457 ms. Um denselben Round Trip verlängert sich die Reaktion über den Client, da sie mit der Abfrage beginnt.

Die Wiederholung im Lauf mit 20 ms Pause ist der einzige Fall einer veralteten Antwort in allen Messungen mit 100 ms Wartezeit. Sie kostete einen Round Trip, die betroffene Reaktion dauerte 1277 ms statt 1161 ms. Der Mechanismus aus Abschnitt 5.2.7 greift damit auch im Betrieb und nicht nur im Entwurf.

7.5 Ressourcenverbrauch

Gemessen wird auf dem Hub mit `gc.mem_alloc()`, also dem vom Speicherverwalter belegten Speicher, jeweils nach einer Speicherbereinigung mit `gc.collect()`. Der erste Messpunkt liegt nach dem Aufbau der Oberfläche aus einem Screen und einem Label und nach drei Aufrufen zum Aufwärmen, der zweite nach den 160 gemessenen Aufrufen. Vor der Messreihe sind 16128 Byte belegt, danach 16272 Byte, und die Werte sind in allen vier Läufen gleich. Darin enthalten ist der geladene Programmcode, dessen gebündelter Quelltext im Messprogramm rund 72 KB umfasst. Die 160 Aufrufe hinterlassen damit 144 Byte, also weniger als ein Byte je Aufruf. Da der Speicherverwalter in Blöcken von mindestens 16 Byte zuteilt, bleibt je Aufruf kein Objekt dauerhaft zurück. Was die Messung nicht erfasst, ist die Spitze während eines Aufrufs, denn die Bereinigung vor jedem Messpunkt gibt den kurzzeitig belegten Speicher wieder frei. Ebenso fehlt ein Messpunkt vor dem Aufbau, sodass sich der Bedarf der Oberfläche selbst nicht beziffern lässt.

Der Speicherbedarf auf dem Server wurde dagegen nicht gemessen. Das ist die empfindlichste Lücke dieser Auswertung, denn der Heap des Servers war nach Abschnitt 6.9 die Ursache dafür, dass sich Aufrufe nicht in eigenen Threads ausführen lassen. Eine belastbare Aussage müsste den freien Heap an vier Stellen vergleichen, nämlich vor dem Laden der Anzeigebibliothek, nach deren Initialisierung, nach dem Aufbau der Oberfläche und nach einer längeren Folge von Aufrufen. Ohne diese Messung bleibt K5 auf der Seite, auf der der Speicher knapp ist, unbelegt.

7.6 Bewertung des Hardwareaufbaus

Die Adapterplatine ist ein Zwischenschritt weg vom Steckbrett, auf dem der Aufbau in einer ersten Vorführung noch beruhte. Da sich Display und Mikrocontroller aufstecken lassen, beschränkt sich die Hardware auf das Nötigste, nämlich auf die Verbindung beider Baugruppen. Die Platine führt alle benötigten Leitungen auf Buchsenleisten, die sich ohne besondere Ausrüstung löten lassen. Dadurch lassen sich weitere Exemplare zügig aufbauen, und die aufgesteckten Baugruppen bleiben für andere Zwecke verwendbar. Die Anforderungen D1, D2, S1 und S3 aus Abschnitt 3.2 erfüllt der Aufbau damit, wie Abschnitt 6.2 zeigt. Für D3 und S2 liegt nur die Beobachtung vor, dass die Anzeigebibliothek auf dem ESP32 läuft. Energieaufnahme und Speicherbedarf wurden nicht gemessen.

Steckverbindungen sind allerdings keine gute Wahl für die Signalintegrität und zählen im Betrieb zu den häufigsten Fehlerquellen. Für einen Aufbau, der im Labor erprobt und zwischen Versuchen umgesteckt wird, ist das vertretbar, für einen dauerhaften Einsatz dagegen nicht. Abbildung 8 zeigt beide Seiten der Platine als dreidimensionale Darstellung aus dem Entwurfswerkzeug. Neben den Buchsenleisten sind dort die Versionsnummer der Platine sowie die Bezeichnungen der Bauteile aufgedruckt, was die Bestückung und die spätere Zuordnung erleichtert.

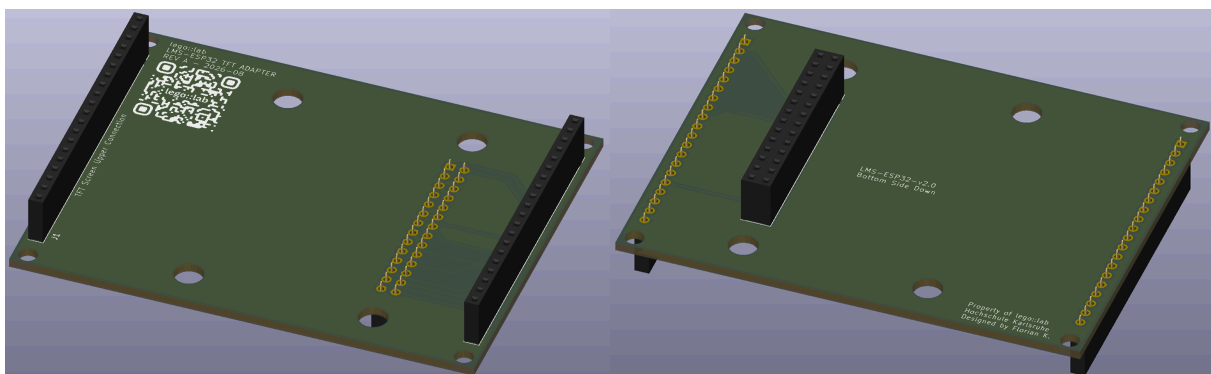


Abbildung 8: Vorderseite (rechts) und Rückseite (links) der Adapterplatine als 3D-Rendering

7.7 Diskussion der Ergebnisse

Die Messungen bestätigen die Annahme, auf der der gesamte Entwurf beruht, denn für die Dauer eines Aufrufs zählt die Anzahl der Übertragungen und nicht die übertragene Datenmenge. Jede eingesparte Übertragung spart 114 ms, unabhängig davon, wie viele Byte sie transportiert hätte. Der Wechsel von der früheren Rahmung und JSON auf den heutigen Stand halbiert somit die Dauer eines Aufrufs.

Erklärungsbedürftig sind die 114 ms selbst, denn bei 115200 Baud dauert die Übertragung eines Blocks von 16 Byte deutlich weniger als eine Millisekunde. Die Zeit entsteht folglich nicht auf der Leitung. Wo sie entsteht, lässt sich an den beteiligten Schichten weiter eingrenzen, auch ohne eine Messung innerhalb der Busanbindung.

Auf deranzeigeeinheit scheidet sie weitgehend aus. Die Transportbibliothek empfiehlt, ihre Verarbeitung wenigstens alle 20 ms aufzurufen [36], tatsächlich geschieht das etwa jede Millisekunde. Im laufenden Betrieb wartet die Bibliothek höchstens eine Millisekunde, nämlich beim zweiten Versuch, ein Byte zu lesen. Alle längeren Wartezeiten ihres Quelltextes fallen nur beim Verbindungsaufbau an, und die Überwachung der Verbindung greift erst nach einer Sekunde Untätigkeit.

Ein Moduswechsel scheidet ebenfalls aus. Ein Round Trip besteht aus einem Schreiben und einem Lesen desselben Modus, denn die Middleware registriert nach Abschnitt 6.3 genau ein Kommando, sodass der Bus den Modus zwischen zwei Aufrufen nicht wechseln muss.

Damit bleibt die Busanbindung auf der Steuereinheit, und dort liegt der größte Teil der Zeit. Die Bibliothek wartet nach dem Schreiben eine feste Zeit, bevor sie liest. Diese Wartezeit wurde während der Entwicklung von 0 auf 100 ms angehoben. Sie greift immer dann, wenn ein Kommando eine Nutzlast vom Hub zur Gegenseite führt. Da die Middleware nach Abschnitt 6.3 ein einziges Kommando mit Nutzlast in beide Richtungen registriert und keinen eigenen Wert übergibt, wird diese Wartezeit bei jedem Austausch ausgeführt, auch bei den Anfragen des Rückkanals.

Nachgewiesen wird das durch eine eigene Messreihe, in der die Wartezeit ausdrücklich übergeben und schrittweise verkleinert wird. Gemessen werden dieselben acht Textlängen wie zuvor, je Wartezeit 160 Aufrufe. Über die drei vollständigen Läufe ergibt eine Ausgleichsgerade durch alle 480 Aufrufe

$$t_{\text{RT}} = 14,39 \text{ ms} + 0,9993 \cdot t_{\text{wait}} \quad (8)$$

bei einer größten Abweichung vom Modell von 2,4 ms. Eine Steigung von eins bedeutet, dass jede Millisekunde Wartezeit unverändert in der Dauer eines Round Trips landet. Von den 114,2 ms sind damit 100 ms die angehobene Wartezeit, und die verbleibenden 14,4 ms entfallen auf das Schreiben, das Lesen und die Verarbeitung auf beiden Seiten. Dieser Rest lässt sich ohne eine Messung innerhalb der Firmware nicht weiter aufteilen.

Die Messreihe entstand auf einem späteren Stand, bei dem das Ergebnis nach Abschnitt 8.3.1 mit der Bestätigung zurückkommt. Für diese Aussage ist das ohne Belang, weil die Ausgleichsgerade die Dauer je Round Trip betrachtet und nicht deren Anzahl. Die Busanbindung selbst ist in beiden Ständen dieselbe.

Erklärt ist damit auch die geringe Streuung, denn ein Block ist unabhängig von der Nutzlast stets 16 Byte lang, und ebenso, warum das Abfrageintervall von 50 ms nie zum Tragen kommt, da der Server nach 100 ms Wartezeit längst fertig ist.

Ohne Wartezeit liest der Client, bevor die Gegenseite den Frame verarbeitet hat, und erhält die vorherige Antwort. Genau diesen Fall behandelt die Transportschicht nach Abschnitt 5.2.7 bereits, indem sie ihn erkennt und den Frame wiederholt. Die Wartezeit schützt also vor einem Fehler, den der Entwurf in Grenzen selbst abfängt, und kostet dafür 100 ms je Übertragung. Was eine kürzere Wartezeit tatsächlich einbringt und wo sie an ihre Grenze stößt, zeigt Abschnitt 8.3.2.

Für K4 ergibt sich ein zweigeteiltes Bild, denn über ein Binding liegt die Reaktion mit höchstens 59 ms sicher innerhalb der geforderten 100 ms, über den Client dagegen im gemessenen Stand rund fünfzehnmal so lange und nach der Rechnung in Abschnitt 8.3.2 auch mit kürzerer Wartezeit noch etwa dreieinhalbmal so lange. Die lokalen Bindings sind somit nicht nur eine Abkürzung, sondern unter den in dieser Arbeit umgesetzten Mechanismen der einzige Weg, eine Reaktion in der geforderten Zeit anzuzeigen. Dass ihr Satz an Aktionen klein ist, begrenzt zugleich, welche Reaktionen in dieser Zeit überhaupt möglich sind.

Die Einleitung motiviert die Arbeit mit der Vermittlung von Technikenkenntnissen an Kinder und Jugendliche. Ob die entstandene Schnittstelle dafür handhabbar ist, wurde nicht untersucht, dazu wäre eine Erprobung mit Lernenden nötig. Am Beispielprogramm in Listing 9 lässt sich immerhin ablesen, was sie voraussetzt. Das Erzeugen und Verändern von Elementen sieht aus wie gewöhnliche Python-Programmierung, und nichts davon verlangt Kenntnis des Protokolls. Drei Stellen treten dennoch nach außen. Erstens blockiert jeder Aufruf, sodass eine Schleife mit Wartezeit nötig ist und die Anzeige nicht nebenbei bedient werden kann. Zweitens müssen Schaltflächen abgefragt werden, was eine Schleife erzwingt, statt auf ein Ereignis zu warten. Drittens werden Stellvertreter nach `clear()` ungültig, was zu einer Ausnahme führt, deren Ursache zeitlich weit von ihrem Auftreten entfernt liegen kann. Die Bindings mildern den ersten und zweiten Punkt, da die häufigste Reaktion, das Umschalten eines Bildschirms, einmal erklärt wird und danach ohne Zutun der Anwendung abläuft. Die in Abschnitt 8.3.3 beschriebenen Rückrufe zielen auf denselben Punkt.

Die ursprüngliche Fassung erfüllte K3 nicht, weil sie nur das jüngste Ereignis je Objekt lieferte. Der Abstand zwischen zwei Abfragen ist durch die Dauer eines Aufrufs nach unten begrenzt, und in dieser Zeit endete eine kurze Berührung, ohne dass die Anwendung davon erfuhr. Mit dem geleerten Puffer und dem gemeldeten Überlauf ist die Anforderung erfüllt. Offen bleibt, dass jeder Button einzeln abgefragt wird, sodass die Anzahl der Aufrufe mit der Anzahl der Bedienelemente wächst. Abschnitt 8.3.3 greift das auf.

Die Aussagekraft der Messung ist in drei Punkten begrenzt. Erstens wurde der Übertragungsaufwand anhand eines einzigen Aufrufs, nämlich `set_text`, auf einer festen Hardwarekombination gemessen, während Aufrufe mit größeren Ergebnissen die Round Trips anders auf Hin- und Rückweg verteilen. Zweitens stammen die Zeiten auf dem Server und auf dem Client aus getrennten Uhren und wurden rechnerisch zusammengesetzt, eine durchgehende Messung von der Berührung bis zum Bild liegt somit nicht vor. Drittens wurden die Berührungen von Hand ausgelöst, sodass die Erkennungszeit zusätzlich davon abhängt, wie der Finger aufsetzt.

8 Fazit und Ausblick

Dieses Kapitel ordnet die Ergebnisse ein, indem zunächst die Grenzen des Entwurfs benannt werden, die sich aus den Messungen in Kapitel 7 ergeben. Anschließend wird geprüft, wie weit sich die Lösung von der betrachteten Plattform gelöst hat, woran sich die möglichen Erweiterungen anschließen. Den Abschluss bildet die Zusammenfassung der Ergebnisse.

8.1 Limitierungen

Die folgenden drei Abschnitte behandeln die Grenzen des Kommunikationsmodells, die erreichbare Latenz und die Stellen, an denen die Abstraktion nicht vollständig trägt.

8.1.1 Kommunikationsmodell

Die Anwendung auf dem Client blockiert, solange ein Aufruf läuft, was unmittelbar aus der fehlenden Nebenläufigkeit in Abschnitt 3.4 folgt. Nach Abschnitt 7.3 dauert ein kurzer Aufruf wie `set_text` mit 21 Byte 457 ms, die Abfrage eines Buttons nach Abschnitt 7.4 572 ms, im ungünstigsten Fall jedoch ein Vielfaches davon. Die Obergrenzen aus Tabelle VII begrenzen nämlich jede Phase für sich und nicht den Aufruf als Ganzes. Je Frame sind acht Versuche zulässig, und die fünf Sekunden der Wartephase zählen allein die Pausen zwischen zwei Abfragen, nicht die Abfragen selbst. Für einen Aufruf aus N gesendeten und M abzuholenden Frames ergibt sich daraus mit der Anzahl der Versuche A , der Dauer eines Round Trips t , dem Abfrageintervall t_p und der Wartegrenze T_w

$$T_{\max} = (N + M) \cdot A \cdot t + \left(\frac{T_w}{t_p} + 1 \right) \cdot t + T_w \quad (9)$$

als Schranke. Mit den Werten der Referenzimplementierung sind das für einen kurzen Aufruf mit zwei gesendeten und einem abzuholenden Frame rund 19 Sekunden und für einen Aufruf über elf Frames rund 27 Sekunden. Der Wert ist eine Schranke und kein erwarteter Fall, denn in den Messungen mit 100 ms Wartezeit trat eine einzige Wiederholung auf. Für ein Programm, das nebenher einen Roboter steuert, ist allerdings schon die Schranke entscheidend, weil sie angibt, wie lange die Steuerung im Fehlerfall stillsteht. In dieser Zeit steht alles still, was sonst im selben Programm läuft, insbesondere die Regelung von Motoren und das Auslesen von Sensoren. Für ein Programm, das eine Anzeige bedient und gleichzeitig einen Roboter steuert, stellt dies die größte Einschränkung des Entwurfs dar.

Pybricks bietet inzwischen kooperatives Multitasking über `async` und `await` [58], womit sich das Warten auf ein Ergebnis unterbrechen ließe, ohne die übrige Anwendung anzuhalten. Der anfragegetriebene Rückkanal bliebe dabei unverändert, da weiterhin der Client jeden Austausch beginnt. Betroffen wären lediglich die Schnittstelle der Stellvertreter, die dann Koroutinen anbieten müssten, sowie die Schleife, die auf das Ergebnis wartet. Ob sich dieser Aufwand lohnt, hängt jedoch davon ab, wie viel die Anwendung neben der Anzeige zu tun hat.

Eine zweite Einschränkung betrifft den Rückfluss von Ereignissen, dessen Abstand zwischen zwei Abfragen durch die Dauer eines Aufrufs nach unten begrenzt ist und somit im Bereich einer halben Sekunde liegt. Seit der in Abschnitt 6.7 beschriebenen Änderung geht dabei zwar kein Ereignis mehr unbemerkt verloren, die Anwendung erfährt jedoch erst mit dieser Verzögerung davon. Hinzu kommt, dass jeder Button einzeln abgefragt wird, sodass die Anzahl der Aufrufe mit der Anzahl der Bedienelemente wächst.

8.1.2 Latenz und Durchsatz

Die Dauer eines Aufrufs ergibt sich aus der Anzahl der Round Trips multipliziert mit 114 ms, wovon zwei unabhängig von der Nachrichtenlänge auf das Abholen des Ergebnisses entfallen. Bei kurzen Aufrufen macht dieser feste Anteil bereits die Hälfte der Gesamtzeit aus. Die 114 ms sind dabei keine Eigenschaft des Busses, sondern bestehen nach Abschnitt 7.7 zu 100 ms

aus einer Wartezeit, die während der Entwicklung in der Busanbindung festgelegt wurde. Die folgende Einschränkung gilt also für den gemessenen Stand und nicht für die Verbindungsklasse.

Eine Reaktion, die über den Client läuft, benötigt zwei Aufrufe und liegt damit im Mittel bei rund 1490 ms. Die Reaktion dauert damit im gemessenen Stand rund fünfzehnmal so lange, wie K4 zulässt, mit den Erweiterungen aus Abschnitt 8.3.2 gerechnet noch etwa dreieinhalbmal so lange. Erreichbar ist sie nur über ein Binding, das der Server selbst ausführt und das nach Tabelle XII höchstens 59 ms benötigt. Damit ist die Zahl der Reaktionen, die in der geforderten Zeit möglich sind, durch den kleinen Satz an Aktionen aus Tabelle VIII begrenzt.

Der Durchsatz war kein Entwurfsziel und fällt entsprechend gering aus, da eine Nachricht von 142 Byte den Bus im gemessenen Stand bereits für 1,5 Sekunden belegt. Für Text und einzelne Bedienelemente reicht das aus, für Bilddaten oder fortlaufende Messwerte jedoch nicht.

8.1.3 Abstraktionsgrenzen

Die Aufrufstransparenz aus Kapitel 4 trägt syntaktisch, nicht zeitlich, denn ein Aufruf sieht zwar wie ein lokaler Aufruf aus, dauert jedoch um Größenordnungen länger. Das entspricht der Einordnung aus Abschnitt 2.1 und ist eine grundsätzliche Eigenschaft entfernter Aufrufe, keine Schwäche der Umsetzung.

Beim Erweitern des Objektmodells bleiben Transport- und Kommunikationsschicht unberührt, das Objektmodell selbst aber nicht. Wie Abschnitt 6.6.4 an einem Fortschrittsbalken zeigt, braucht ein neuer Objekttyp eine Basisklasse, einen Stellvertreter, einen Adapter und zusätzlich eine Fabrikmethode in dem Objekt, das ihn erzeugt. Auch das Ereignismodell ist bisher auf Buttons zugeschnitten, da nur dort Rückrufe bei der Anzeigebibliothek angemeldet werden. Erweiterbar ist das Modell also, aber nicht ohne Eingriff an mehreren Stellen.

Die Prüfung von Referenzen stößt an eine Grenze, die sich aus ihrer Breite ergibt, da sich die Generation eines Slots nach 64 Freigaben wiederholt. Ein Stellvertreter, der die ganze Zeit besteht, passt danach wieder auf ein fremdes Objekt. Hinzu kommt, dass der Client lediglich die Stellvertreter kennt, die er selbst hält. Löscht er einen Screen, bleiben die Stellvertreter der Elemente darauf lokal gültig und werden erst vom Server abgewiesen.

Schließlich führt das schemafreie Format keine Typinformationen mit, sodass sich erst zur Laufzeit herausstellt, ob eine Methode existiert und ob die Argumente passen. Verwenden beide Seiten unterschiedliche Codecs, lässt sich die Nutzlast zwar dekodieren, ergibt aber keinen gültigen Aufruf mehr. Beide Seiten weisen diesen Fall inzwischen mit einer eindeutigen Meldung ab, verhindern lässt er sich im Entwurf jedoch nicht.

Für die Anwendung sichtbar ist auch der Wertebereich der Ganzzahlen. Die Kodierung trägt nach Abschnitt 6.4.3 nur Werte von -32768 bis 65535 , sodass etwa ein Zeitstempel aus `ticks_ms` oder ein Zähler oberhalb von 65535 nicht übertragen werden kann. Als Argument scheitert ein solcher Wert noch auf dem Client mit einem `OverflowError`. Diese Ausnahme gehört nicht zu den Fehlerklassen der Middleware und wird von einer Behandlung für `RemoteError` deshalb nicht erfasst. Steckt der Wert im Ergebnis, fällt er erst auf, wenn die Methode bereits ausgeführt ist, und der Client erhält eine Fehlermeldung für einen Aufruf, der stattgefunden hat.

8.2 Übertragbarkeit auf andere Verbindungsklassen

Der Entwurf nimmt in Kapitel 4 und Kapitel 5 durchgehend auf die Merkmale der Verbindungsklasse aus Abschnitt 3.6 Bezug und nicht auf den LEGO-Bus. Alles Wegabhängige liegt unterhalb der Interceptor-Schnittstelle, die lediglich aus einer einzigen Operation besteht, sodass eine andere Verbindung angebunden wird, indem genau diese Operation neu umgesetzt wird. Im Stand dieser Arbeit gilt das allerdings nur für den Entwurf und nicht für die Ablage des Codes, wie Tabelle III zeigt. Abschnitt 8.3.4 beschreibt den Umbau, der das nachholt.

Prüfen lässt sich das an den Größen, die von der Verbindung abhängen, also an der Blockgröße und den daraus abgeleiteten Werten für die Nutzlast je Frame, die Breite des Längenfilds und die maximale Nachrichtenlänge. Alle vier stehen als Konstanten an einer Stelle,

und Kapitel 7 zeigt mit dem Wechsel von 13 auf 7 Byte, dass eine Änderung tatsächlich nur dort stattfindet. Für I2C oder Modbus im RTU-Modus wäre derselbe Weg gangbar, sofern die Gegenseite Blöcke fester Größe auf Anfrage austauschen kann. Für BLE, das Abschnitt 3.5 als Verbindung zwischen beiden Einheiten verwirft, gilt das für Zerlegung und Rahmung ebenfalls. Den Rückkanal könnte BLE über Benachrichtigungen kürzer gestalten, als der Entwurf ihn vorsieht. Nutzen ließe sich das nur mit einer Erweiterung der Interceptor-Schnittstelle, da sie ausschließlich den Austausch kennt, den der Client beginnt.

Wie weit die Übertragbarkeit belegt ist, unterscheidet sich dabei je Stufe. Architektonisch ist sie vorgesehen, da alles Wegabhängige unterhalb einer einzigen Operation liegt. Nachgewiesen ist sie für eine zweite Umsetzung dieser Operation in Software, wie Abschnitt 7.2 zeigt, und nach dem Umbau aus Abschnitt 8.3.4 für eine zweite Busanbindung, über die alle Testläufe bestehen, ohne dass PUPRemote geladen wird. Auf einer zweiten realen Verbindung ist sie nicht erprobt, und erst dort träten die Eigenschaften zutage, die die Testverbindung nicht hat, nämlich Laufzeit, Verlust und eine eigene Fehlersemantik. Die Aussagen dieses Abschnitts sind deshalb als begründete Erwartung zu lesen und nicht als Messergebnis.

Nicht übertragbar ist hingegen die Annahme, dass immer nur eine Nachricht unterwegs ist, da sie aus dem anfragegetriebenen Kanal und der fehlenden Nebenläufigkeit folgt. Auf einer Verbindung, die mehrere gleichzeitige Übertragungen erlaubt, müssten die Rekonstruktion und die Verwaltung der Ergebnisse je Kennung getrennt werden, wie Abschnitt 5.2.5 beschreibt.

8.3 Mögliche Erweiterungen

Die folgenden Erweiterungen setzen an den Grenzen aus Kapitel 7 an, wobei die ersten drei den Austausch über den Bus betreffen, also Anzahl und Dauer der Übertragungen, die vierte die Trennung der Busanbindung und die übrigen die Serialisierung, den Hardwareaufbau sowie den Umfang des Objektmodells.

8.3.1 Ergebnis mit der Bestätigung

Nach Kapitel 7 kostet das Abholen des Ergebnisses unabhängig von der Nachrichtenlänge zwei Round Trips und somit 228 ms je Aufruf. Die Ausführungszeit auf dem Server wurde nicht gemessen, sie liegt nach Abschnitt 7.3 aber unter der Dauer eines Round Trips, denn das Ergebnis liegt bereits bei der ersten Anfrage des Clients vor. Das Ergebnis selbst ist mit vier Byte kürzer als ein einzelner Frame. Statt die Ausführung aufzuschieben und das Ergebnis abfragen zu lassen, führt der Server den Aufruf deshalb besser beim letzten Datenframe direkt aus und gibt das Ergebnis zusammen mit der Bestätigung zurück.

Der Kanal bleibt dabei anfragegetrieben, da es sich weiterhin um eine Antwort handelt. Passt das Ergebnis nicht in einen Frame, meldet der Server lediglich, dass es bereitliegt, und das Abholen läuft wie bisher ab.

Diese Erweiterung ist nach dem Stand `stand-projektarbeit` umgesetzt und gemessen worden. Der Messlauf verwendet dieselbe Konfiguration wie der ursprüngliche, also MessagePack, 13 Byte Nutzlast je Frame und dieselbe Wartezeit von 100 ms, und unterscheidet sich allein im Rückweg des Ergebnisses. Tabelle XIV stellt beide gegenüber.

Tabelle XIV: Aufrufe mit abgeholtem und mit mitgeliefertem Ergebnis, Mediane aus je 20 Aufrufen

Nutzlast	abholen		mitgeliefert	
	Round Trips	Dauer	Round Trips	Dauer
21 Byte	4	457 ms	2	229 ms
41 Byte	6	685 ms	4	457 ms
82 Byte	9	1027 ms	7	799 ms
142 Byte	13	1483 ms	11	1255 ms

Über alle acht gemessenen Textlängen hinweg entfallen genau zwei Round Trips, und die Ersparnis beträgt jeweils 228 ms. Das entspricht der Erwartung, denn beide eingesparten Übertragungen sind unabhängig von der Nachrichtenlänge.

Zwei Punkte waren dabei zu beachten, und beide sind in der Umsetzung berücksichtigt. Zum einen ist der Bus während der Ausführung belegt, was für Aufrufe an die Anzeigebibliothek unkritisch ist, für längere Operationen jedoch ein zusätzliches Kennzeichen erfordern würde. Zum anderen prüfte die Sendeschleife des Clients im gemessenen Stand nur Position und Art der Antwort, wie Abschnitt 6.9 beschreibt. Trägt die Bestätigung das Ergebnis, ist eine veraltete Antwort kein verworfener Frame mehr, sondern ein falscher Rückgabewert, weshalb die Sendeschleife nun zusätzlich die Kennung der Nachricht auswertet. Dass diese Prüfung greift, zeigt Abschnitt 8.3.2.

8.3.2 Kürzere Wartezeit der Busanbindung

Nach Abschnitt 7.7 entfallen 100 der 114 ms je Round Trip auf eine feste Wartezeit, mit der die Busanbindung zwischen Schreiben und Lesen pausiert. Der Wert lässt sich je Aufruf übergeben, ohne Angabe verwendete die eingesetzte Kopie der Bibliothek 100 ms. Der Entwurf ist auf die Folge einer zu kurzen Wartezeit in Grenzen vorbereitet, denn eine zu früh gelesene Antwort ist eine veraltete Antwort, und die erkennt die Transportschicht nach Abschnitt 5.2.7 und wiederholt den Frame.

Damit wird die Wartezeit zu einer Stellgröße. Jede eingesparte Millisekunde wirkt auf jeden Round Trip und damit auf jeden Aufruf, während jede zusätzliche Wiederholung einen vollen Round Trip kostet. Tabelle XV stellt beides gegenüber.

Tabelle XV: Wirkung der Wartezeit, je Einstellung 160 Aufrufe über acht Textlängen, Dauer je Round Trip als Mittelwert

Wartezeit	je Round Trip	Wiederholungen	Lauf
100 ms	114,3 ms	0 von 1000	vollständig
50 ms	64,3 ms	1 von 1001	vollständig
20 ms	34,4 ms	54 von 1054	vollständig
5 ms	20,4 ms	195 von 327	nach 48 Aufrufen abgebrochen
0 ms	entfällt	entfällt	bricht beim ersten Aufruf ab

Die Zeit folgt der Einstellung genau, wie Formel 8 zeigt. Der Preis steigt dagegen nicht gleichmäßig, sondern springt. Bei 50 ms tritt in tausend Übertragungen eine einzige Wiederholung auf, bei 20 ms sind es fünf Prozent, und bei 5 ms in beiden Läufen zusammen zwischen 42 und 78 Prozent, je nach Textlänge. Der Lauf mit 5 ms bricht ab, auch dann noch, wenn die Obergrenze der Versuche von acht auf zwanzig angehoben wird.

Brauchbar ist damit eine Wartezeit von 20 ms. Ein Round Trip kostet dort 34 statt 114 ms, ein Aufruf über zwei Round Trips 69 statt 229 ms. Bei so kurzen Aufrufen trat keine Wiederholung auf, über alle Textlängen kommen fünf Prozent Übertragungen hinzu.

Aufschlussreich ist, woran der Lauf mit 5 ms scheitert. Im Lauf mit zwanzig Versuchen nennt die Fehlermeldung als letzte Antwort einen Frame der Art `DATA_LAST` mit dem Ergebnis des vorangegangenen Aufrufs. Trägt die Bestätigung das Ergebnis, ist eine veraltete Antwort eben kein verworfener Frame mehr, sondern sieht wie ein gültiges Ergebnis aus. Die Prüfung der Nachrichtenkennung, die Abschnitt 8.3.1 als Voraussetzung nennt, greift und weist sie ab. Der Client kommt dann allerdings nicht mehr voran und gibt nach der Obergrenze auf. Die Zusicherung bleibt gewahrt, der Aufruf scheitert mit einer Ausnahme statt ein falsches Ergebnis zu liefern.

Für K4 ändert das nichts am Ergebnis, wohl aber am Abstand. Mit dem Ergebnis in der Bestätigung nach Abschnitt 8.3.1 kostet das Ändern des Textes aus Tabelle XII drei statt fünf Round Trips, die Abfrage dagegen vier, da ihr Ergebnis zwei Frames belegt und weiterhin abgeholt wird. Bei 20 ms Wartezeit und fünf Prozent Wiederholungen sind das zusammen etwa 253 ms. Das Warten auf die nächste Abfrage sinkt mit einer Abfrage von etwa 138 ms und der Pause von 20 ms im Mittel auf etwa 79 ms. Mit Erkennung, Verarbeitung und Neuzeichnen ergibt das im Mittel rund 360 ms statt rund 1490 ms und damit immer noch etwa das Dreieinhalbfache der geforderten 100 ms. Die Zahl ist berechnet und nicht gemessen, da die Messreihe nur `set_text` betrachtet. Die Folgerung aus Abschnitt 7.7, dass K4 nur über Bindings erreichbar ist, bleibt damit bestehen.

Statt die Wartezeit fest einzustellen, ließe sie sich auch ganz vermeiden. Der Client könnte nach dem Schreiben wiederholt lesen, bis die passende Antwort vorliegt, und würde damit je Frame genau so lange warten, wie die Gegenseite tatsächlich braucht. Dafür spricht, dass der Anteil der Wiederholungen bei 20 ms tendenziell mit der Länge der Nachricht steigt. Das deutet auf einen langsameren letzten Frame hin, bei dem der Server den Aufruf ausführt, während er die übrigen Frames nur puffert. Voraussetzung ist, dass ein Lesevorgang auf dem Hub ohne eigene Übertragung den zuletzt empfangenen Wert liefert, was noch zu prüfen ist.

Ähnlich ließe sich die Blockgröße angehen. Oberhalb von 16 Byte treten nach Abschnitt 3.4 Prüfsummenfehler auf, deren Ursache in dieser Arbeit nicht geklärt ist. Ließen sie sich beheben, stiege die Nutzlast je Frame von 13 auf 29 Byte, und eine Nachricht bräuhete etwa halb so viele Frames, während ein Round Trip nur um die Übertragungszeit von 16 Byte länger würde. Das Längenfeld des Headers müsste dafür um ein Bit wachsen, was ohne größeren Header möglich ist, da für die acht Opcodes drei Bit genügen. Beide Änderungen betreffen allein die Busanbindung und die Transportschicht.

8.3.3 Ereignisgetriebener Rückkanal

Eine weitere Erweiterung betrifft die Anzahl der Abfragen. Heute wird jeder Button einzeln abgefragt, und jede Abfrage ist ein vollständiger Aufruf. Bei fünf Buttons sind das fünf Aufrufe je Durchlauf, also nach Kapitel 7 etwa drei Sekunden. Stattdessen könnte das Display eine Methode anbieten, die alle seit der letzten Abfrage aufgelaufenen Ereignisse aller Objekte auf einmal liefert. Der Puffer läge dann nicht je Objekt, sondern gemeinsam vor, wodurch die Anzahl der Abfragen unabhängig von der Anzahl der Bedienelemente wäre. Zugleich fiel der zusätzliche Round Trip wieder weg, den die Antwort je Button nach Abschnitt 7.4 heute kostet, da eine gemeinsame Antwort nur einmal übertragen werden muss.

Darauf aufbauend könnte die Anwendung Rückrufe je Button registrieren, statt die Zustände selbst abzufragen, sodass die Schleife nur noch aus einem einzigen Aufruf bestünde, der die Ereignisse verteilt. Beides ist nach dem Stand `stand-projektarbeit` umgesetzt und in der Testumgebung geprüft, auf der Hardware aber nicht gemessen. Eine weitere Verbesserung wäre ein Hinweis in jeder Bestätigung, ob überhaupt Ereignisse anliegen. Der Client müsste dann lediglich nachfragen, wenn es etwas abzuholen gibt, wodurch die Abfrage im Leerlauf vollständig entfiel.

8.3.4 Trennung der Busanbindung

Nach Tabelle III liegt der Aufrufteil der Kommunikationsschicht im Stand dieser Arbeit in denselben beiden Klassen wie die Busanbindung, weshalb Tabelle X K6 nur als teilweise erfüllt führt. Nach dem Stand `stand-projektarbeit` ist diese Trennung nachgeholt. Rufende und ausführende Seite liegen nun je in einer Klasse ohne Bezug zum Bus, und die Busanbindung an PUPRemote beschränkt sich auf das, was nur auf diesem Bus existiert. Auf der rufenden Seite sind das die eine Operation der Interceptor-Schnittstelle, also ein Austausch über `remote.call`, und die Registrierung des Kommandos. Auf der ausführenden Seite sind es der Rückruf, den PUPRemote für jeden Frame aufruft, die Umwandlung seiner Nutzlast in Bytes und ebenfalls die Registrierung. Das Objektmodell kennt nur noch die Klasse ohne Bezug zum Bus.

Eine weitere Busanbindung kostet damit auf der rufenden Seite eine Klasse, die genau diese eine Operation umsetzt, und auf der ausführenden Seite die Weitergabe jedes empfangenen Blocks. Die Testumgebung enthält eine solche Busanbindung von rund zehn Zeilen, die beide Seiten im selben Prozess verbindet. Alle Testläufe bestehen auch über sie, und PUPRemote wird dabei nicht geladen. Für den Code ist K6 damit erfüllt. Offen bleibt wie in Abschnitt 8.2 die Erprobung auf einer zweiten realen Verbindung.

8.3.5 Alternative Serialisierung

JSON benötigt nach Kapitel 7 das 1,21-fache an Round Trips gegenüber MessagePack. Der Wechsel auf das binäre Format spart also etwa 17 % der Übertragungen. Ein schemagebundenes Format wie Protocol Buffers könnte die Nutzlast weiter verkürzen, da Schlüssel dann gar nicht mehr übertragen werden. Der Preis dafür wären ein Übersetzungsschritt und eine Beschreibung der Schnittstelle, die beide Seiten teilen, was nach Abschnitt 5.2.6 gerade der Grund für ein schemafreies Format war. Ob sich der Aufwand lohnt, ließe sich mit dem vorhandenen Messaufbau beantworten, da der Codec an einer einzigen Stelle austauschbar ist.

8.3.6 Integrierter Hardwareaufbau

Der Aufbau aus Abschnitt 6.2 besteht aus drei Baugruppen und zwei Steckverbindungen. Eine eigenständige Platine, die den Mikrocontroller, das Display und die Versorgung vereint, war als zweite Ausbaustufe geplant, wurde aus Zeitgründen jedoch nicht gebaut. Sie würde den Aufbau mechanisch robuster machen und die Spannungsversorgung sauber lösen, während sich am Protokoll, an der Latenz und an der Software nichts ändern würde. Der folgende Abschnitt umreißt, welche Funktionsgruppen eine solche Platine enthalten müsste, da die bisher verwendeten Baugruppen diese Aufgaben mitbringen und sie bei deren Wegfall neu zu lösen sind.

Ausgangspunkt ist der Sensorport des Hubs. Er führt neben Masse und den beiden Datenleitungen eine Logikversorgung von 3,3 V sowie die höhere Versorgung auf der Leitung M+, die das Gerät nach Abschnitt 6.3 ausdrücklich anfordert. Die Logikversorgung des Ports ist für die Elektronik eines Sensors ausgelegt und reicht für ein Display mit Hintergrundbeleuchtung nicht aus, weshalb die Platine ihre eigene Spannung aus M+ erzeugen muss.

Spannungsversorgung. Ein Abwärtswandler erzeugt aus den etwa 8 V der Leitung M+ die 3,3 V für Mikrocontroller, Display und Touchcontroller. Ein linearer Regler ist dafür ungeeignet, da er die Differenz in Wärme umsetzt und bei einem Strom von einigen hundert Milliampere über ein Watt abführen müsste. Der Wandler braucht außerdem einen Pufferkondensator, der den Einschaltstrom der Hintergrundbeleuchtung abfängt.

Versorgung über USB. Für die Entwicklung wird die Platine über USB versorgt, im Betrieb am Hub dagegen über M+. Beide Quellen dürfen sich nicht gegenseitig speisen, weshalb eine Umschaltung über zwei Dioden oder über einen Lastschalter nötig ist. Diese Schaltung entscheidet zugleich, welche Quelle Vorrang hat, wenn beide anliegen.

Schutz der Eingänge. Der Port ist steckbar und damit den üblichen Gefahren ausgesetzt. Vorgesehen werden sollten ein Verpolschutz auf M+, eine rückstellende Sicherung sowie Schutzdioden gegen elektrostatische Entladung auf allen Leitungen, die nach außen führen.

Anbindung an den Bus. Die beiden Datenleitungen des Ports arbeiten mit 3,3 V und können direkt an den Mikrocontroller geführt werden. Sinnvoll sind Serienwiderstände und ein definierter Ruhepegel auf der Sendeleitung, da die Gegenseite beim Verbindungsaufbau nach Abschnitt 6.3 einen festen Zustand erwartet.

Mikrocontroller mit seiner Grundbeschaltung. Dazu gehören die Abblockkondensatoren, die Beschaltung des Reset-Eingangs und die Taster für Reset und Startmodus. Besonderes Augenmerk verdienen die Pins, die beim Start abgefragt werden, da sie den Startmodus und die Flash-Spannung festlegen. Abschnitt 6.2 beschreibt den Fall, in dem die SD-Karte eine solche Leitung beim Start auf einen ungünstigen Pegel zieht.

Programmierschnittstelle. Ohne Steckplatine fehlt der Weg, die Firmware aufzuspielen. Nötig ist entweder ein Brückenbaustein von USB auf eine serielle Schnittstelle samt der üblichen Schaltung, die Reset und Startmodus automatisch setzt, oder ein Mikrocontroller mit eigener USB-Schnittstelle.

Anschluss des Displays. Das Display wird über die vorhandenen Leitungen für SPI angebunden, der Touchcontroller über I2C mit den beiden Pull-up-Widerständen, die der Bus verlangt. Die Hintergrundbeleuchtung sollte über einen Transistor geschaltet werden, damit sie sich dimmen und im Leerlauf abschalten lässt. Im heutigen Aufbau leuchtet sie dauerhaft.

Zwei Punkte gehen über die reine Schaltung hinaus. Die Platine muss sich mechanisch in den Aufbau einfügen, also zu den Maßen des Displays und zu den Befestigungspunkten des Hubs passen. Zudem sollte sie Messpunkte für die Versorgungsspannungen und die beiden Datenleitungen vorsehen, da eine Messung an einem gesteckten Aufbau sonst kaum möglich ist. Der Nutzen bliebe dabei auf Mechanik, Versorgung und Handhabung beschränkt. An der Anzahl der Übertragungen, der Latenz aus Kapitel 7 und der Software würde sich nichts ändern.

8.3.7 Weitere Peripherieklassen

Das Objektmodell bildet bisher Screens, Labels und Buttons ab, wobei sich weitere Elemente wie Eingabefelder oder Listen nach demselben Muster ergänzen ließen. Interessanter ist jedoch der Schritt darüber hinaus, denn die Schichten unterhalb des Objektmodells kennen keine Anzeige, sondern lediglich Aufrufe an benannte Empfänger. Eine andere Geräteklasse, beispielsweise ein Sensor mit eigener Auswertung, ließe sich somit über dieselbe Middleware anbinden, indem ein weiterer Dispatcher registriert wird.

8.4 Zusammenfassung

Die vorliegende Arbeit entwirft eine Middleware, die entfernte Objekte über eine anfragegetriebene Verbindung mit sehr kleiner Paketgröße nutzbar macht, und setzt diese für die Anbindung eines Touchscreen-Displays an einen LEGO-Hub um. Der Entwurf gliedert sich in fünf Schichten von der Busanbindung bis zur Anwendung und bündelt im Entwurf alles Wegabhängige in der untersten Schicht.

Die Messungen zeigen, dass die Anzahl der Übertragungen die Dauer eines Aufrufs bestimmt. Ein Round Trip kostet 114 ms, unabhängig von Format und Nachrichtenlänge. Der schlankere Header und das binäre Format zusammen halbieren die Anzahl der nötigen Übertragungen gegenüber der ersten Fassung. Drei der sechs Kommunikationsanforderungen sind vollständig erfüllt, drei nur teilweise. Teilweise erfüllt sind die Antwortzeit, die nur über lokal ausgeführte Bindings eingehalten wird, der Speicherbedarf, der nur auf der Steuereinheit gemessen ist, und die Unabhängigkeit vom Übertragungsweg.

Damit steht ein System zur Verfügung, das eine interaktive Oberfläche über eine Verbindung bedient, die dafür ursprünglich nicht vorgesehen war. Die verbleibenden Schwächen sind benannt und, soweit der Messaufbau es zulässt, beziffert. Für die wichtigsten liegt ein Weg vor, der ohne Änderung des Kommunikationsmodells auskommt. Das gilt auch für die bestimmende Größe des Systems, denn 100 der 114 ms je Round Trip sind nach Abschnitt 7.7 eine Wartezeit der Busanbindung, die sich verkürzen lässt.

Bibliografie

- [1] A. D. Birrell und B. J. Nelson, „Implementing Remote Procedure Calls“, *ACM Transactions on Computer Systems*, Bd. 2, Nr. 1, S. 39–59, Feb. 1984, doi: [10.1145/2080.357392](https://doi.org/10.1145/2080.357392).
- [2] J. Waldo, G. Wyant, A. Wollrath, und S. Kendall, „A Note on Distributed Computing“, Mountain View, CA, Technical Report SMLI TR-94-29, Nov. 1994. Zugegriffen: 28. September 2026. [Online]. Verfügbar unter: <https://waldo.scholars.harvard.edu/publications/note-distributed-computing>
- [3] M. van Steen und A. S. Tanenbaum, *Distributed Systems*, 3. Aufl. distributed-systems.net, 2017. Zugegriffen: 28. September 2026. [Online]. Verfügbar unter: <https://www.distributed-systems.net/index.php/books/ds3/>
- [4] F. Buschmann, R. Meunier, H. Rohnert, P. Sommerlad, und M. Stal, *Pattern-Oriented Software Architecture: A System of Patterns*. Chichester: Wiley, 1996.
- [5] E. Gamma, R. Helm, R. Johnson, und J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Pearson Education, 1995.
- [6] Object Management Group, „Common Object Request Broker Architecture (CORBA), Version 3.4“. Zugegriffen: 28. September 2026. [Online]. Verfügbar unter: <https://www.omg.org/spec/CORBA/3.4/About-CORBA>
- [7] Oracle, „Java Remote Method Invocation Specification“. Zugegriffen: 28. September 2026. [Online]. Verfügbar unter: <https://docs.oracle.com/en/java/javase/17/docs/specs/rmi/index.html>
- [8] C. Bormann, M. Ersue, und A. Keranen, „Terminology for Constrained-Node Networks“, Request for Comments RFC7228, Mai 2014. doi: [10.17487/RFC7228](https://doi.org/10.17487/RFC7228).
- [9] „MicroPython - Python for Microcontrollers“. Zugegriffen: 28. September 2026. [Online]. Verfügbar unter: <https://micropython.org/>
- [10] J. Hill, R. Szewczyk, A. Woo, S. Hollar, D. Culler, und K. Pister, „System Architecture Directions for Networked Sensors“, in *Proceedings of the Ninth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS-IX)*, Cambridge, MA, USA: ACM, Nov. 2000, S. 93–104. doi: [10.1145/378993.379006](https://doi.org/10.1145/378993.379006).
- [11] A. Dunkels, B. Gronvall, und T. Voigt, „Contiki - a Lightweight and Flexible Operating System for Tiny Networked Sensors“, in *29th Annual IEEE International Conference on Local Computer Networks*, Nov. 2004, S. 455–462. doi: [10.1109/LCN.2004.38](https://doi.org/10.1109/LCN.2004.38).
- [12] NXP Semiconductors, „I2C-Bus Specification and User Manual“, UM10204 Rev. 7.0, Okt. 2021. [Online]. Verfügbar unter: <https://www.nxp.com/docs/en/user-guide/UM10204.pdf>
- [13] P. Dhaker, „Introduction to SPI Interface“, *Analog Dialogue*, Bd. 52, Nr. 3, Sep. 2018, Zugegriffen: 2. Oktober 2026. [Online]. Verfügbar unter: <https://www.analog.com/en/resources/analog-dialogue/articles/introduction-to-spi-interface.html>
- [14] Modbus Organization, „MODBUS Application Protocol Specification V1.1b3“, Apr. 2012. Zugegriffen: 2. Oktober 2026. [Online]. Verfügbar unter: <https://www.modbus.org/file/secure/modbusprotocolspecification.pdf>
- [15] Sun Microsystems, „RPC: Remote Procedure Call Protocol Specification“, Request for Comments RFC1050, Apr. 1988. doi: [10.17487/RFC1050](https://doi.org/10.17487/RFC1050).
- [16] R. Thurlow, „RPC: Remote Procedure Call Protocol Specification Version 2“, Request for Comments RFC5531, Mai 2009. doi: [10.17487/RFC5531](https://doi.org/10.17487/RFC5531).
- [17] „JSON-RPC 2.0 Specification“. Zugegriffen: 4. August 2026. [Online]. Verfügbar unter: <https://www.jsonrpc.org/specification>
- [18] „gRPC“. Zugegriffen: 4. August 2026. [Online]. Verfügbar unter: <https://grpc.io/>
- [19] „Introducing eRPC“. Zugegriffen: 30. September 2026. [Online]. Verfügbar unter: <https://community.nxp.com/t5/MCUXpresso-SDK-Knowledge-Base/Introducing-eRPC/ta-p/1099139>

-
- [20] Object Management Group, „DDS for Extremely Resource Constrained Environments (DDS-XRCE), Version 1.0“. Zugegriffen: 30. September 2026. [Online]. Verfügbar unter: <https://www.omg.org/spec/DDS-XRCE/1.0/About-DDS-XRCE>
 - [21] micro-ROS, „micro-ROS-Agent: ROS 2 Package Using Micro XRCE-DDS Agent“. Zugegriffen: 2. Oktober 2026. [Online]. Verfügbar unter: <https://github.com/micro-ROS/micro-ROS-Agent>
 - [22] T. Bray, „The JavaScript Object Notation (JSON) Data Interchange Format“, Request for Comments RFC8259, Dez. 2017. doi: [10.17487/RFC8259](https://doi.org/10.17487/RFC8259).
 - [23] „MessagePack: It's like JSON. but Fast and Small.“. Zugegriffen: 4. August 2026. [Online]. Verfügbar unter: <https://msgpack.org/>
 - [24] C. Bormann und P. E. Hoffman, „Concise Binary Object Representation (CBOR)“, Request for Comments RFC8949, Dez. 2020. doi: [10.17487/RFC8949](https://doi.org/10.17487/RFC8949).
 - [25] „CBOR — Concise Binary Object Representation“. Zugegriffen: 7. August 2026. [Online]. Verfügbar unter: <https://cbor.io/>
 - [26] „Protocol Buffers“. Zugegriffen: 4. August 2026. [Online]. Verfügbar unter: <https://protobuf.dev/>
 - [27] „Nanopb — Zephyr Project Documentation“. Zugegriffen: 4. August 2026. [Online]. Verfügbar unter: <https://docs.zephyrproject.org/latest/services/serialization/nanopb.html>
 - [28] „Firmata/Arduino“. Zugegriffen: 4. August 2026. [Online]. Verfügbar unter: <https://github.com/firmata/arduino>
 - [29] „MQTT - The Standard for IoT Messaging“. Zugegriffen: 4. August 2026. [Online]. Verfügbar unter: <https://mqtt.org/>
 - [30] International Organization for Standardization, „Road Vehicles — Diagnostic Communication over Controller Area Network (DoCAN) — Part 2: Transport Protocol and Network Layer Services“, Genf, Norm ISO 15765-2:2024, Apr. 2024.
 - [31] CAN in Automation, „CANopen Application Layer and Communication Profile“, Spezifikation CiA 301 Version 4.2.0, Feb. 2011. Zugegriffen: 2. Oktober 2026. [Online]. Verfügbar unter: <https://www.can-cia.org/cia-groups/technical-documents>
 - [32] Z. Shelby, K. Hartke, und C. Bormann, „The Constrained Application Protocol (CoAP)“, Request for Comments RFC7252, Juni 2014. doi: [10.17487/RFC7252](https://doi.org/10.17487/RFC7252).
 - [33] C. Bormann und Z. Shelby, „Block-Wise Transfers in the Constrained Application Protocol (CoAP)“, Request for Comments RFC7959, Aug. 2016. doi: [10.17487/RFC7959](https://doi.org/10.17487/RFC7959).
 - [34] Nextion, „Instruction Set“. Zugegriffen: 2. Oktober 2026. [Online]. Verfügbar unter: <https://nextion.tech/instruction-set/>
 - [35] „leJOS / EV3 Wiki / UART Sensor Protocol“. Zugegriffen: 28. September 2026. [Online]. Verfügbar unter: <https://sourceforge.net/p/lejos/wiki/UART%20Sensor%20Protocol/>
 - [36] „PUPRemote Documentation — Anton's Mindstorms Docs Documentation“. Zugegriffen: 7. August 2026. [Online]. Verfügbar unter: <https://docs.antonsmindstorms.com/en/latest/Software/PUPRemote/docs/index.html>
 - [37] LEGO Education, „LEGO Education SPIKE Prime Set“. Zugegriffen: 2. Oktober 2026. [Online]. Verfügbar unter: <https://education.lego.com/de-de/products/lego-education-spike-prime-set/45678/>
 - [38] STMicroelectronics, „STM32F413xG/STM32F413xH Datasheet: ARM Cortex-M4 32b MCU+FPU, 125 DMIPS, up to 1.5 MB Flash, 320 KB RAM“, DS11581 Rev. 4, 2018. [Online]. Verfügbar unter: <https://www.st.com/resource/en/datasheet/stm32f413cg.pdf>
 - [39] „Lego::Lab Karlsruhe“. Zugegriffen: 7. August 2026. [Online]. Verfügbar unter: <https://legolab-karlsruhe.de/mindstorms/spikeprime>
 - [40] ILI Technology Corp., „ILI9341: A-Si TFT LCD Single Chip Driver, 240rgbx320 Resolution and 262K Color“, Version 1.11, 2011. [Online]. Verfügbar unter: <https://cdn-shop.adafruit.com/datasheets/ILI9341.pdf>

- [41] FocalTech Systems, „FT6x06: Self-capacitive Touch Panel Controller“, Version 0.1, 2014. [Online]. Verfügbar unter: https://cdn-shop.adafruit.com/datasheets/FT6x06+Datasheet_V0.1_Preliminary_20120723.pdf
- [42] „LVGL — Light and Versatile Embedded UI Ecosystem“. Zugegriffen: 7. August 2026. [Online]. Verfügbar unter: <https://lvgl.io/>
- [43] „LMS-ESP32 v2.0: The Clever Wi-Fi and Gamepad Board for LEGO SPIKE“. Zugegriffen: 7. August 2026. [Online]. Verfügbar unter: <https://www.antonsmindstorms.com/product/powerful-lms-esp32-board-for-spike-and-mindstorms/>
- [44] R. B. Miller, „Response Time in Man-Computer Conversational Transactions“, in *Proceedings of the December 9–11, 1968, Fall Joint Computer Conference, Part I (AFIPS '68)*, San Francisco, California: ACM Press, 1968, S. 267. doi: 10.1145/1476589.1476628.
- [45] S. K. Card, T. P. Moran, und A. Newell, *The Psychology of Human-Computer Interaction*. Hillsdale, NJ: L. Erlbaum Associates, 1983. Zugegriffen: 28. September 2026. [Online]. Verfügbar unter: <http://archive.org/details/psychologyofhuma00stua>
- [46] L. Valk, „Pybricks“. Zugegriffen: 28. September 2026. [Online]. Verfügbar unter: <https://pybricks.com/>
- [47] Bluetooth SIG, „Core Specification 5.4“. Zugegriffen: 28. September 2026. [Online]. Verfügbar unter: <https://www.bluetooth.com/specifications/specs/core-specification-5-4/>
- [48] P. Levis und D. Culler, „Maté: A Tiny Virtual Machine for Sensor Networks“, in *Proceedings of the 10th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS-X)*, San Jose, CA, USA: ACM, Okt. 2002, S. 85–95. doi: 10.1145/605397.605407.
- [49] D. C. Schmidt, M. Stal, H. Rohnert, und F. Buschmann, *Pattern-Oriented Software Architecture: Patterns for Concurrent and Networked Objects, Volume 2*. Chichester: Wiley, 2000.
- [50] A. S. Tanenbaum und D. Wetherall, *Computernetzwerke*, 5., aktualisierte Auflage. in Always learning. München [u.a.]: Pearson, 2012.
- [51] R. Fielding, M. Nottingham, und J. Reschke, „HTTP Semantics“, Request for Comments RFC9110, Juni 2022. doi: 10.17487/RFC9110.
- [52] P. T. Eugster, P. A. Felber, R. Guerraoui, und A.-M. Kermarrec, „The Many Faces of Publish/Subscribe“, *ACM Computing Surveys*, Bd. 35, Nr. 2, S. 114–131, Juni 2003, doi: 10.1145/857076.857078.
- [53] O. Yigit, „Hash Functions“. Zugegriffen: 28. September 2026. [Online]. Verfügbar unter: <http://www.cse.yorku.ca/~oz/hash.html>
- [54] N. Frykholm, „Managing Decoupling Part 4 – The ID Lookup Table“. Zugegriffen: 28. September 2026. [Online]. Verfügbar unter: <https://bitsquid.blogspot.com/2011/09/managing-decoupling-part-4-id-lookup.html>
- [55] „LVGL Micropython“. Zugegriffen: 28. September 2026. [Online]. Verfügbar unter: https://github.com/lvgl/lv_binding_micropython
- [56] A. Dunkels, O. Schmidt, T. Voigt, und M. Ali, „Protothreads: Simplifying Event-Driven Programming of Memory-Constrained Embedded Systems“, in *Proceedings of the 4th International Conference on Embedded Networked Sensor Systems (SenSys '06)*, Boulder, CO, USA: ACM, Okt. 2006, S. 29–42. doi: 10.1145/1182807.1182811.
- [57] „Timer (lv_timer)“. Zugegriffen: 28. September 2026. [Online]. Verfügbar unter: <https://lvgl.io/docs/open/main-modules/timer>
- [58] „Tools – General Purpose Tools — Pybricks v4.1.0b1 Documentation“. Zugegriffen: 28. September 2026. [Online]. Verfügbar unter: <https://docs.pybricks.com/en/latest/tools/index.html>

Anhang

Quelltext und Testumgebung

Der vollständige Quelltext liegt unter github.com/mat-mv/legolab-tft-display. Die Angaben in Kapitel 6 und Kapitel 7 beziehen sich, sofern nicht anders vermerkt, auf den Stand, der dort mit `stand-projektarbeit` markiert ist. Spätere Stände setzen die Erweiterungen aus Kapitel 8 um und verändern dabei das Protokoll. Die folgenden Ordner sind für das Nachvollziehen der Arbeit die wichtigsten.

Tabelle XVI: Aufbau des Quelltextes

Pfad	Inhalt
<code>src/middleware/</code>	Transport- und Kommunikationsschicht, Registratur, Codec
<code>src/display/</code>	Objektmodell, getrennt nach <code>protocol</code> , <code>client</code> und <code>server</code>
<code>src/hub/</code>	Beispielprogramm der Steuereinheit
<code>src/tests/</code>	Testumgebung aus Abschnitt 7.2
<code>src/tools/</code>	Bundler und Auswertung der Messungen
<code>src/measurements/</code>	Rohdaten aller Messläufe
<code>pcb/</code>	Entwurf der Adapterplatine

Die Testumgebung wird mit `python3 tests/test_all.py` und `tests/test_events.py` ausgeführt und benötigt weder Hardware noch zusätzliche Pakete. `tests/fake_env.py` enthält die zweite Umsetzung der Interceptor-Schnittstelle sowie die Attrappen für Anzeigebibliothek und Laufzeitumgebungen. Spätere Stände ergänzen `tests/test_poll.py` für die Erweiterungen aus Abschnitt 8.3.1 und Abschnitt 8.3.3 und `tests/test_bindung.py`, das alle Testläufe über eine zweite Busanbindung wiederholt, wie Abschnitt 8.3.4 beschreibt.

Für das Nachvollziehen der Messungen ist die Kopie der Bibliothek PUPRemote zu beachten. Im Stand `stand-projektarbeit` liegen zwei Kopien der Version 1.6 bei, `src/hub/pupremote.py` und `src/vendor/pupremote/pupremote.py`, deren Funktion `call` ohne Angabe 0 ms wartet. Auf dem Hub lief dagegen eine Fassung, in der dieser Wert nach Abschnitt 6.3 auf 100 ms angehoben ist. Spätere Stände enthalten diese Fassung unter `src/hub/pupremote.py` und dieselbe Datei mit Paketpfaden unter `src/vendor/pupremote/pupremote.py`. Seit die Middleware die Wartezeit ausdrücklich übergibt, hängen die Messwerte nicht mehr davon ab, welche Kopie installiert ist.

Nach dem Stand `stand-projektarbeit` ist die Middleware ohne die Anteile von Display und Bus in ein eigenes Repository ausgelagert, github.com/KIngGrim/micro-rpc. Das Repository dieser Arbeit bindet es als Submodul ein.

```

from pybricks.parameters import Button, Color, Direction, Port, Side, Stop
from pupremote import PUPRemoteHub
from pybricks.tools import wait, Stopwatch

from pybricks_bundle import *

p = PUPRemoteHub(Port.B)

interceptor = PyBricksInterceptor(p)
interceptor.register_interceptor()

display = Display(interceptor)

display.clear()

info = display.display_info()
print(info)

# --- Aufbau -----
# Display elements live on a screen, so one has to exist and be shown before
# anything can be placed on it. The second screen is created but not shown.
main_screen = display.create_screen(set_active=True, name="main")
menu_screen = display.create_screen(name="menu")

counter_label = main_screen.create_label(10, 10, 230, 20, "Counter: 0")
menu_label = menu_screen.create_label(10, 10, 230, 20, "Menue")

# Captions without umlauts: the default font of the display library carries a
# reduced set of glyphs, and a missing one is drawn as a placeholder box.
to_menu = main_screen.create_button(10, 60, 100, 40, "Menue")
to_main = menu_screen.create_button(10, 60, 100, 40, "Zurueck")

# --- Bindings -----
# Declared once, here. From now on a press switches the screen on the display
# unit itself. Each of these two calls costs the usual number of transfers -
# once. Every switch afterwards costs none.
to_menu.on_press(ACT_SHOW_SCREEN, menu_screen)
to_main.on_press(ACT_SHOW_SCREEN, main_screen)

print("bindings declared")

# --- Ablauf -----
counter = 0

while True:
    counter_label.set_text("Counter: {}".format(counter))
    counter += 1

    # The screen has already changed by the time this query happens - the
    # binding did that. What arrives here is the notification, not the trigger.
    # Polling both buttons costs one full call each; leaving it out would not
    # affect the switching at all.
    if to_menu.is_button_pressed():
        print("menu opened")

    if to_main.is_button_pressed():
        print("back on the main screen")

    wait(1000)

```

Listing 9: Beispielprogramm auf dem Hub

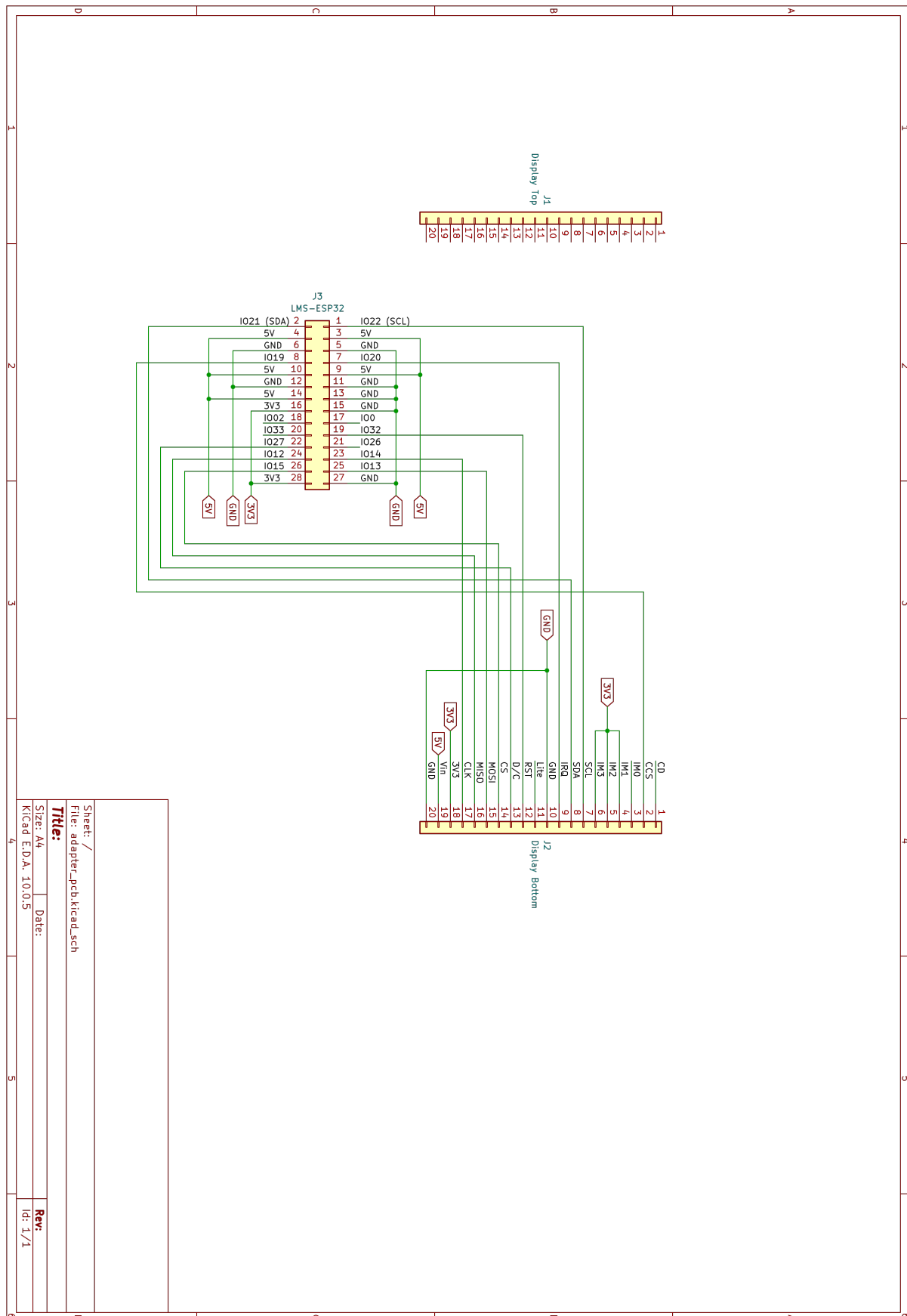


Abbildung 9: Schaltplan der Adapterplatine